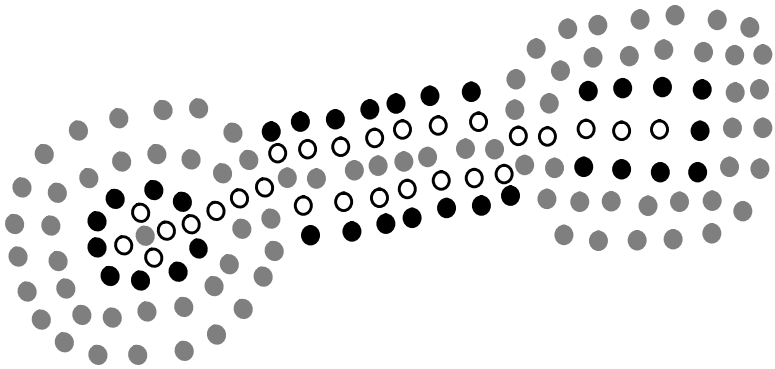
Chapter

6 Stacks, Queues, and Deques



Contents

6.1 Stacks 229
6.1.1 The Stack Abstract Data Type 230
6.1.2 Simple Array-Based Stack Implementation 231
6.1.3 Reversing Data Using a Stack 235
6.1.4 Matching Parentheses and HTML Tags 236
6.2 Queues 239
6.2.1 The Queue Abstract Data Type 240
6.2.2 Array-Based Queue Implementation 241
6.3 Double-Ended Queues 247
6.3.1 The Deque Abstract Data Type 247
6.3.2 Implementing a Deque with a Circular Array 248
6.3.3 Deques in the Python Collections Module 249
6.4 Exercises 250

6.1 Stacks

A ***stack*** is a collection of objects that are inserted and removed according to the ***last-in, first-out*** (***LIFO***) principle. A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called "top" of the stack). The name "stack" is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser. In this case, the fundamental operations involve the "pushing" and "popping" of plates on the stack. When we need a new plate from the dispenser, we "pop" the top plate off the stack, and when we add a plate, we "push" it down on the stack to become the new top plate. Perhaps an even more amusing example is a PEZ® candy dispenser, which stores mint candies in a spring-loaded container that "pops" out the topmost candy in the stack when the top of the dispenser is lifted (see Figure 6.1). Stacks are a fundamental data structure. They are used in many applications, including the following.

Example 6.1: *Internet Web browsers store the addresses of recently visited sites in a stack. Each time a user visits a new site, that site's address is "pushed" onto the stack of addresses. The browser then allows the user to "pop" back to previously visited sites using the "back" button.*

Example 6.2: *Text editors usually provide an "undo" mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.*

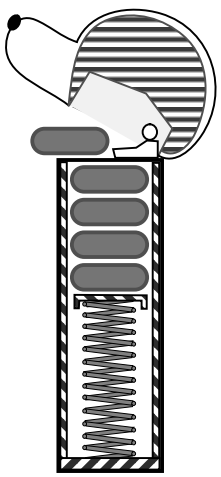


Figure 6.1: A schematic drawing of a PEZ® dispenser; a physical implementation of the stack ADT. (PEZ® is a registered trademark of PEZ Candy, Inc.)

6.1.1 The Stack Abstract Data Type

Stacks are the simplest of all data structures, yet they are also among the most important. They are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms. Formally, a stack is an abstract data type (ADT) such that an instance S supports the following two methods:

S.push(e): Add element e to the top of stack S.

S.pop( ): Remove and return the top element from the stack S; an error occurs if the stack is empty.

Additionally, let us define the following accessor methods for convenience:

S.top( ): Return a reference to the top element of stack S, without removing it; an error occurs if the stack is empty.

S.is_empty( ): Return True if stack S does not contain any elements.

len(S): Return the number of elements in stack S; in Python, we implement this with the special method __len__.

By convention, we assume that a newly created stack is empty, and that there is no a priori bound on the capacity of the stack. Elements added to the stack can have arbitrary type.

Example 6.3: *The following table shows a series of stack operations and their effects on an initially empty stack S of integers.*

Operation	Return Value	Stack Contents
S.push(5)	–	[5]
S.push(3)	–	[5, 3]
len(S)	2	[5, 3]
S.pop( )	3	[5]
S.is_empty( )	False	[5]
S.pop( )	5	[ ]
S.is_empty( )	True	[ ]
S.pop( )	"error"	[ ]
S.push(7)	–	[7]
S.push(9)	–	[7, 9]
S.top( )	9	[7, 9]
S.push(4)	–	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop( )	4	[7, 9]
S.push(6)	–	[7, 9, 6]
S.push(8)	–	[7, 9, 6, 8]
S.pop( )	8	[7, 9, 6]

6.1.2 Simple Array-Based Stack Implementation

We can implement a stack quite easily by storing its elements in a Python list. The list class already supports adding an element to the end with the append method, and removing the last element with the pop method, so it is natural to align the top of the stack at the end of the list, as shown in Figure 6.2.

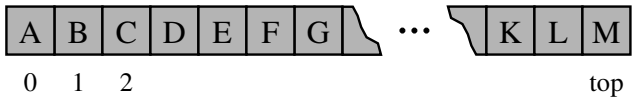


Figure 6.2: Implementing a stack with a Python list, storing the top element in the rightmost cell.

Although a programmer could directly use the list class in place of a formal stack class, lists also include behaviors (e.g., adding or removing elements from arbitrary positions) that would break the abstraction that the stack ADT represents. Also, the terminology used by the list class does not precisely align with traditional nomenclature for a stack ADT, in particular the distinction between append and push. Instead, we demonstrate how to use a list for internal storage while providing a public interface consistent with a stack.

The Adapter Pattern

The ***adapter*** design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface. One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable. By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way. In the context of the stack ADT, we can adapt Python's list class using the correspondences shown in Table 6.1.

Stack Method	*Realization with Python list*
S.push(e)	L.append(e)
S.pop( )	L.pop( )
S.top( )	L[−1]
S.is_empty( )	len(L) == 0
len(S)	len(L)

Table 6.1: Realization of a stack S as an adaptation of a Python list L.

Implementing a Stack Using a Python List

We use the adapter design pattern to define an ArrayStack class that uses an underlying Python list for storage. (We choose the name ArrayStack to emphasize that the underlying storage is inherently array based.) One question that remains is what our code should do if a user calls pop or top when the stack is empty. Our ADT suggests that an error occurs, but we must decide what type of error. When pop is called on an empty Python list, it formally raises an IndexError, as lists are index-based sequences. That choice does not seem appropriate for a stack, since there is no assumption of indices. Instead, we can define a new exception class that is more appropriate. Code Fragment 6.1 defines such an Empty class as a trivial subclass of the Python Exception class.

```
class Empty(Exception):
  """Error attempting to access an element from an empty container."""
  pass
```

Code Fragment 6.1: Definition for an Empty exception class.

The formal definition for our ArrayStack class is given in Code Fragment 6.2. The constructor establishes the member self._data as an initially empty Python list, for internal storage. The rest of the public stack behaviors are implemented, using the corresponding adaptation that was outlined in Table 6.1.

Example Usage

Below, we present an example of the use of our ArrayStack class, mirroring the operations at the beginning of Example 6.3 on page 230.

```
S = ArrayStack( )                 # contents: [ ]
S.push(5)                         # contents: [5]
S.push(3)                         # contents: [5, 3]
print(len(S))                     # contents: [5, 3];        outputs 2
print(S.pop( ))                   # contents: [5];           outputs 3
print(S.is_empty( ))              # contents: [5];           outputs False
print(S.pop( ))                   # contents: [ ];           outputs 5
print(S.is_empty( ))              # contents: [ ];           outputs True
S.push(7)                         # contents: [7]
S.push(9)                         # contents: [7, 9]
print(S.top( ))                   # contents: [7, 9];        outputs 9
S.push(4)                         # contents: [7, 9, 4]
print(len(S))                     # contents: [7, 9, 4];     outputs 3
print(S.pop( ))                   # contents: [7, 9];        outputs 4
S.push(6)                         # contents: [7, 9, 6]
```

```
1  class ArrayStack:
2    """LIFO Stack implementation using a Python list as underlying storage."""
3
4    def __init__(self):
5      """Create an empty stack."""
6      self._data = [ ]                       # nonpublic list instance
7
8    def __len__(self):
9      """Return the number of elements in the stack."""
10     return len(self._data)
11
12   def is_empty(self):
13     """Return True if the stack is empty."""
14     return len(self._data) == 0
15
16   def push(self, e):
17     """Add element e to the top of the stack."""
18     self._data.append(e)                   # new item stored at end of list
19
20   def top(self):
21     """Return (but do not remove) the element at the top of the stack.
22
23     Raise Empty exception if the stack is empty.
24     """
25     if self.is_empty( ):
26       raise Empty('Stack is empty')
27     return self._data[-1]                  # the last item in the list
28
29   def pop(self):
30     """Remove and return the element from the top of the stack (i.e., LIFO).
31
32     Raise Empty exception if the stack is empty.
33     """
34     if self.is_empty( ):
35       raise Empty('Stack is empty')
36     return self._data.pop( )               # remove last item from list
```

Code Fragment 6.2: Implementing a stack using a Python list as storage.

Analyzing the Array-Based Stack Implementation

Table 6.2 shows the running times for our ArrayStack methods. The analysis directly mirrors the analysis of the list class given in Section 5.3. The implementations for top, is_empty, and len use constant time in the worst case. The $O(1)$ time for push and pop are ***amortized*** bounds (see Section 5.3.2); a typical call to either of these methods uses constant time, but there is occasionally an $O(n)$-time worst case, where n is the current number of elements in the stack, when an operation causes the list to resize its internal array. The space usage for a stack is $O(n)$.

Operation	Running Time
S.push(e)	$O(1)^*$
S.pop( )	$O(1)^*$
S.top( )	$O(1)$
S.is_empty( )	$O(1)$
len(S)	$O(1)$

*amortized

Table 6.2: Performance of our array-based stack implementation. The bounds for push and pop are amortized due to similar bounds for the list class. The space usage is $O(n)$, where n is the current number of elements in the stack.

Avoiding Amortization by Reserving Capacity

In some contexts, there may be additional knowledge that suggests a maximum size that a stack will reach. Our implementation of ArrayStack from Code Fragment 6.2 begins with an empty list and expands as needed. In the analysis of lists from Section 5.4.1, we emphasized that it is more efficient in practice to construct a list with initial length n than it is to start with an empty list and append n items (even though both approaches run in $O(n)$ time).

As an alternate model for a stack, we might wish for the constructor to accept a parameter specifying the maximum capacity of a stack and to initialize the _data member to a list of that length. Implementing such a model requires significant changes relative to Code Fragment 6.2. The size of the stack would no longer be synonymous with the length of the list, and pushes and pops of the stack would not require changing the length of the list. Instead, we suggest maintaining a separate integer as an instance variable that denotes the current number of elements in the stack. Details of such an implementation are left as Exercise C-6.17.

6.1.3 Reversing Data Using a Stack

As a consequence of the LIFO protocol, a stack can be used as a general tool to reverse a data sequence. For example, if the values 1, 2, and 3 are pushed onto a stack in that order, they will be popped from the stack in the order 3, 2, and then 1.

This idea can be applied in a variety of settings. For example, we might wish to print lines of a file in reverse order in order to display a data set in decreasing order rather than increasing order. This can be accomplished by reading each line and pushing it onto a stack, and then writing the lines in the order they are popped. An implementation of such a process is given in Code Fragment 6.3.

```
1  def reverse_file(filename):
2    """Overwrite given file with its contents line-by-line reversed."""
3    S = ArrayStack( )
4    original = open(filename)
5    for line in original:
6      S.push(line.rstrip('\n'))        # we will re-insert newlines when writing
7    original.close( )
8
9    # now we overwrite with contents in LIFO order
10   output = open(filename, 'w')     # reopening file overwrites original
11   while not S.is_empty( ):
12     output.write(S.pop( ) + '\n')  # re-insert newline characters
13   output.close( )
```

Code Fragment 6.3: A function that reverses the order of lines in a file.

One technical detail worth noting is that we intentionally strip trailing newlines from lines as they are read, and then re-insert newlines after each line when writing the resulting file. Our reason for doing this is to handle a special case in which the original file does not have a trailing newline for the final line. If we exactly echoed the lines read from the file in reverse order, then the original last line would be followed (without newline) by the original second-to-last line. In our implementation, we ensure that there will be a separating newline in the result.

The idea of using a stack to reverse a data set can be applied to other types of sequences. For example, Exercise R-6.5 explores the use of a stack to provide yet another solution for reversing the contents of a Python list (a recursive solution for this goal was discussed in Section 4.4.1). A more challenging task is to reverse the order in which elements are stored within a stack. If we were to move them from one stack to another, they would be reversed, but if we were to then replace them into the original stack, they would be reversed again, thereby reverting to their original order. Exercise C-6.18 explores a solution for this task.

6.1.4 Matching Parentheses and HTML Tags

In this subsection, we explore two related applications of stacks, both of which involve testing for pairs of matching delimiters. In our first application, we consider arithmetic expressions that may contain various pairs of grouping symbols, such as

- Parentheses: "(" and ")"
- Braces: "{" and "}"
- Brackets: "[" and "]"

Each opening symbol must match its corresponding closing symbol. For example, a left bracket, "[," must match a corresponding right bracket, "]," as in the expression [(5+x)-(y+z)]. The following examples further illustrate this concept:

- Correct: ()(()){([()])}
- Correct: ((()(()){([()])}))
- Incorrect:)(()){([()])}
- Incorrect: ({[])}
- Incorrect: (

We leave the precise definition of a matching group of symbols to Exercise R-6.6.

An Algorithm for Matching Delimiters

An important task when processing arithmetic expressions is to make sure their delimiting symbols match up correctly. Code Fragment 6.4 presents a Python implementation of such an algorithm. A discussion of the code follows.

```
1  def is_matched(expr):
2    """Return True if all delimiters are properly match; False otherwise."""
3    lefty = '({['                                  # opening delimiters
4    righty = ')}]'                                 # respective closing delims
5    S = ArrayStack()
6    for c in expr:
7      if c in lefty:
8        S.push(c)                                  # push left delimiter on stack
9      elif c in righty:
10       if S.is_empty():
11         return False                             # nothing to match with
12       if righty.index(c) != lefty.index(S.pop()):
13         return False                             # mismatched
14   return S.is_empty( )                           # were all symbols matched?
```

Code Fragment 6.4: Function for matching delimiters in an arithmetic expression.

We assume the input is a sequence of characters, such as `'[(5+x)-(y+z)]'`. We perform a left-to-right scan of the original sequence, using a stack S to facilitate the matching of grouping symbols. Each time we encounter an opening symbol, we push that symbol onto S, and each time we encounter a closing symbol, we pop a symbol from the stack S (assuming S is not empty), and check that these two symbols form a valid pair. If we reach the end of the expression and the stack is empty, then the original expression was properly matched. Otherwise, there must be an opening delimiter on the stack without a matching symbol.

If the length of the original expression is n, the algorithm will make at most n calls to push and n calls to pop. Those calls run in a total of $O(n)$ time, even considering the amortized nature of the $O(1)$ time bound for those methods. Given that our selection of possible delimiters, ({[, has constant size, auxiliary tests such as c in lefty and righty.index(c) each run in $O(1)$ time. Combining these operations, the matching algorithm on a sequence of length n runs in $O(n)$ time.

Matching Tags in a Markup Language

Another application of matching delimiters is in the validation of markup languages such as HTML or XML. HTML is the standard format for hyperlinked documents on the Internet and XML is an extensible markup language used for a variety of structured data sets. We show a sample HTML document and a possible rendering in Figure 6.3.

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine.  The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 6.3: Illustrating HTML tags. (a) An HTML document; (b) its rendering.

In an HTML document, portions of text are delimited by ***HTML tags***. A simple opening HTML tag has the form "<name>" and the corresponding closing tag has the form "</name>". For example, we see the <body> tag on the first line of Figure 6.3(a), and the matching </body> tag at the close of that document. Other commonly used HTML tags that are used in this example include:

- body: document body
- h1: section header
- center: center justify
- p: paragraph
- ol: numbered (ordered) list
- li: list item

Ideally, an HTML document should have matching tags, although most browsers tolerate a certain number of mismatching tags. In Code Fragment 6.5, we give a Python function that matches tags in a string representing an HTML document. We make a left-to-right pass through the raw string, using index j to track our progress and the find method of the str class to locate the '<' and '>' characters that define the tags. Opening tags are pushed onto the stack, and matched against closing tags as they are popped from the stack, just as we did when matching delimiters in Code Fragment 6.4. By similar analysis, this algorithm runs in $O(n)$ time, where n is the number of characters in the raw HTML source.

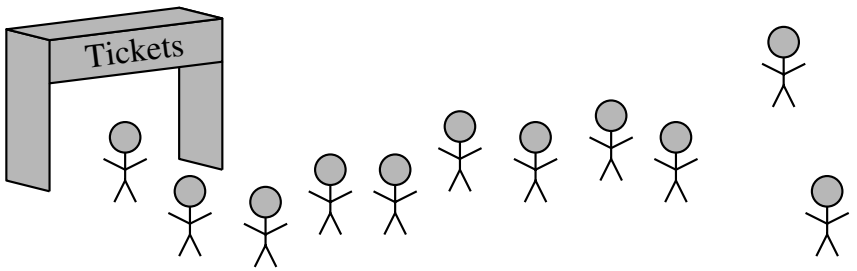
```
1  def is_matched_html(raw):
2    """Return True if all HTML tags are properly match; False otherwise."""
3    S = ArrayStack( )
4    j = raw.find('<')                        # find first '<' character (if any)
5    while j != −1:
6      k = raw.find('>', j+1)                 # find next '>' character
7      if k == −1:
8        return False                         # invalid tag
9      tag = raw[j+1:k]                       # strip away < >
10     if not tag.startswith('/'):            # this is opening tag
11       S.push(tag)
12     else:                                  # this is closing tag
13       if S.is_empty( ):
14         return False                       # nothing to match with
15       if tag[1:] != S.pop( ):
16         return False                       # mismatched delimiter
17     j = raw.find('<', k+1)                 # find next '<' character (if any)
18   return S.is_empty( )                     # were all opening tags matched?
```

Code Fragment 6.5: Function for testing if an HTML document has matching tags.

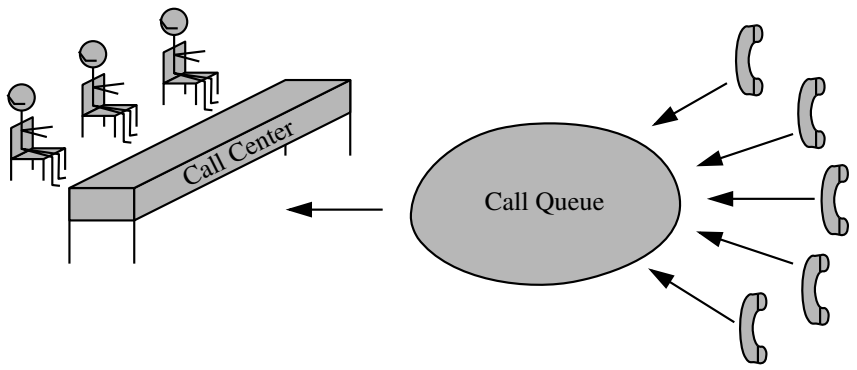
6.2 Queues

Another fundamental data structure is the ***queue***. It is a close "cousin" of the stack, as a queue is a collection of objects that are inserted and removed according to the ***first-in, first-out*** (***FIFO***) principle. That is, elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.

We usually say that elements enter a queue at the back and are removed from the front. A metaphor for this terminology is a line of people waiting to get on an amusement park ride. People waiting for such a ride enter at the back of the line and get on the ride from the front of the line. There are many other applications of queues (see Figure 6.4). Stores, theaters, reservation centers, and other similar services typically process customer requests according to the FIFO principle. A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant. FIFO queues are also used by many computing devices, such as a networked printer, or a Web server responding to requests.



(a)



(b)

Figure 6.4: Real-world examples of a first-in, first-out queue. (a) People waiting in line to purchase tickets; (b) phone calls being routed to a customer service center.

6.2.1 The Queue Abstract Data Type

Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the ***first*** element in the queue, and element insertion is restricted to the back of the sequence. This restriction enforces the rule that items are inserted and deleted in a queue according to the first-in, first-out (FIFO) principle. The ***queue*** abstract data type (ADT) supports the following two fundamental methods for a queue Q:

Q.enqueue(e): Add element e to the back of queue Q.

Q.dequeue( ): Remove and return the first element from queue Q; an error occurs if the queue is empty.

The queue ADT also includes the following supporting methods (with first being analogous to the stack's top method):

Q.first( ): Return a reference to the element at the front of queue Q, without removing it; an error occurs if the queue is empty.

Q.is_empty( ): Return True if queue Q does not contain any elements.

len(Q): Return the number of elements in queue Q; in Python, we implement this with the special method __len__.

By convention, we assume that a newly created queue is empty, and that there is no a priori bound on the capacity of the queue. Elements added to the queue can have arbitrary type.

Example 6.4: *The following table shows a series of queue operations and their effects on an initially empty queue Q of integers.*

Operation	Return Value	first ← Q ← last
Q.enqueue(5)	–	[5]
Q.enqueue(3)	–	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue( )	5	[3]
Q.is_empty( )	False	[3]
Q.dequeue( )	3	[ ]
Q.is_empty( )	True	[ ]
Q.dequeue( )	"error"	[ ]
Q.enqueue(7)	–	[7]
Q.enqueue(9)	–	[7, 9]
Q.first( )	7	[7, 9]
Q.enqueue(4)	–	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue( )	7	[9, 4]

6.2.2 Array-Based Queue Implementation

For the stack ADT, we created a very simple adapter class that used a Python list as the underlying storage. It may be very tempting to use a similar approach for supporting the queue ADT. We could enqueue element e by calling append(e) to add it to the end of the list. We could use the syntax pop(0), as opposed to pop( ), to intentionally remove the ***first*** element from the list when dequeuing.

As easy as this would be to implement, it is tragically inefficient. As we discussed in Section 5.4.1, when pop is called on a list with a non-default index, a loop is executed to shift all elements beyond the specified index to the left, so as to fill the hole in the sequence caused by the pop. Therefore, a call to pop(0) always causes the worst-case behavior of $\Theta(n)$ time.

We can improve on the above strategy by avoiding the call to pop(0) entirely. We can replace the dequeued entry in the array with a reference to None, and maintain an explicit variable f to store the index of the element that is currently at the front of the queue. Such an algorithm for dequeue would run in $O(1)$ time. After several dequeue operations, this approach might lead to the configuration portrayed in Figure 6.5.

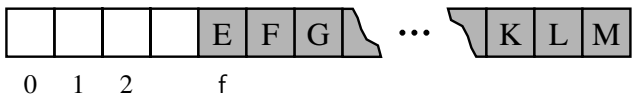


Figure 6.5: Allowing the front of the queue to drift away from index 0.

Unfortunately, there remains a drawback to the revised approach. In the case of a stack, the length of the list was precisely equal to the size of the stack (even if the underlying array for the list was slightly larger). With the queue design that we are considering, the situation is worse. We can build a queue that has relatively few elements, yet which are stored in an arbitrarily large list. This occurs, for example, if we repeatedly enqueue a new element and then dequeue another (allowing the front to drift rightward). Over time, the size of the underlying list would grow to $O(m)$ where m is the ***total*** number of enqueue operations since the creation of the queue, rather than the current number of elements in the queue.

This design would have detrimental consequences in applications in which queues have relatively modest size, but which are used for long periods of time. For example, the wait-list for a restaurant might never have more than 30 entries at one time, but over the course of a day (or a week), the overall number of entries would be significantly larger.

Using an Array Circularly

In developing a more robust queue implementation, we allow the front of the queue to drift rightward, and we allow the contents of the queue to "wrap around" the end of an underlying array. We assume that our underlying array has fixed length N that is greater that the actual number of elements in the queue. New elements are enqueued toward the "end" of the current queue, progressing from the front to index $N-1$ and continuing at index 0, then 1. Figure 6.6 illustrates such a queue with first element E and last element M.

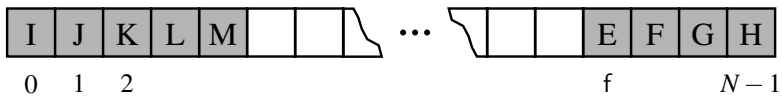


Figure 6.6: Modeling a queue with a circular array that wraps around the end.

Implementing this circular view is not difficult. When we dequeue an element and want to "advance" the front index, we use the arithmetic f = (f + 1) % N. Recall that the % operator in Python denotes the ***modulo*** operator, which is computed by taking the remainder after an integral division. For example, 14 divided by 3 has a quotient of 4 with remainder 2, that is, $\frac{14}{3} = 4\frac{2}{3}$. So in Python, 14 // 3 evaluates to the quotient 4, while 14 % 3 evaluates to the remainder 2. The modulo operator is ideal for treating an array circularly. As a concrete example, if we have a list of length 10, and a front index 7, we can advance the front by formally computing (7+1) % 10, which is simply 8, as 8 divided by 10 is 0 with a remainder of 8. Similarly, advancing index 8 results in index 9. But when we advance from index 9 (the last one in the array), we compute (9+1) % 10, which evaluates to index 0 (as 10 divided by 10 has a remainder of zero).

A Python Queue Implementation

A complete implementation of a queue ADT using a Python list in circular fashion is presented in Code Fragments 6.6 and 6.7. Internally, the queue class maintains the following three instance variables:

_data: is a reference to a list instance with a fixed capacity.

_size: is an integer representing the current number of elements stored in the queue (as opposed to the length of the _data list).

_front: is an integer that represents the index within _data of the first element of the queue (assuming the queue is not empty).

We initially reserve a list of moderate size for storing data, although the queue formally has size zero. As a technicality, we initialize the _front index to zero.

When front or dequeue are called with no elements in the queue, we raise an instance of the Empty exception, defined in Code Fragment 6.1 for our stack.

```
1  class ArrayQueue:
2    """FIFO queue implementation using a Python list as underlying storage."""
3    DEFAULT_CAPACITY = 10          # moderate capacity for all new queues
4
5    def __init__(self):
6      """Create an empty queue."""
7      self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
8      self._size = 0
9      self._front = 0
10
11   def __len__(self):
12     """Return the number of elements in the queue."""
13     return self._size
14
15   def is_empty(self):
16     """Return True if the queue is empty."""
17     return self._size == 0
18
19   def first(self):
20     """Return (but do not remove) the element at the front of the queue.
21
22     Raise Empty exception if the queue is empty.
23     """
24     if self.is_empty():
25       raise Empty('Queue is empty')
26     return self._data[self._front]
27
28   def dequeue(self):
29     """Remove and return the first element of the queue (i.e., FIFO).
30
31     Raise Empty exception if the queue is empty.
32     """
33     if self.is_empty():
34       raise Empty('Queue is empty')
35     answer = self._data[self._front]
36     self._data[self._front] = None                    # help garbage collection
37     self._front = (self._front + 1) % len(self._data)
38     self._size -= 1
39     return answer
```

Code Fragment 6.6: Array-based implementation of a queue (continued in Code Fragment 6.7).

```
40   def enqueue(self, e):
41     """Add an element to the back of queue."""
42     if self._size == len(self._data):
43       self._resize(2 * len(self.data))          # double the array size
44     avail = (self._front + self._size) % len(self._data)
45     self._data[avail] = e
46     self._size += 1
47
48   def _resize(self, cap):                       # we assume cap >= len(self)
49     """Resize to a new list of capacity >= len(self)."""
50     old = self._data                            # keep track of existing list
51     self._data = [None] * cap                   # allocate list with new capacity
52     walk = self._front
53     for k in range(self._size):                 # only consider existing elements
54       self._data[k] = old[walk]                 # intentionally shift indices
55       walk = (1 + walk) % len(old)              # use old size as modulus
56     self._front = 0                             # front has been realigned
```

Code Fragment 6.7: Array-based implementation of a queue (continued from Code Fragment 6.6).

The implementation of __len__ and is_empty are trivial, given knowledge of the size. The implementation of the front method is also simple, as the _front index tells us precisely where the desired element is located within the _data list, assuming that list is not empty.

Adding and Removing Elements

The goal of the enqueue method is to add a new element to the back of the queue. We need to determine the proper index at which to place the new element. Although we do not explicitly maintain an instance variable for the back of the queue, we compute the location of the next opening based on the formula:

```
avail = (self._front + self._size) % len(self._data)
```

Note that we are using the size of the queue as it exists ***prior*** to the addition of the new element. For example, consider a queue with capacity 10, current size 3, and first element at index 5. The three elements of such a queue are stored at indices 5, 6, and 7. The new element should be placed at index $(\text{front} + \text{size}) = 8$. In a case with wrap-around, the use of the modular arithmetic achieves the desired circular semantics. For example, if our hypothetical queue had 3 elements with the first at index 8, our computation of $(8+3)$ % 10 evaluates to 1, which is perfect since the three existing elements occupy indices 8, 9, and 0.

When the dequeue method is called, the current value of self._front designates the index of the value that is to be removed and returned. We keep a local reference to the element that will be returned, setting answer = self._data[self._front] just prior to removing the reference to that object from the list, with the assignment self._data[self._front] = None. Our reason for the assignment to None relates to Python's mechanism for reclaiming unused space. Internally, Python maintains a count of the number of references that exist to each object. If that count reaches zero, the object is effectively inaccessible, thus the system may reclaim that memory for future use. (For more details, see Section 15.1.2.) Since we are no longer responsible for storing a dequeued element, we remove the reference to it from our list so as to reduce that element's reference count.

The second significant responsibility of the dequeue method is to update the value of _front to reflect the removal of the element, and the presumed promotion of the second element to become the new first. In most cases, we simply want to increment the index by one, but because of the possibility of a wrap-around configuration, we rely on modular arithmetic as originally described on page 242.

Resizing the Queue

When enqueue is called at a time when the size of the queue equals the size of the underlying list, we rely on a standard technique of doubling the storage capacity of the underlying list. In this way, our approach is similar to the one used when we implemented a DynamicArray in Section 5.3.1.

However, more care is needed in the queue's _resize utility than was needed in the corresponding method of the DynamicArray class. After creating a temporary reference to the old list of values, we allocate a new list that is twice the size and copy references from the old list to the new list. While transferring the contents, we intentionally realign the front of the queue with index 0 in the new array, as shown in Figure 6.7. This realignment is not purely cosmetic. Since the modular arithmetic depends on the size of the array, our state would be flawed had we transferred each element to its same index in the new array.

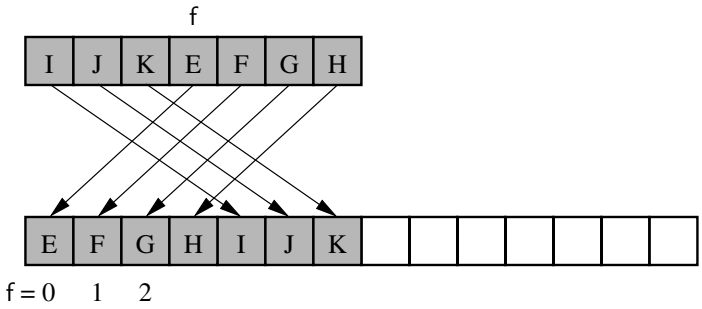


Figure 6.7: Resizing the queue, while realigning the front element with index 0.

Shrinking the Underlying Array

A desirable property of a queue implementation is to have its space usage be $\Theta(n)$ where n is the current number of elements in the queue. Our ArrayQueue implementation, as given in Code Fragments 6.6 and 6.7, does not have this property. It expands the underlying array when enqueue is called with the queue at full capacity, but the dequeue implementation never shrinks the underlying array. As a consequence, the capacity of the underlying array is proportional to the maximum number of elements that have ever been stored in the queue, not the current number of elements.

We discussed this very issue on page 200, in the context of dynamic arrays, and in subsequent Exercises C-5.16 through C-5.20 of that chapter. A robust approach is to reduce the array to half of its current size, whenever the number of elements stored in it falls below *one fourth* of its capacity. We can implement this strategy by adding the following two lines of code in our dequeue method, just after reducing self._size at line 38 of Code Fragment 6.6, to reflect the loss of an element.

```
if 0 < self._size < len(self._data) // 4:
  self._resize(len(self._data) // 2)
```

Analyzing the Array-Based Queue Implementation

Table 6.3 describes the performance of our array-based implementation of the queue ADT, assuming the improvement described above for occasionally shrinking the size of the array. With the exception of the _resize utility, all of the methods rely on a constant number of statements involving arithmetic operations, comparisons, and assignments. Therefore, each method runs in worst-case $O(1)$ time, except for enqueue and dequeue, which have ***amortized*** bounds of $O(1)$ time, for reasons similar to those given in Section 5.3.

Operation	Running Time
Q.enqueue(e)	$O(1)^*$
Q.dequeue( )	$O(1)^*$
Q.first( )	$O(1)$
Q.is_empty( )	$O(1)$
len(Q)	$O(1)$

*amortized

Table 6.3: Performance of an array-based implementation of a queue. The bounds for enqueue and dequeue are amortized due to the resizing of the array. The space usage is $O(n)$, where n is the current number of elements in the queue.

6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a ***double-ended queue***, or ***deque***, which is usually pronounced "deck" to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation "D.Q."

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occassionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will re-insert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

To provide a symmetrical abstraction, the deque ADT is defined so that deque D supports the following methods:

D.add_first(e): Add element e to the front of deque D.

D.add_last(e): Add element e to the back of deque D.

D.delete_first( ): Remove and return the first element from deque D; an error occurs if the deque is empty.

D.delete_last( ): Remove and return the last element from deque D; an error occurs if the deque is empty.

Additionally, the deque ADT will include the following accessors:

D.first( ): Return (but do not remove) the first element of deque D; an error occurs if the deque is empty.

D.last( ): Return (but do not remove) the last element of deque D; an error occurs if the deque is empty.

D.is_empty( ): Return True if deque D does not contain any elements.

len(D): Return the number of elements in deque D; in Python, we implement this with the special method __len__.

Example 6.5: *The following table shows a series of operations and their effects on an initially empty deque D of integers.*

Operation	Return Value	Deque
D.add_last(5)	–	[5]
D.add_first(3)	–	[3, 5]
D.add_first(7)	–	[7, 3, 5]
D.first( )	7	[7, 3, 5]
D.delete_last( )	5	[7, 3]
len(D)	2	[7, 3]
D.delete_last( )	3	[7]
D.delete_last( )	7	[ ]
D.add_first(6)	–	[6]
D.last( )	6	[6]
D.add_first(8)	–	[8, 6]
D.is_empty( )	False	[8, 6]
D.last( )	6	[8, 6]

6.3.2 Implementing a Deque with a Circular Array

We can implement the deque ADT in much the same way as the ArrayQueue class provided in Code Fragments 6.6 and 6.7 of Section 6.2.2 (so much so that we leave the details of an ArrayDeque implementation to Exercise P-6.32). We recommend maintaining the same three instance variables: _data, _size, and _front. Whenever we need to know the index of the back of the deque, or the first available slot beyond the back of the deque, we use modular arithmetic for the computation. For example, our implementation of the last( ) method uses the index

```
back = (self._front + self._size - 1) % len(self._data)
```

Our implementation of the ArrayDeque.add_last method is essentially the same as that for ArrayQueue.enqueue, including the reliance on a _resize utility. Likewise, the implementation of the ArrayDeque.delete_first method is the same as ArrayQueue.dequeue. Implementations of add_first and delete_last use similar techniques. One subtlety is that a call to add_first may need to wrap around the beginning of the array, so we rely on modular arithmetic to circularly *decrement* the index, as

```
self._front = (self._front - 1) % len(self._data)          # cyclic shift
```

The efficiency of an ArrayDeque is similar to that of an ArrayQueue, with all operations having $O(1)$ running time, but with that bound being amortized for operations that may change the size of the underlying list.

6.3.3 Deques in the Python Collections Module

An implementation of a deque class is available in Python's standard collections module. A summary of the most commonly used behaviors of the collections.deque class is given in Table 6.4. It uses more asymmetric nomenclature than our ADT.

Our Deque ADT	**collections.deque**	Description
len(D)	len(D)	number of elements
D.add_first( )	D.appendleft( )	add to beginning
D.add_last( )	D.append( )	add to end
D.delete_first( )	D.popleft( )	remove from beginning
D.delete_last( )	D.pop( )	remove from end
D.first( )	D[0]	access first element
D.last( )	D[−1]	access last element
	D[j]	access arbitrary entry by index
	D[j] = val	modify arbitrary entry by index
	D.clear( )	clear all contents
	D.rotate(k)	circularly shift rightward k steps
	D.remove(e)	remove first matching element
	D.count(e)	count number of matches for e

Table 6.4: Comparison of our deque ADT and the collections.deque class.

The collections.deque interface was chosen to be consistent with established naming conventions of Python's list class, for which append and pop are presumed to act at the end of the list. Therefore, appendleft and popleft designate an operation at the beginning of the list. The library deque also mimics a list in that it is an indexed sequence, allowing arbitrary access or modification using the D[j] syntax.

The library deque constructor also supports an optional maxlen parameter to force a fixed-length deque. However, if a call to append at either end is invoked when the deque is full, it does not throw an error; instead, it causes one element to be dropped from the opposite side. That is, calling appendleft when the deque is full causes an implicit pop from the right side to make room for the new element.

The current Python distribution implements collections.deque with a hybrid approach that uses aspects of circular arrays, but organized into blocks that are themselves organized in a doubly linked list (a data structure that we will introduce in the next chapter). The deque class is formally documented to guarantee $O(1)$-time operations at either end, but $O(n)$-time worst-case operations when using index notation near the middle of the deque.

6.4 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-6.1 What values are returned during the following series of stack operations, if executed upon an initially empty stack? push(5), push(3), pop(), push(2), push(8), pop(), pop(), push(9), push(1), pop(), push(7), push(6), pop(), pop(), push(4), pop(), pop().

R-6.2 Suppose an initially empty stack S has executed a total of 25 push operations, 12 top operations, and 10 pop operations, 3 of which raised Empty errors that were caught and ignored. What is the current size of S?

R-6.3 Implement a function with signature transfer(S, T) that transfers all elements from stack S onto stack T, so that the element that starts at the top of S is the first to be inserted onto T, and the element at the bottom of S ends up at the top of T.

R-6.4 Give a recursive method for removing all the elements from a stack.

R-6.5 Implement a function that reverses a list of elements by pushing them onto a stack in one order, and writing them back to the list in reversed order.

R-6.6 Give a precise and complete definition of the concept of matching for grouping symbols in an arithmetic expression. Your definition may be recursive.

R-6.7 What values are returned during the following sequence of queue operations, if executed on an initially empty queue? enqueue(5), enqueue(3), dequeue(), enqueue(2), enqueue(8), dequeue(), dequeue(), enqueue(9), enqueue(1), dequeue(), enqueue(7), enqueue(6), dequeue(), dequeue(), enqueue(4), dequeue(), dequeue().

R-6.8 Suppose an initially empty queue Q has executed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which raised Empty errors that were caught and ignored. What is the current size of Q?

R-6.9 Had the queue of the previous problem been an instance of ArrayQueue that used an initial array of capacity 30, and had its size never been greater than 30, what would be the final value of the _front instance variable?

R-6.10 Consider what happens if the loop in the ArrayQueue._resize method at lines 53–55 of Code Fragment 6.7 had been implemented as:

```
for k in range(self._size):
  self._data[k] = old[k]                    # rather than old[walk]
```

Give a clear explanation of what could go wrong.

R-6.11 Give a simple adapter that implements our queue ADT while using a collections.deque instance for storage.

R-6.12 What values are returned during the following sequence of deque ADT operations, on initially empty deque? add_first(4), add_last(8), add_last(9), add_first(5), back( ), delete_first( ), delete_last( ), add_last(7), first( ), last( ), add_last(6), delete_first( ), delete_first( ).

R-6.13 Suppose you have a deque D containing the numbers $(1,2,3,4,5,6,7,8)$, in this order. Suppose further that you have an initially empty queue Q. Give a code fragment that uses only D and Q (and no other variables) and results in D storing the elements in the order $(1,2,3,5,4,6,7,8)$.

R-6.14 Repeat the previous problem using the deque D and an initially empty stack S.

Creativity

C-6.15 Suppose Alice has picked three distinct integers and placed them into a stack S in random order. Write a short, straight-line piece of pseudo-code (with no loops or recursion) that uses only one comparison and only one variable x, yet that results in variable x storing the largest of Alice's three integers with probability $2/3$. Argue why your method is correct.

C-6.16 Modify the ArrayStack implementation so that the stack's capacity is limited to maxlen elements, where maxlen is an optional parameter to the constructor (that defaults to None). If push is called when the stack is at full capacity, throw a Full exception (defined similarly to Empty).

C-6.17 In the previous exercise, we assume that the underlying list is initially empty. Redo that exercise, this time preallocating an underlying list with length equal to the stack's maximum capacity.

C-6.18 Show how to use the transfer function, described in Exercise R-6.3, and two temporary stacks, to replace the contents of a given stack S with those same elements, but in reversed order.

C-6.19 In Code Fragment 6.5 we assume that opening tags in HTML have form `<name>`, as with `<li>`. More generally, HTML allows optional attributes to be expressed as part of an opening tag. The general form used is `<name attribute1="value1" attribute2="value2">`; for example, a table can be given a border and additional padding by using an opening tag of `<table border="3" cellpadding="5">`. Modify Code Fragment 6.5 so that it can properly match tags, even when an opening tag may include one or more such attributes.

C-6.20 Describe a nonrecursive algorithm for enumerating all permutations of the numbers $\{1,2,\ldots,n\}$ using an explicit stack.

C-6.21 Show how to use a stack S and a queue Q to generate all possible subsets of an n-element set T nonrecursively.

C-6.22 ***Postfix notation*** is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if "(exp_1) **op** (exp_2)" is a normal, fully parenthesized expression whose operation is **op**, the postfix version of this is "$pexp_1$ $pexp_2$ **op**", where $pexp_1$ is the postfix version of exp_1 and $pexp_2$ is the postfix version of exp_2. The postfix version of a single number or variable is just that number or variable. For example, the postfix version of "$((5+2)*(8-3))/4$" is "$5\ 2\ +\ 8\ 3\ -\ *\ 4\ /$". Describe a nonrecursive way of evaluating an expression in postfix notation.

C-6.23 Suppose you have three nonempty stacks R, S, and T. Describe a sequence of operations that results in S storing all elements originally in T below all of S's original elements, with both sets of those elements in their original order. The final configuration for R should be the same as its original configuration. For example, if $R = [1,2,3]$, $S = [4,5]$, and $T = [6,7,8,9]$, the final configuration should have $R = [1,2,3]$ and $S = [6,7,8,9,4,5]$.

C-6.24 Describe how to implement the stack ADT using a single queue as an instance variable, and only constant additional local memory within the method bodies. What is the running time of the push(), pop(), and top() methods for your design?

C-6.25 Describe how to implement the queue ADT using two stacks as instance variables, such that all queue operations execute in amortized $O(1)$ time. Give a formal proof of the amortized bound.

C-6.26 Describe how to implement the double-ended queue ADT using two stacks as instance variables. What are the running times of the methods?

C-6.27 Suppose you have a stack S containing n elements and a queue Q that is initially empty. Describe how you can use Q to scan S to see if it contains a certain element x, with the additional constraint that your algorithm must return the elements back to S in their original order. You may only use S, Q, and a constant number of other variables.

C-6.28 Modify the ArrayQueue implementation so that the queue's capacity is limited to maxlen elements, where maxlen is an optional parameter to the constructor (that defaults to None). If enqueue is called when the queue is at full capacity, throw a Full exception (defined similarly to Empty).

C-6.29 In certain applications of the queue ADT, it is common to repeatedly dequeue an element, process it in some way, and then immediately enqueue the same element. Modify the ArrayQueue implementation to include a rotate( ) method that has semantics identical to the combination, Q.enqueue(Q.dequeue( )). However, your implementation should be more efficient than making two separate calls (for example, because there is no need to modify _size).

C-6.30 Alice has two queues, Q and R, which can store integers. Bob gives Alice 50 odd integers and 50 even integers and insists that she store all 100 integers in Q and R. They then play a game where Bob picks Q or R at random and then applies the round-robin scheduler, described in the chapter, to the chosen queue a random number of times. If the last number to be processed at the end of this game was odd, Bob wins. Otherwise, Alice wins. How can Alice allocate integers to queues to optimize her chances of winning? What is her chance of winning?

C-6.31 Suppose Bob has four cows that he wants to take across a bridge, but only one yoke, which can hold up to two cows, side by side, tied to the yoke. The yoke is too heavy for him to carry across the bridge, but he can tie (and untie) cows to it in no time at all. Of his four cows, Mazie can cross the bridge in 2 minutes, Daisy can cross it in 4 minutes, Crazy can cross it in 10 minutes, and Lazy can cross it in 20 minutes. Of course, when two cows are tied to the yoke, they must go at the speed of the slower cow. Describe how Bob can get all his cows across the bridge in 34 minutes.

Projects

P-6.32 Give a complete ArrayDeque implementation of the double-ended queue ADT as sketched in Section 6.3.2.

P-6.33 Give an array-based implementation of a double-ended queue supporting all of the public behaviors shown in Table 6.4 for the collections.deque class, including use of the maxlen optional parameter. When a length-limited deque is full, provide semantics similar to the collections.deque class, whereby a call to insert an element on one end of a deque causes an element to be lost from the opposite side.

P-6.34 Implement a program that can input an expression in postfix notation (see Exercise C-6.22) and output its value.

P-6.35 The introduction of Section 6.1 notes that stacks are often used to provide "undo" support in applications like a Web browser or text editor. While support for undo can be implemented with an unbounded stack, many applications provide only ***limited*** support for such an undo history, with a fixed-capacity stack. When push is invoked with the stack at full capacity, rather than throwing a Full exception (as described in Exercise C-6.16), a more typical semantic is to accept the pushed element at the top while "leaking" the oldest element from the bottom of the stack to make room.

Give an implementation of such a LeakyStack abstraction, using a circular array with appropriate storage capacity. This class should have a public interface similar to the bounded-capacity stack in Exercise C-6.16, but with the desired leaky semantics when full.

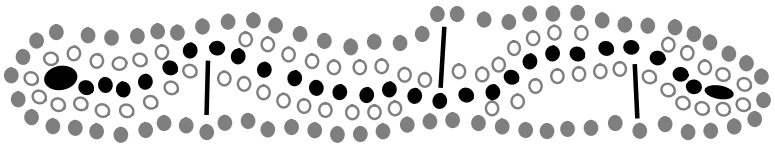
P-6.36 When a share of common stock of some company is sold, the ***capital gain*** (or, sometimes, loss) is the difference between the share's selling price and the price originally paid to buy it. This rule is easy to understand for a single share, but if we sell multiple shares of stock bought over a long period of time, then we must identify the shares actually being sold. A standard accounting principle for identifying which shares of a stock were sold in such a case is to use a FIFO protocol—the shares sold are the ones that have been held the longest (indeed, this is the default method built into several personal finance software packages). For example, suppose we buy 100 shares at $20 each on day 1, 20 shares at $24 on day 2, 200 shares at $36 on day 3, and then sell 150 shares on day 4 at $30 each. Then applying the FIFO protocol means that of the 150 shares sold, 100 were bought on day 1, 20 were bought on day 2, and 30 were bought on day 3. The capital gain in this case would therefore be $100 \cdot 10 + 20 \cdot 6 + 30 \cdot (-6)$, or $940. Write a program that takes as input a sequence of transactions of the form "`buy` x `share(s) at $`y `each`" or "`sell` x `share(s) at $`y `each`," assuming that the transactions occur on consecutive days and the values x and y are integers. Given this input sequence, the output should be the total capital gain (or loss) for the entire sequence, using the FIFO protocol to identify shares.

P-6.37 Design an ADT for a two-color, double-stack ADT that consists of two stacks—one "red" and one "blue"—and has as its operations color-coded versions of the regular stack ADT operations. For example, this ADT should support both a red push operation and a blue push operation. Give an efficient implementation of this ADT using a single array whose capacity is set at some value N that is assumed to always be larger than the sizes of the red and blue stacks combined.

Chapter Notes

We were introduced to the approach of defining data structures first in terms of their ADTs and then in terms of concrete implementations by the classic books by Aho, Hopcroft, and Ullman [5, 6]. Exercises C-6.30, and C-6.31 are similar to interview questions said to be from a well-known software company. For further study of abstract data types, see Liskov and Guttag [71], Cardelli and Wegner [23], or Demurjian [33].

Chapter 7 Linked Lists



Contents

7.1 Singly Linked Lists 256
7.1.1 Implementing a Stack with a Singly Linked List 261
7.1.2 Implementing a Queue with a Singly Linked List 264
7.2 Circularly Linked Lists 266
7.2.1 Round-Robin Schedulers 267
7.2.2 Implementing a Queue with a Circularly Linked List 268
7.3 Doubly Linked Lists 270
7.3.1 Basic Implementation of a Doubly Linked List 273
7.3.2 Implementing a Deque with a Doubly Linked List 275
7.4 The Positional List ADT 277
7.4.1 The Positional List Abstract Data Type 279
7.4.2 Doubly Linked List Implementation 281
7.5 Sorting a Positional List 285
7.6 Case Study: Maintaining Access Frequencies 286
7.6.1 Using a Sorted List 286
7.6.2 Using a List with the Move-to-Front Heuristic 289
7.7 Link-Based vs. Array-Based Sequences 292
7.8 Exercises 294

In Chapter 5 we carefully examined Python's array-based list class, and in Chapter 6 we demonstrated use of that class in implementing the classic stack, queue, and dequeue ADTs. Python's list class is highly optimized, and often a great choice for storage. With that said, there are some notable disadvantages:

1. The length of a dynamic array might be longer than the actual number of elements that it stores.
2. Amortized bounds for operations may be unacceptable in real-time systems.
3. Insertions and deletions at interior positions of an array are expensive.

In this chapter, we introduce a data structure known as a ***linked list***, which provides an alternative to an array-based sequence (such as a Python list). Both array-based sequences and linked lists keep elements in a certain order, but using a very different style. An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements. A linked list, in contrast, relies on a more distributed representation in which a lightweight object, known as a ***node***, is allocated for each element. Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

We will demonstrate a trade-off of advantages and disadvantages when contrasting array-based sequences and linked lists. Elements of a linked list cannot be efficiently accessed by a numeric index k, and we cannot tell just by examining a node if it is the second, fifth, or twentieth node in the list. However, linked lists avoid the three disadvantages noted above for array-based sequences.

7.1 Singly Linked Lists

A ***singly linked list***, in its simplest form, is a collection of ***nodes*** that collectively form a linear sequence. Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figures 7.1 and 7.2).

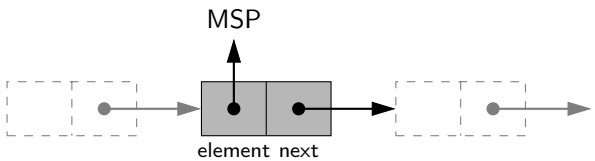


Figure 7.1: Example of a node instance that forms part of a singly linked list. The node's element member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the next member references the subsequent node of the linked list (or None if there is no further node).

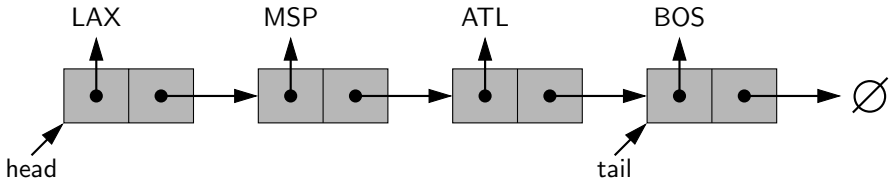


Figure 7.2: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named head that identifies the first node of the list, and in some applications another member named tail that identifies the last node of the list. The **None** object is denoted as Ø.

The first and last node of a linked list are known as the ***head*** and ***tail*** of the list, respectively. By starting at the head, and moving from one node to another by following each node's next reference, we can reach the tail of the list. We can identify the tail as the node having None as its next reference. This process is commonly known as ***traversing*** the linked list. Because the next reference of a node can be viewed as a ***link*** or ***pointer*** to another node, the process of traversing a list is also known as ***link hopping*** or ***pointer hopping***.

A linked list's representation in memory relies on the collaboration of many objects. Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None). Another object represents the linked list as a whole. Minimally, the linked list instance must keep a reference to the head of the list. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list. However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal. In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the ***size*** of the list), to avoid the need to traverse the list to count the nodes.

For the remainder of this chapter, we continue to illustrate nodes as objects, and each node's "next" reference as a pointer. However, for the sake of simplicity, we illustrate a node's element embedded directly within the node structure, even though the element is, in fact, an independent object. For example, Figure 7.3 is a more compact illustration of the linked list from Figure 7.2.

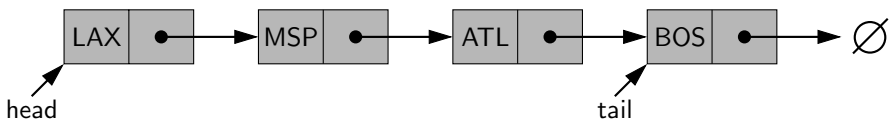


Figure 7.3: A compact illustration of a singly linked list, with elements embedded in the nodes (rather than more accurately drawn as references to external objects).

Inserting an Element at the Head of a Singly Linked List

An important property of a linked list is that it does not have a predetermined fixed size; it uses space proportionally to its current number of elements. When using a singly linked list, we can easily insert an element at the head of the list, as shown in Figure 7.4, and described with pseudo-code in Code Fragment 7.1. The main idea is that we create a new node, set its element to the new element, set its next link to refer to the current head, and then set the list's head to point to the new node.

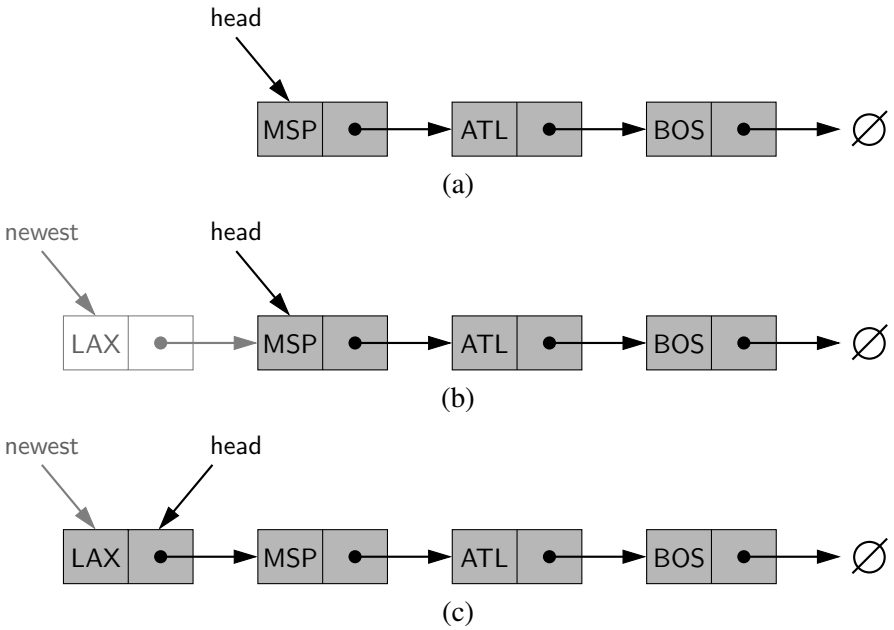


Figure 7.4: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the head reference.

Algorithm add_first(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = L.head  {set new node's next to reference the old head node}
L.head = newest       {set variable head to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.1: Inserting a new element at the beginning of a singly linked list L. Note that we set the next pointer of the new node ***before*** we reassign variable L.head to it. If the list were initially empty (i.e., L.head is None), then a natural consequence is that the new node has its next reference set to None.

Inserting an Element at the Tail of a Singly Linked List

We can also easily insert an element at the tail of the list, provided we keep a reference to the tail node, as shown in Figure 7.5. In this case, we create a new node, assign its next reference to None, set the next reference of the tail to point to this new node, and then update the tail reference itself to this new node. We give the details in Code Fragment 7.2.

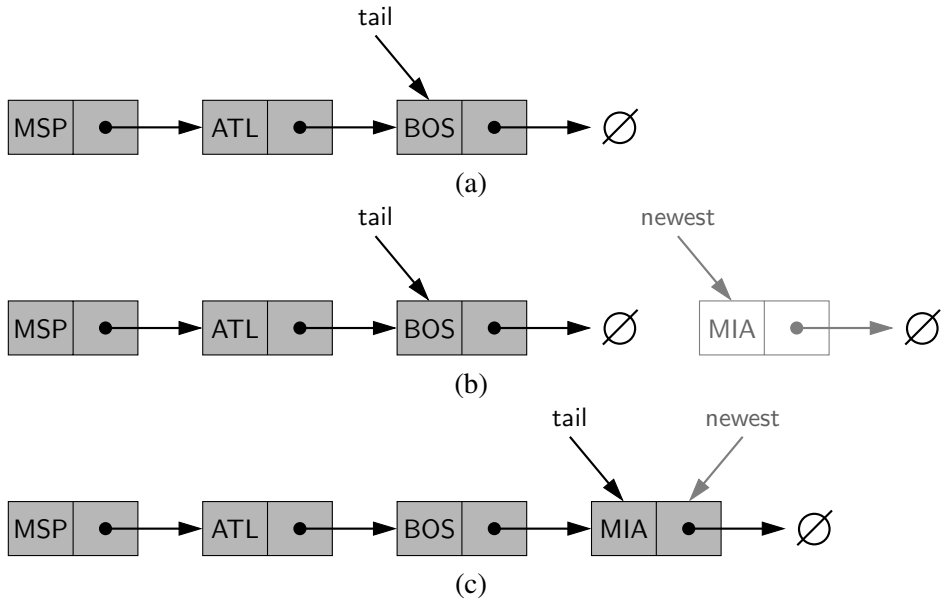


Figure 7.5: Insertion at the tail of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the tail reference. Note that we must set the next link of the tail in (b) before we assign the tail variable to point to the new node in (c).

Algorithm add_last(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = None    {set new node's next to reference the None object}
L.tail.next = newest  {make old tail node point to new node}
L.tail = newest       {set variable tail to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.2: Inserting a new node at the end of a singly linked list. Note that we set the next pointer for the old tail node ***before*** we make variable tail point to the new node. This code would need to be adjusted for inserting onto an empty list, since there would not be an existing tail node.

Removing an Element from a Singly Linked List

Removing an element from the ***head*** of a singly linked list is essentially the reverse operation of inserting a new element at the head. This operation is illustrated in Figure 7.6 and given in detail in Code Fragment 7.3.

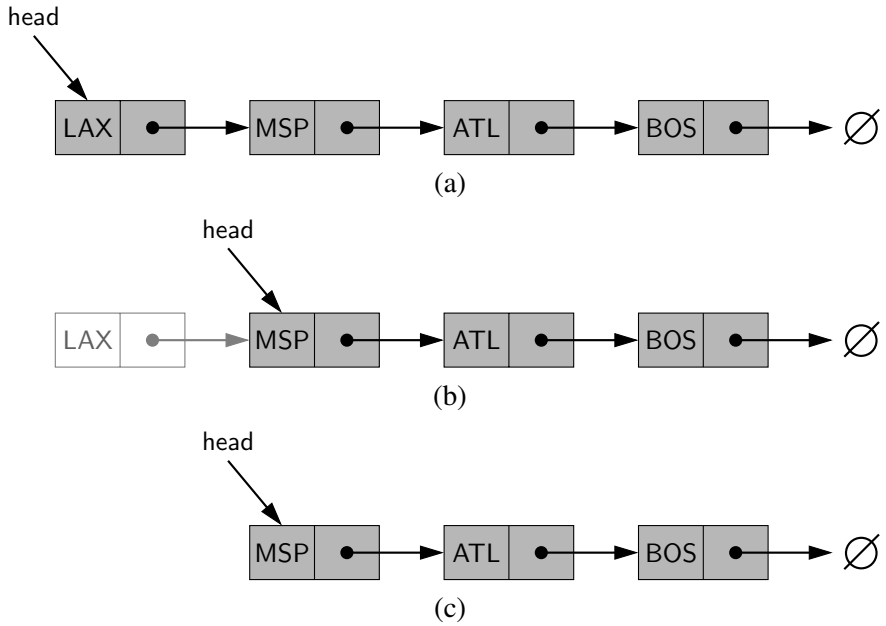


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after "linking out" the old head; (c) final configuration.

Algorithm remove_first(L):
  if L.head is None **then**
    Indicate an error: the list is empty.
  L.head = L.head.next {make head point to next node (or None)}
  L.size = L.size − 1 {decrement the node count}

Code Fragment 7.3: Removing the node at the beginning of a singly linked list.

Unfortunately, we cannot easily delete the last node of a singly linked list. Even if we maintain a tail reference directly to the last node of the list, we must be able to access the node ***before*** the last node in order to remove the last node. But we cannot reach the node before the tail by following next links from the tail. The only way to access this node is to start from the head of the list and search all the way through the list. But such a sequence of link-hopping operations could take a long time. If we want to support such an operation efficiently, we will need to make our list ***doubly linked*** (as we do in Section 7.3).

7.1.1 Implementing a Stack with a Singly Linked List

In this section, we demonstrate use of a singly linked list by providing a complete Python implementation of the stack ADT (see Section 6.1). In designing such an implementation, we need to decide whether to model the top of the stack at the head or at the tail of the list. There is clearly a best choice here; we can efficiently insert and delete elements in constant time only at the head. Since all stack operations affect the top, we orient the top of the stack at the head of our list.

To represent individual nodes of the list, we develop a lightweight _Node class. This class will never be directly exposed to the user of our stack class, so we will formally define it as a nonpublic, nested class of our eventual LinkedStack class (see Section 2.5.1 for discussion of nested classes). The definition of the _Node class is shown in Code Fragment 7.4.

```
class _Node:
  """Lightweight, nonpublic class for storing a singly linked node."""
  __slots__ = '_element', '_next'          # streamline memory usage

  def __init__(self, element, next):       # initialize node's fields
    self._element = element                # reference to user's element
    self._next = next                      # reference to next node
```

Code Fragment 7.4: A lightweight _Node class for a singly linked list.

A node has only two instance variables: _element and _next. We intentionally define __slots__ to streamline the memory usage (see page 99 of Section 2.5.1 for discussion), because there may potentially be many node instances in a single list. The constructor of the _Node class is designed for our convenience, allowing us to specify initial values for both fields of a newly created node.

A complete implementation of our LinkedStack class is given in Code Fragments 7.5 and 7.6. Each stack instance maintains two variables. The _head member is a reference to the node at the head of the list (or None, if the stack is empty). We keep track of the current number of elements with the _size instance variable, for otherwise we would be forced to traverse the entire list to count the number of elements when reporting the size of the stack.

The implementation of push essentially mirrors the pseudo-code for insertion at the head of a singly linked list as outlined in Code Fragment 7.1. When we push a new element e onto the stack, we accomplish the necessary changes to the linked structure by invoking the constructor of the _Node class as follows:

```
self._head = self._Node(e, self._head)      # create and link a new node
```

Note that the _next field of the new node is set to the ***existing*** top node, and then self._head is reassigned to the new node.

```
1  class LinkedStack:
2    """LIFO Stack implementation using a singly linked list for storage."""
3
4    #---------------------------- nested _Node class ----------------------------
5    class _Node:
6      """Lightweight, nonpublic class for storing a singly linked node."""
7      __slots__ = '_element', '_next'         # streamline memory usage
8
9      def __init__(self, element, next):      # initialize node's fields
10       self._element = element               # reference to user's element
11       self._next = next                     # reference to next node
12
13   #------------------------------- stack methods -------------------------------
14   def __init__(self):
15     """Create an empty stack."""
16     self._head = None                       # reference to the head node
17     self._size = 0                          # number of stack elements
18
19   def __len__(self):
20     """Return the number of elements in the stack."""
21     return self._size
22
23   def is_empty(self):
24     """Return True if the stack is empty."""
25     return self._size == 0
26
27   def push(self, e):
28     """Add element e to the top of the stack."""
29     self._head = self._Node(e, self._head)    # create and link a new node
30     self._size += 1
31
32   def top(self):
33     """Return (but do not remove) the element at the top of the stack.
34
35     Raise Empty exception if the stack is empty.
36     """
37     if self.is_empty():
38       raise Empty('Stack is empty')
39     return self._head._element              # top of stack is at head of list
```

Code Fragment 7.5: Implementation of a stack ADT using a singly linked list (continued in Code Fragment 7.6).

```
40    def pop(self):
41      """Remove and return the element from the top of the stack (i.e., LIFO).
42
43      Raise Empty exception if the stack is empty.
44      """
45      if self.is_empty( ):
46        raise Empty('Stack is empty')
47      answer = self._head._element
48      self._head = self._head._next              # bypass the former top node
49      self._size -= 1
50      return answer
```

Code Fragment 7.6: Implementation of a stack ADT using a singly linked list (continued from Code Fragment 7.5).

When implementing the top method, the goal is to return the ***element*** that is at the top of the stack. When the stack is empty, we raise an Empty exception, as originally defined in Code Fragment 6.1 of Chapter 6. When the stack is nonempty, self._head is a reference to the first ***node*** of the linked list. The top element can be identified as self._head._element.

Our implementation of pop essentially mirrors the pseudo-code given in Code Fragment 7.3, except that we maintain a local reference to the element that is stored at the node that is being removed, and we return that element to the caller of pop.

The analysis of our LinkedStack operations is given in Table 7.1. We see that all of the methods complete in ***worst-case*** constant time. This is in contrast to the amortized bounds for the ArrayStack that were given in Table 6.2.

Operation	Running Time
S.push(e)	$O(1)$
S.pop( )	$O(1)$
S.top( )	$O(1)$
len(S)	$O(1)$
S.is_empty( )	$O(1)$

Table 7.1: Performance of our LinkedStack implementation. All bounds are worst-case and our space usage is $O(n)$, where n is the current number of elements in the stack.

7.1.2 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can use a singly linked list to implement the queue ADT while supporting worst-case $O(1)$-time for all operations. Because we need to perform operations on both ends of the queue, we will explicitly maintain both a _head reference and a _tail reference as instance variables for each queue. The natural orientation for a queue is to align the front of the queue with the head of the list, and the back of the queue with the tail of the list, because we must be able to enqueue elements at the back, and dequeue them from the front. (Recall from the introduction of Section 7.1 that we are unable to efficiently remove elements from the tail of a singly linked list.) Our implementation of a LinkedQueue class is given in Code Fragments 7.7 and 7.8.

```
1  class LinkedQueue:
2    """FIFO queue implementation using a singly linked list for storage."""
3
4    class _Node:
5      """Lightweight, nonpublic class for storing a singly linked node."""
6      (omitted here; identical to that of LinkedStack._Node)
7
8    def __init__(self):
9      """Create an empty queue."""
10     self._head = None
11     self._tail = None
12     self._size = 0                          # number of queue elements
13
14   def __len__(self):
15     """Return the number of elements in the queue."""
16     return self._size
17
18   def is_empty(self):
19     """Return True if the queue is empty."""
20     return self._size == 0
21
22   def first(self):
23     """Return (but do not remove) the element at the front of the queue."""
24     if self.is_empty( ):
25       raise Empty('Queue is empty')
26     return self._head._element              # front aligned with head of list
```

Code Fragment 7.7: Implementation of a queue ADT using a singly linked list (continued in Code Fragment 7.8).

```
27    def dequeue(self):
28      """Remove and return the first element of the queue (i.e., FIFO).
29
30      Raise Empty exception if the queue is empty.
31      """
32      if self.is_empty( ):
33        raise Empty('Queue is empty')
34      answer = self._head._element
35      self._head = self._head._next
36      self._size -= 1
37      if self.is_empty( ):                    # special case as queue is empty
38        self._tail = None                     # removed head had been the tail
39      return answer
40
41    def enqueue(self, e):
42      """Add an element to the back of queue."""
43      newest = self._Node(e, None)            # node will be new tail node
44      if self.is_empty( ):
45        self._head = newest                   # special case: previously empty
46      else:
47        self._tail._next = newest
48      self._tail = newest                     # update reference to tail node
49      self._size += 1
```

Code Fragment 7.8: Implementation of a queue ADT using a singly linked list (continued from Code Fragment 7.7).

Many aspects of our implementation are similar to that of the LinkedStack class, such as the definition of the nested _Node class. Our implementation of dequeue for LinkedQueue is similar to that of pop for LinkedStack, as both remove the head of the linked list. However, there is a subtle difference because our queue must accurately maintain the _tail reference (no such variable was maintained for our stack). In general, an operation at the head has no effect on the tail, but when dequeue is invoked on a queue with one element, we are simultaneously removing the tail of the list. We therefore set self._tail to None for consistency.

There is a similar complication in our implementation of enqueue. The newest node always becomes the new tail. Yet a distinction is made depending on whether that new node is the only node in the list. In that case, it also becomes the new head; otherwise the new node must be linked immediately after the existing tail node.

In terms of performance, the LinkedQueue is similar to the LinkedStack in that all operations run in worst-case constant time, and the space usage is linear in the current number of elements.

7.2 Circularly Linked Lists

In Section 6.2.2, we introduced the notion of a "circular" array and demonstrated its use in implementing the queue ADT. In reality, the notion of a circular array was artificial, in that there was nothing about the representation of the array itself that was circular in structure. It was our use of modular arithmetic when "advancing" an index from the last slot to the first slot that provided such an abstraction.

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown in Figure 7.7. We call such a structure a ***circularly linked list***.

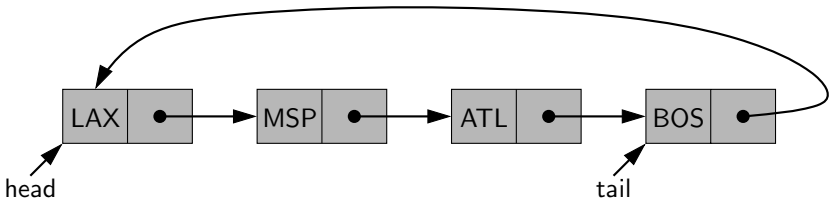


Figure 7.7: Example of a singly linked list with circular structure.

A circularly linked list provides a more general model than a standard linked list for data sets that are cyclic, that is, which do not have any particular notion of a beginning and end. Figure 7.8 provides a more symmetric illustration of the same circular list structure as Figure 7.7.

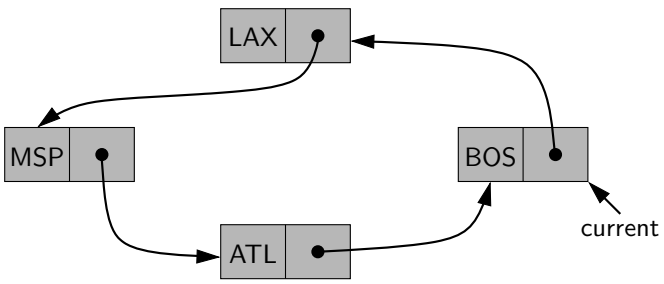


Figure 7.8: Example of a circular linked list, with current denoting a reference to a select node.

A circular view similar to Figure 7.8 could be used, for example, to describe the order of train stops in the Chicago loop, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list. We use the identifier ***current*** to describe such a designated node. By setting current = current.next, we can effectively advance through the nodes of the list.

7.2.1 Round-Robin Schedulers

To motivate the use of a circularly linked list, we consider a ***round-robin*** scheduler, which iterates through a collection of elements in a circular fashion and "services" each element by performing a given action on it. Such a scheduler is used, for example, to fairly allocate a resource that must be shared by a collection of clients. For instance, round-robin scheduling is often used to allocate slices of CPU time to various applications running concurrently on a computer.

A round-robin scheduler could be implemented with the general queue ADT, by repeatedly performing the following steps on queue Q (see Figure 7.9):

1. $e = Q.\mathsf{dequeue}()$
2. Service element e
3. $Q.\mathsf{enqueue}(e)$

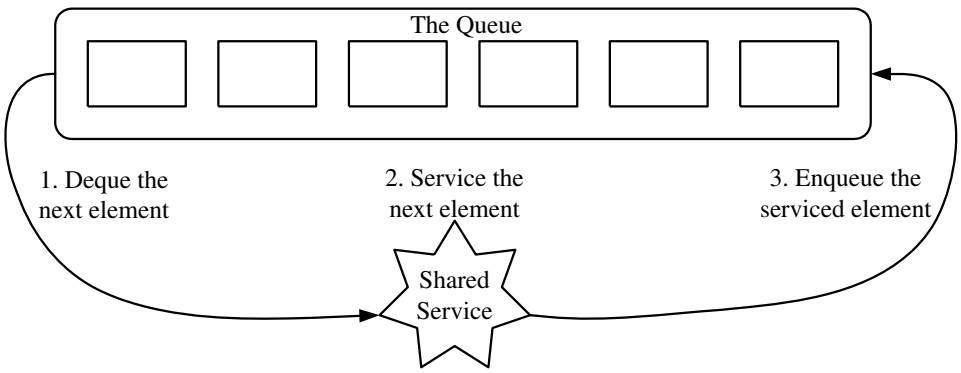


Figure 7.9: The three iterative steps for round-robin scheduling using a queue.

If we use of the LinkedQueue class of Section 7.1.2 for such an application, there is unnecessary effort in the combination of a dequeue operation followed soon after by an enqueue of the same element. One node is removed from the list, with appropriate adjustments to the head of the list and the size decremented, and then a new node is created to reinsert at the tail of the list and the size is incremented.

If using a circularly linked list, the effective transfer of an item from the "head" of the list to the "tail" of the list can be accomplished by advancing a reference that marks the boundary of the queue. We will next provide an implementation of a CircularQueue class that supports the entire queue ADT, together with an additional method, rotate( ), that moves the first element of the queue to the back. (A similar method is supported by the deque class of Python's collections module; see Table 6.4.) With this operation, a round-robin schedule can more efficiently be implemented by repeatedly performing the following steps:

1. Service element $Q.\mathsf{front}()$
2. $Q.\mathsf{rotate}()$

7.2.2 Implementing a Queue with a Circularly Linked List

To implement the queue ADT using a circularly linked list, we rely on the intuition of Figure 7.7, in which the queue has a head and a tail, but with the next reference of the tail linked to the head. Given such a model, there is no need for us to explicitly store references to both the head and the tail; as long as we keep a reference to the tail, we can always find the head by following the tail's next reference.

Code Fragments 7.9 and 7.10 provide an implementation of a CircularQueue class based on this model. The only two instance variables are _tail, which is a reference to the tail node (or None when empty), and _size, which is the current number of elements in the queue. When an operation involves the front of the queue, we recognize self._tail._next as the head of the queue. When enqueue is called, a new node is placed just after the tail but before the current head, and then the new node becomes the tail.

In addition to the traditional queue operations, the CircularQueue class supports a rotate method that more efficiently enacts the combination of removing the front element and reinserting it at the back of the queue. With the circular representation, we simply set self._tail = self._tail._next to make the old head become the new tail (with the node after the old head becoming the new head).

```
1  class CircularQueue:
2    """Queue implementation using circularly linked list for storage."""
3
4    class _Node:
5      """Lightweight, nonpublic class for storing a singly linked node."""
6      (omitted here; identical to that of LinkedStack._Node)
7
8    def __init__(self):
9      """Create an empty queue."""
10     self._tail = None                    # will represent tail of queue
11     self._size = 0                       # number of queue elements
12
13   def __len__(self):
14     """Return the number of elements in the queue."""
15     return self._size
16
17   def is_empty(self):
18     """Return True if the queue is empty."""
19     return self._size == 0
```

Code Fragment 7.9: Implementation of a CircularQueue class, using a circularly linked list as storage (continued in Code Fragment 7.10).

```
20    def first(self):
21      """Return (but do not remove) the element at the front of the queue.
22
23      Raise Empty exception if the queue is empty.
24      """
25      if self.is_empty( ):
26        raise Empty('Queue is empty')
27      head = self._tail._next
28      return head._element
29
30    def dequeue(self):
31      """Remove and return the first element of the queue (i.e., FIFO).
32
33      Raise Empty exception if the queue is empty.
34      """
35      if self.is_empty( ):
36        raise Empty('Queue is empty')
37      oldhead = self._tail._next
38      if self._size == 1:                    # removing only element
39        self._tail = None                    # queue becomes empty
40      else:
41        self._tail._next = oldhead._next     # bypass the old head
42      self._size -= 1
43      return oldhead._element
44
45    def enqueue(self, e):
46      """Add an element to the back of queue."""
47      newest = self._Node(e, None)           # node will be new tail node
48      if self.is_empty( ):
49        newest._next = newest                # initialize circularly
50      else:
51        newest._next = self._tail._next      # new node points to head
52        self._tail._next = newest            # old tail points to new node
53      self._tail = newest                    # new node becomes the tail
54      self._size += 1
55
56    def rotate(self):
57      """Rotate front element to the back of the queue."""
58      if self._size > 0:
59        self._tail = self._tail._next        # old head becomes new tail
```

Code Fragment 7.10: Implementation of a CircularQueue class, using a circularly linked list as storage (continued from Code Fragment 7.9).

7.3 Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In the opening of Section 7.1, we emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately *precedes* the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a ***doubly linked list***. These lists allow a greater variety of $O(1)$-time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term "next" for the reference to the node that follows another, and we introduce the term "prev" for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a ***header*** node at the beginning of the list, and a ***trailer*** node at the end of the list. These "dummy" nodes are known as ***sentinels*** (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 7.10.

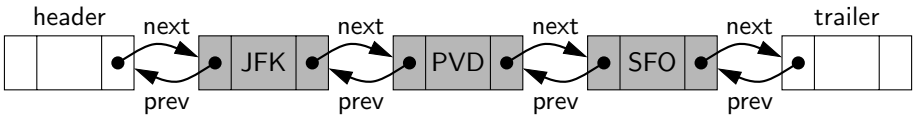


Figure 7.10: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header; the remaining fields of the sentinels are irrelevant (presumably None, in Python). For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

Advantage of Using Sentinels

Although we could implement a doubly linked list without sentinel nodes (as we did with our singly linked list in Section 7.1), the slight extra space devoted to the sentinels greatly simplifies the logic of our operations. Most notably, the header and trailer nodes never change—only the nodes between them change. Furthermore, we can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

For contrast, look back at our LinkedQueue implementation from Section 7.1.2. Its enqueue method, given in Code Fragment 7.8, adds a new node to the end of the list. However, its implementation required a conditional to manage the special case of inserting into an empty list. In the general case, the new node was linked after the existing tail. But when adding to an empty list, there is no existing tail; instead it is necessary to reassign self._head to reference the new node. The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting and Deleting with a Doubly Linked List

Every insertion into our doubly linked list representation will take place between a pair of existing nodes, as diagrammed in Figure 7.11. For example, when a new element is inserted at the front of the sequence, we will simply add the new node *between* the header and the node that is currently after the header. (See Figure 7.12.)

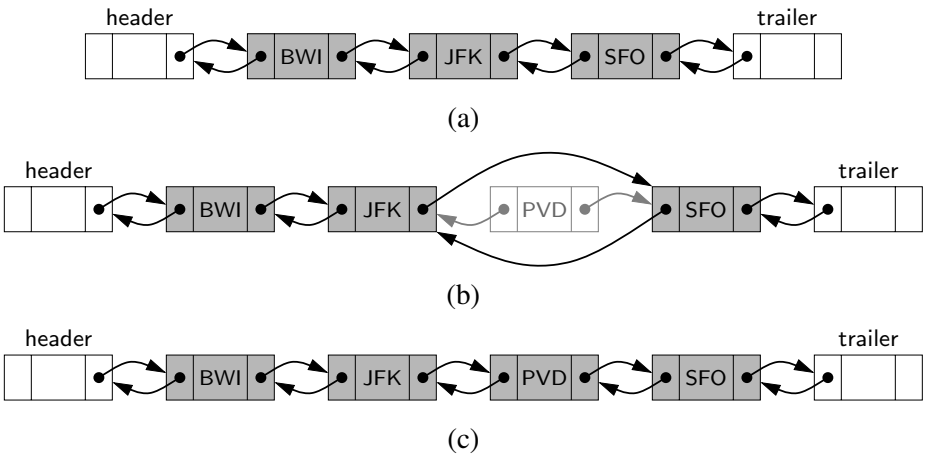


Figure 7.11: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

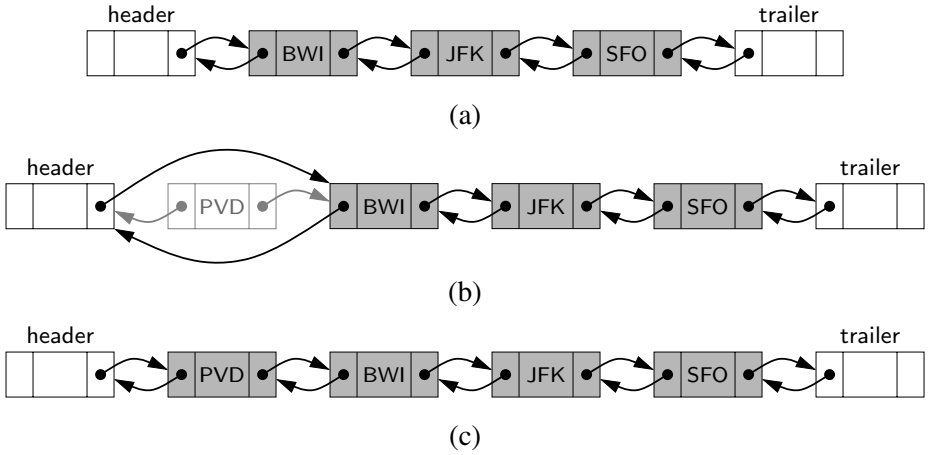


Figure 7.12: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

The deletion of a node, portrayed in Figure 7.13, proceeds in the opposite fashion of an insertion. The two neighbors of the node to be deleted are linked directly to each other, thereby bypassing the original node. As a result, that node will no longer be considered part of the list and it can be reclaimed by the system. Because of our use of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.

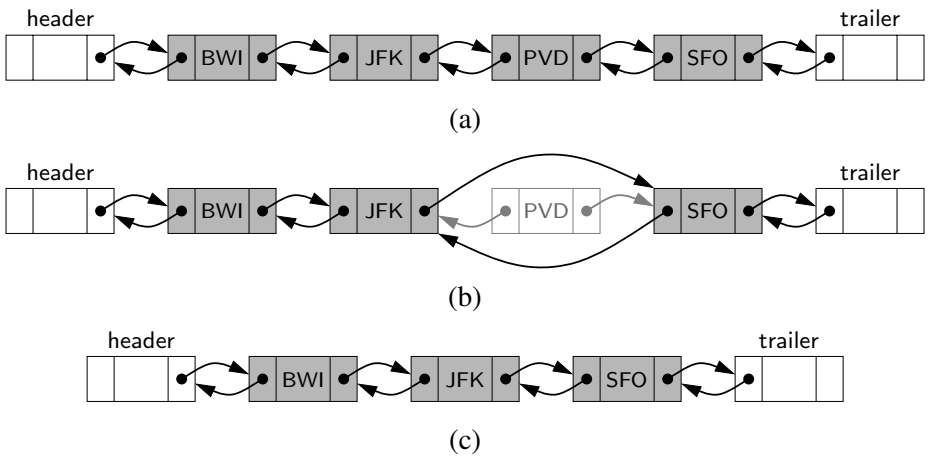


Figure 7.13: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

7.3.1 Basic Implementation of a Doubly Linked List

We begin by providing a preliminary implementation of a doubly linked list, in the form of a class named _DoublyLinkedBase. We intentionally name the class with a leading underscore because we do not intend for it to provide a coherent public interface for general use. We will see that linked lists can support general insertions and deletions in $O(1)$ worst-case time, but only if the location of an operation can be succinctly identified. With array-based sequences, an integer index was a convenient means for describing a position within a sequence. However, an index is not convenient for linked lists as there is no efficient way to find the jth element; it would seem to require a traversal of a portion of the list.

When working with a linked list, the most direct way to describe the location of an operation is by identifying a relevant node of the list. However, we prefer to encapsulate the inner workings of our data structure to avoid having users directly access nodes of a list. In the remainder of this chapter, we will develop two public classes that inherit from our _DoublyLinkedBase class to provide more coherent abstractions. Specifically, in Section 7.3.2, we provide a LinkedDeque class that implements the double-ended queue ADT introduced in Section 6.3; that class only supports operations at the ends of the queue, so there is no need for a user to identify an interior position within the list. In Section 7.4, we introduce a new PositionalList abstraction that provides a public interface that allows arbitrary insertions and deletions from a list.

Our low-level _DoublyLinkedBase class relies on the use of a nonpublic _Node class that is similar to that for a singly linked list, as given in Code Fragment 7.4, except that the doubly linked version includes a _prev attribute, in addition to the _next and _element attributes, as shown in Code Fragment 7.11.

```
class _Node:
  """Lightweight, nonpublic class for storing a doubly linked node."""
  __slots__ = '_element', '_prev', '_next'  # streamline memory

  def __init__(self, element, prev, next):          # initialize node's fields
    self._element = element                         # user's element
    self._prev = prev                               # previous node reference
    self._next = next                               # next node reference
```

Code Fragment 7.11: A Python _Node class for use in a doubly linked list.

The remainder of our _DoublyLinkedBase class is given in Code Fragment 7.12. The constructor instantiates the two sentinel nodes and links them directly to each other. We maintain a _size member and provide public support for __len__ and is_empty so that these behaviors can be directly inherited by the subclasses.

```
class _DoublyLinkedBase:
  """A base class providing a doubly linked list representation."""

  class _Node:
    """Lightweight, nonpublic class for storing a doubly linked node."""
    (omitted here; see previous code fragment)

  def __init__(self):
    """Create an empty list."""
    self._header = self._Node(None, None, None)
    self._trailer = self._Node(None, None, None)
    self._header._next = self._trailer                # trailer is after header
    self._trailer._prev = self._header                # header is before trailer
    self._size = 0                                    # number of elements

  def __len__(self):
    """Return the number of elements in the list."""
    return self._size

  def is_empty(self):
    """Return True if list is empty."""
    return self._size == 0

  def _insert_between(self, e, predecessor, successor):
    """Add element e between two existing nodes and return new node."""
    newest = self._Node(e, predecessor, successor)   # linked to neighbors
    predecessor._next = newest
    successor._prev = newest
    self._size += 1
    return newest

  def _delete_node(self, node):
    """Delete nonsentinel node from the list and return its element."""
    predecessor = node._prev
    successor = node._next
    predecessor._next = successor
    successor._prev = predecessor
    self._size -= 1
    element = node._element                           # record deleted element
    node._prev = node._next = node._element = None    # deprecate node
    return element                                    # return deleted element
```

Code Fragment 7.12: A base class for managing a doubly linked list.

The other two methods of our class are the nonpublic utilities, _insert_between and _delete_node. These provide generic support for insertions and deletions, respectively, but require one or more node references as parameters. The implementation of the _insert_between method is modeled upon the algorithm that was previously portrayed in Figure 7.11. It creates a new node, with that node's fields initialized to link to the specified neighboring nodes. Then the fields of the neighboring nodes are updated to include the newest node in the list. For later convenience, the method returns a reference to the newly created node.

The implementation of the _delete_node method is modeled upon the algorithm portrayed in Figure 7.13. The neighbors of the node to be deleted are linked directly to each other, thereby bypassing the deleted node from the list. As a formality, we intentionally reset the _prev, _next, and _element fields of the deleted node to None (after recording the element to be returned). Although the deleted node will be ignored by the rest of the list, setting its fields to None is advantageous as it may help Python's garbage collection, since unnecessary links to the other nodes and the stored element are eliminated. We will also rely on this configuration to recognize a node as "deprecated" when it is no longer part of the list.

7.3.2 Implementing a Deque with a Doubly Linked List

The double-ended queue (deque) ADT was introduced in Section 6.3. With an array-based implementation, we achieve all operations in *amortized* $O(1)$ time, due to the occasional need to resize the array. With an implementation based upon a doubly linked list, we can achieve all deque operation in *worst-case* $O(1)$ time.

We provide an implementation of a LinkedDeque class (Code Fragment 7.13) that inherits from the _DoublyLinkedBase class of the preceding section. We do not provide an explicit __init__ method for the LinkedDeque class, as the inherited version of that method suffices to initialize a new instance. We also rely on the inherited methods __len__ and is_empty in meeting the deque ADT.

With the use of sentinels, the key to our implementation is to remember that the header does not store the first element of the deque—it is the node just *after* the header that stores the first element (assuming the deque is nonempty). Similarly, the node just *before* the trailer stores the last element of the deque.

We use the inherited _insert_between method to insert at either end of the deque. To insert an element at the front of the deque, we place it immediately between the header and the node just after the header. An insertion at the end of deque is placed immediately before the trailer node. Note that these operations succeed, even when the deque is empty; in such a situation, the new node is placed between the two sentinels. When deleting an element from a nonempty deque, we rely upon the inherited _delete_node method, knowing that the designated node is assured to have neighbors on each side.

```
1  class LinkedDeque(_DoublyLinkedBase):          # note the use of inheritance
2    """Double-ended queue implementation based on a doubly linked list."""
3
4    def first(self):
5      """Return (but do not remove) the element at the front of the deque."""
6      if self.is_empty( ):
7        raise Empty("Deque is empty")
8      return self._header._next._element           # real item just after header
9
10   def last(self):
11     """Return (but do not remove) the element at the back of the deque."""
12     if self.is_empty( ):
13       raise Empty("Deque is empty")
14     return self._trailer._prev._element          # real item just before trailer
15
16   def insert_first(self, e):
17     """Add an element to the front of the deque."""
18     self._insert_between(e, self._header, self._header._next)   # after header
19
20   def insert_last(self, e):
21     """Add an element to the back of the deque."""
22     self._insert_between(e, self._trailer._prev, self._trailer)    # before trailer
23
24   def delete_first(self):
25     """Remove and return the element from the front of the deque.
26
27     Raise Empty exception if the deque is empty.
28     """
29     if self.is_empty( ):
30       raise Empty("Deque is empty")
31     return self._delete_node(self._header._next)       # use inherited method
32
33   def delete_last(self):
34     """Remove and return the element from the back of the deque.
35
36     Raise Empty exception if the deque is empty.
37     """
38     if self.is_empty( ):
39       raise Empty("Deque is empty")
40     return self._delete_node(self._trailer._prev)      # use inherited method
```

Code Fragment 7.13: Implementation of a LinkedDeque class that inherits from the _DoublyLinkedBase class.

7.4 The Positional List ADT

The abstract data types that we have considered thus far, namely stacks, queues, and double-ended queues, only allow update operations that occur at one end of a sequence or the other. We wish to have a more general abstraction. For example, although we motivated the FIFO semantics of a queue as a model for customers who are waiting to speak with a customer service representative, or fans who are waiting in line to buy tickets to a show, the queue ADT is too limiting. What if a waiting customer decides to hang up before reaching the front of the customer service queue? Or what if someone who is waiting in line to buy tickets allows a friend to "cut" into line at that position? We would like to design an abstract data type that provides a user a way to refer to elements anywhere in a sequence, and to perform arbitrary insertions and deletions.

When working with array-based sequences (such as a Python list), integer indices provide an excellent means for describing the location of an element, or the location at which an insertion or deletion should take place. However, numeric indices are not a good choice for describing positions within a linked list because we cannot efficiently access an entry knowing only its index; finding an element at a given index within a linked list requires traversing the list incrementally from its beginning or end, counting elements as we go.

Furthermore, indices are not a good abstraction for describing a local position in some applications, because the index of an entry changes over time due to insertions or deletions that happen earlier in the sequence. For example, it may not be convenient to describe the location of a person waiting in line by knowing precisely how far away that person is from the front of the line. We prefer an abstraction, as characterized in Figure 7.14, in which there is some other means for describing a position. We then wish to model situations such as when an identified person leaves the line before reaching the front, or in which a new person is added to a line immediately behind another identified person.

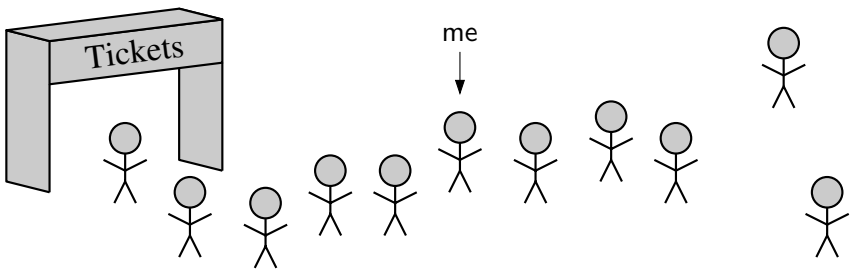


Figure 7.14: We wish to be able to identify the position of an element in a sequence without the use of an integer index.

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a ***cursor*** to describe a position within the document without explicit use of an integer index, allowing operations such as "delete the character at the cursor" or "insert a new character just after the cursor." Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular section, without relying on a character index (or even a section number) that may change as the document evolves.

A Node Reference as a Position?

One of the great benefits of a linked list structure is that it is possible to perform $O(1)$-time insertions and deletions at arbitrary positions of the list, as long as we are given a reference to a relevant node of the list. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our _DoublyLinkedBase class of Section 7.3.1 has methods _insert_between and _delete_node that accept node references as parameters.

However, such direct use of nodes would violate the object-oriented design principles of abstraction and encapsulation that were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a linked list, for both our sake and for the benefit of users of our abstraction.

- It will be simpler for users of our data structure if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the _insert_between method of our _DoublyLinkedBase class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data structure if we do not permit users to directly access or manipulate the nodes. In that way, we ensure that users cannot invalidate the consistency of a list by mismanaging the linking of nodes. A more subtle problem arises if a user were allowed to call the _insert_between or _delete_node method of our _DoublyLinkedBase class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)
- By better encapsulating the internal details of our implementation, we have greater flexibility to redesign the data structure and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a non-numeric position, even if using an array-based sequence.

For these reasons, instead of relying directly on nodes, we introduce an independent ***position*** abstraction to denote the location of an element within a list, and then a complete ***positional list ADT*** that can encapsulate a doubly linked list (or even an array-based sequence; see Exercise P-7.46).

7.4.1 The Positional List Abstract Data Type

To provide for a general abstraction of a sequence of elements with the ability to identify the location of an element, we define a ***positional list ADT*** as well as a simpler ***position*** abstract data type to describe a location within a list. A position acts as a marker or token within the broader positional list. A position p is unaffected by changes elsewhere in a list; the only way in which a position becomes invalid is if an explicit command is issued to delete it.

A position instance is a simple object, supporting only the following method:

p.element( ): Return the element stored at position p.

In the context of the positional list ADT, positions serve as parameters to some methods and as return values from other methods. In describing the behaviors of a positional list, we being by presenting the accessor methods supported by a list L:

L.first( ): Return the position of the first element of L, or None if L is empty.

L.last( ): Return the position of the last element of L, or None if L is empty.

L.before(p): Return the position of L immediately before position p, or None if p is the first position.

L.after(p): Return the position of L immediately after position p, or None if p is the last position.

L.is_empty( ): Return True if list L does not contain any elements.

len(L): Return the number of elements in the list.

iter(L): Return a forward iterator for the *elements* of the list. See Section 1.8 for discussion of iterators in Python.

The positional list ADT also includes the following ***update*** methods:

L.add_first(e): Insert a new element e at the front of L, returning the position of the new element.

L.add_last(e): Insert a new element e at the back of L, returning the position of the new element.

L.add_before(p, e): Insert a new element e just before position p in L, returning the position of the new element.

L.add_after(p, e): Insert a new element e just after position p in L, returning the position of the new element.

L.replace(p, e): Replace the element at position p with element e, returning the element formerly at position p.

L.delete(p): Remove and return the element at position p in L, invalidating the position.

For those methods of the ADT that accept a position p as a parameter, an error occurs if p is not a valid position for list L.

Note well that the first( ) and last( ) methods of the positional list ADT return the associated *positions*, not the *elements*. (This is in contrast to the corresponding first and last methods of the deque ADT.) The first element of a positional list can be determined by subsequently invoking the element method on that position, as L.first( ).element( ). The advantage of receiving a position as a return value is that we can use that position to navigate the list. For example, the following code fragment prints all elements of a positional list named data.

```
cursor = data.first( )
while cursor is not None:
  print(cursor.element( ))        # print the element stored at the position
  cursor = data.after(cursor)     # advance to the next position (if any)
```

This code relies on the stated convention that the None object is returned when after is called upon the last position. That return value is clearly distinguishable from any legitimate position. The positional list ADT similarly indicates that the None value is returned when the before method is invoked at the front of the list, or when first or last methods are called upon an empty list. Therefore, the above code fragment works correctly even if the data list is empty.

Because the ADT includes support for Python's iter function, users may rely on the traditional for-loop syntax for such a forward traversal of a list named data.

```
for e in data:
  print(e)
```

More general navigational and update methods of the positional list ADT are shown in the following example.

Example 7.1: *The following table shows a series of operations on an initially empty positional list L. To identify position instances, we use variables such as p and q. For ease of exposition, when displaying the list contents, we use subscript notation to denote its positions.*

Operation	Return Value	L
L.add_last(8)	p	8_p
L.first( )	p	8_p
L.add_after(p, 5)	q	$8_p, 5_q$
L.before(q)	p	$8_p, 5_q$
L.add_before(q, 3)	r	$8_p, 3_r, 5_q$
r.element( )	3	$8_p, 3_r, 5_q$
L.after(p)	r	$8_p, 3_r, 5_q$
L.before(p)	None	$8_p, 3_r, 5_q$
L.add_first(9)	s	$9_s, 8_p, 3_r, 5_q$
L.delete(L.last( ))	5	$9_s, 8_p, 3_r$
L.replace(p, 7)	8	$9_s, 7_p, 3_r$

7.4.2 Doubly Linked List Implementation

In this section, we present a complete implementation of a PositionalList class using a doubly linked list that satisfies the following important proposition.

Proposition 7.2: *Each method of the positional list ADT runs in worst-case $O(1)$ time when implemented with a doubly linked list.*

We rely on the _DoublyLinkedBase class from Section 7.3.1 for our low-level representation; the primary responsibility of our new class is to provide a public interface in accordance with the positional list ADT. We begin our class definition in Code Fragment 7.14 with the definition of the public Position class, nested within our PositionalList class. Position instances will be used to represent the locations of elements within the list. Our various PositionalList methods may end up creating redundant Position instances that reference the same underlying node (for example, when first and last are the same). For that reason, our Position class defines the __eq__ and __ne__ special methods so that a test such as p == q evaluates to True when two positions refer to the same node.

Validating Positions

Each time a method of the PositionalList class accepts a position as a parameter, we want to verify that the position is valid, and if so, to determine the underlying node associated with the position. This functionality is implemented by a non-public method named _validate. Internally, a position maintains a reference to the associated node of the linked list, and also a reference to the list instance that contains the specified node. With the container reference, we can robustly detect when a caller sends a position instance that does not belong to the indicated list.

We are also able to detect a position instance that belongs to the list, but that refers to a node that is no longer part of that list. Recall that the _delete_node of the base class sets the previous and next references of a deleted node to None; we can recognize that condition to detect a deprecated node.

Access and Update Methods

The access methods of the PositionalList class are given in Code Fragment 7.15 and the update methods are given in Code Fragment 7.16. All of these methods trivially adapt the underlying doubly linked list implementation to support the public interface of the positional list ADT. Those methods rely on the _validate utility to "unwrap" any position that is sent. They also rely on a _make_position utility to "wrap" nodes as Position instances to return to the user, making sure never to return a position referencing a sentinel. For convenience, we have overridden the inherited _insert_between utility method so that ours returns a *position* associated with the newly created node (whereas the inherited version returns the node itself).

```
class PositionalList(_DoublyLinkedBase):
  """A sequential container of elements allowing positional access."""

  #------------------------- nested Position class -------------------------
  class Position:
    """An abstraction representing the location of a single element."""

    def __init__(self, container, node):
      """Constructor should not be invoked by user."""
      self._container = container
      self._node = node

    def element(self):
      """Return the element stored at this Position."""
      return self._node._element

    def __eq__(self, other):
      """Return True if other is a Position representing the same location."""
      return type(other) is type(self) and other._node is self._node

    def __ne__(self, other):
      """Return True if other does not represent the same location."""
      return not (self == other)          # opposite of __eq__

  #------------------------------ utility method ------------------------------
  def _validate(self, p):
    """Return position's node, or raise appropriate error if invalid."""
    if not isinstance(p, self.Position):
      raise TypeError('p must be proper Position type')
    if p._container is not self:
      raise ValueError('p does not belong to this container')
    if p._node._next is None:             # convention for deprecated nodes
      raise ValueError('p is no longer valid')
    return p._node
```

Code Fragment 7.14: A PositionalList class based on a doubly linked list. (Continues in Code Fragments 7.15 and 7.16.)

```
35    #------------------------------ utility method ------------------------------
36    def _make_position(self, node):
37      """Return Position instance for given node (or None if sentinel)."""
38      if node is self._header or node is self._trailer:
39        return None                                  # boundary violation
40      else:
41        return self.Position(self, node)             # legitimate position
42
43    #------------------------------ accessors ------------------------------
44    def first(self):
45      """Return the first Position in the list (or None if list is empty)."""
46      return self._make_position(self._header._next)
47
48    def last(self):
49      """Return the last Position in the list (or None if list is empty)."""
50      return self._make_position(self._trailer._prev)
51
52    def before(self, p):
53      """Return the Position just before Position p (or None if p is first)."""
54      node = self._validate(p)
55      return self._make_position(node._prev)
56
57    def after(self, p):
58      """Return the Position just after Position p (or None if p is last)."""
59      node = self._validate(p)
60      return self._make_position(node._next)
61
62    def __iter__(self):
63      """Generate a forward iteration of the elements of the list."""
64      cursor = self.first( )
65      while cursor is not None:
66        yield cursor.element( )
67        cursor = self.after(cursor)
```

Code Fragment 7.15: A PositionalList class based on a doubly linked list. (Continued from Code Fragment 7.14; continues in Code Fragment 7.16.)

```
68    #------------------------------ mutators ------------------------------
69    # override inherited version to return Position, rather than Node
70    def _insert_between(self, e, predecessor, successor):
71      """Add element between existing nodes and return new Position."""
72      node = super( )._insert_between(e, predecessor, successor)
73      return self._make_position(node)
74
75    def add_first(self, e):
76      """Insert element e at the front of the list and return new Position."""
77      return self._insert_between(e, self._header, self._header._next)
78
79    def add_last(self, e):
80      """Insert element e at the back of the list and return new Position."""
81      return self._insert_between(e, self._trailer._prev, self._trailer)
82
83    def add_before(self, p, e):
84      """Insert element e into list before Position p and return new Position."""
85      original = self._validate(p)
86      return self._insert_between(e, original._prev, original)
87
88    def add_after(self, p, e):
89      """Insert element e into list after Position p and return new Position."""
90      original = self._validate(p)
91      return self._insert_between(e, original, original._next)
92
93    def delete(self, p):
94      """Remove and return the element at Position p."""
95      original = self._validate(p)
96      return self._delete_node(original)      # inherited method returns element
97
98    def replace(self, p, e):
99      """Replace the element at Position p with e.
100
101     Return the element formerly at Position p.
102     """
103     original = self._validate(p)
104     old_value = original._element           # temporarily store old element
105     original._element = e                   # replace with new element
106     return old_value                        # return the old element value
```

Code Fragment 7.16: A PositionalList class based on a doubly linked list. (Continued from Code Fragments 7.14 and 7.15.)

7.5 Sorting a Positional List

In Section 5.5.2, we introduced the ***insertion-sort*** algorithm, in the context of an array-based sequence. In this section, we develop an implementation that operates on a PositionalList, relying on the same high-level algorithm in which each element is placed relative to a growing collection of previously sorted elements.

We maintain a variable named marker that represents the rightmost position of the currently sorted portion of a list. During each pass, we consider the position just past the marker as the pivot and consider where the pivot's element belongs relative to the sorted portion; we use another variable, named walk, to move leftward from the marker, as long as there remains a preceding element with value larger than the pivot's. A typical configuration of these variables is diagrammed in Figure 7.15. A Python implementation of this strategy is given in Code 7.17.

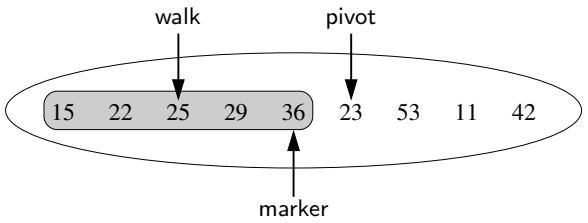


Figure 7.15: Overview of one step of our insertion-sort algorithm. The shaded elements, those up to and including marker, have already been sorted. In this step, the pivot's element should be relocated immediately before the walk position.

```
1  def insertion_sort(L):
2    """Sort PositionalList of comparable elements into nondecreasing order."""
3    if len(L) > 1:                          # otherwise, no need to sort it
4      marker = L.first( )
5      while marker != L.last( ):
6        pivot = L.after(marker)             # next item to place
7        value = pivot.element( )
8        if value > marker.element( ):       # pivot is already sorted
9          marker = pivot                    # pivot becomes new marker
10       else:                               # must relocate pivot
11         walk = marker                     # find leftmost item greater than value
12         while walk != L.first( ) and L.before(walk).element( ) > value:
13           walk = L.before(walk)
14         L.delete(pivot)
15         L.add_before(walk, value)         # reinsert value before walk
```

Code Fragment 7.17: Python code for performing insertion-sort on a positional list.

7.6 Case Study: Maintaining Access Frequencies

The positional list ADT is useful in a number of settings. For example, a program that simulates a game of cards could model each person's hand as a positional list (Exercise P-7.47). Since most people keep cards of the same suit together, inserting and removing cards from a person's hand could be implemented using the methods of the positional list ADT, with the positions being determined by a natural order of the suits. Likewise, a simple text editor embeds the notion of positional insertion and deletion, since such editors typically perform all updates relative to a ***cursor***, which represents the current position in the list of characters of text being edited.

In this section, we consider maintaining a collection of elements while keeping track of the number of times each element is accessed. Keeping such access counts allows us to know which elements are among the most popular. Examples of such scenarios include a Web browser that keeps track of a user's most accessed URLs, or a music collection that maintains a list of the most frequently played songs for a user. We model this with a new ***favorites list ADT*** that supports the len and is_empty methods as well as the following:

access(**e**): Access the element e, incrementing its access count, and adding it to the favorites list if it is not already present.

remove(**e**): Remove element e from the favorites list, if present.

top(**k**): Return an iteration of the k most accessed elements.

7.6.1 Using a Sorted List

Our first approach for managing a list of favorites is to store elements in a linked list, keeping them in nonincreasing order of access counts. We access or remove an element by searching the list from the most frequently accessed to the least frequently accessed. Reporting the top k most accessed elements is easy, as they are the first k entries of the list.

To maintain the invariant that elements are stored in nonincreasing order of access counts, we must consider how a single access operation may affect the order. The accessed element's count increases by one, and so it may become larger than one or more of its preceding neighbors in the list, thereby violating the invariant.

Fortunately, we can reestablish the sorted invariant using a technique similar to a single pass of the insertion-sort algorithm, introduced in the previous section. We can perform a backward traversal of the list, starting at the position of the element whose access count has increased, until we locate a valid position after which the element can be relocated.

Using the Composition Pattern

We wish to implement a favorites list by making use of a PositionalList for storage. If elements of the positional list were simply elements of the favorites list, we would be challenged to maintain access counts and to keep the proper count with the associated element as the contents of the list are reordered. We use a general object-oriented design pattern, the ***composition pattern***, in which we define a single object that is composed of two or more other objects. Specifically, we define a nonpublic nested class, _Item, that stores the element and its access count as a single instance. We then maintain our favorites list as a PositionalList of *item* instances, so that the access count for a user's element is embedded alongside it in our representation. (An _Item is never exposed to a user of a FavoritesList.)

```
1  class FavoritesList:
2    """List of elements ordered from most frequently accessed to least."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      __slots__ = '_value', '_count'              # streamline memory usage
7      def __init__(self, e):
8        self._value = e                           # the user's element
9        self._count = 0                           # access count initially zero
10
11   #------------------------------ nonpublic utilities ------------------------------
12   def _find_position(self, e):
13     """Search for element e and return its Position (or None if not found)."""
14     walk = self._data.first( )
15     while walk is not None and walk.element( )._value != e:
16       walk = self._data.after(walk)
17     return walk
18
19   def _move_up(self, p):
20     """Move item at Position p earlier in the list based on access count."""
21     if p != self._data.first( ):                  # consider moving...
22       cnt = p.element( )._count
23       walk = self._data.before(p)
24       if cnt > walk.element( )._count:           # must shift forward
25         while (walk != self._data.first( ) and
26                cnt > self._data.before(walk).element( )._count):
27           walk = self._data.before(walk)
28         self._data.add_before(walk, self._data.delete(p))     # delete/reinsert
```

Code Fragment 7.18: Class FavoritesList. (Continues in Code Fragment 7.19.)

```
29    #------------------------------ public methods ------------------------------
30    def __init__(self):
31      """Create an empty list of favorites."""
32      self._data = PositionalList( )                # will be list of _Item instances
33
34    def __len__(self):
35      """Return number of entries on favorites list."""
36      return len(self._data)
37
38    def is_empty(self):
39      """Return True if list is empty."""
40      return len(self._data) == 0
41
42    def access(self, e):
43      """Access element e, thereby increasing its access count."""
44      p = self._find_position(e)                    # try to locate existing element
45      if p is None:
46        p = self._data.add_last(self._Item(e))      # if new, place at end
47      p.element( )._count += 1                      # always increment count
48      self._move_up(p)                              # consider moving forward
49
50    def remove(self, e):
51      """Remove element e from the list of favorites."""
52      p = self._find_position(e)                    # try to locate existing element
53      if p is not None:
54        self._data.delete(p)                        # delete, if found
55
56    def top(self, k):
57      """Generate sequence of top k elements in terms of access count."""
58      if not 1 <= k <= len(self):
59        raise ValueError('Illegal value for k')
60      walk = self._data.first( )
61      for j in range(k):
62        item = walk.element( )                      # element of list is _Item
63        yield item._value                           # report user's element
64        walk = self._data.after(walk)
```

Code Fragment 7.19: Class FavoritesList. (Continued from Code Fragment 7.18.)

7.6.2 Using a List with the Move-to-Front Heuristic

The previous implementation of a favorites list performs the access(e) method in time proportional to the index of e in the favorites list. That is, if e is the kth most popular element in the favorites list, then accessing it takes $O(k)$ time. In many real-life access sequences (e.g., Web pages visited by a user), once an element is accessed it is more likely to be accessed again in the near future. Such scenarios are said to possess ***locality of reference***.

A ***heuristic***, or rule of thumb, that attempts to take advantage of the locality of reference that is present in an access sequence is the ***move-to-front heuristic***. To apply this heuristic, each time we access an element we move it all the way to the front of the list. Our hope, of course, is that this element will be accessed again in the near future. Consider, for example, a scenario in which we have n elements and the following series of n^2 accesses:

- element 1 is accessed n times
- element 2 is accessed n times
- $\cdots$
- element n is accessed n times.

If we store the elements sorted by their access counts, inserting each element the first time it is accessed, then

- each access to element 1 runs in $O(1)$ time
- each access to element 2 runs in $O(2)$ time
- $\cdots$
- each access to element n runs in $O(n)$ time.

Thus, the total time for performing the series of accesses is proportional to

$$n + 2n + 3n + \cdots + n \cdot n = n(1 + 2 + 3 + \cdots + n) = n \cdot \frac{n(n+1)}{2},$$

which is $O(n^3)$.

On the other hand, if we use the move-to-front heuristic, inserting each element the first time it is accessed, then

- each subsequent access to element 1 takes $O(1)$ time
- each subsequent access to element 2 takes $O(1)$ time
- $\cdots$
- each subsequent access to element n runs in $O(1)$ time.

So the running time for performing all the accesses in this case is $O(n^2)$. Thus, the move-to-front implementation has faster access times for this scenario. Still, the move-to-front approach is just a heuristic, for there are access sequences where using the move-to-front approach is slower than simply keeping the favorites list ordered by access counts.

The Trade-Offs with the Move-to-Front Heuristic

If we no longer maintain the elements of the favorites list ordered by their access counts, when we are asked to find the k most accessed elements, we need to search for them. We will implement the top(k) method as follows:

1. We copy all entries of our favorites list into another list, named temp.
2. We scan the temp list k times. In each scan, we find the entry with the largest access count, remove this entry from temp, and report it in the results.

This implementation of method top takes $O(kn)$ time. Thus, when k is a constant, method top runs in $O(n)$ time. This occurs, for example, when we want to get the "top ten" list. However, if k is proportional to n, then top runs in $O(n^2)$ time. This occurs, for example, when we want a "top 25%" list.

In Chapter 9 we will introduce a data structure that will allow us to implement top in $O(n + k \log n)$ time (see Exercise P-9.54), and more advanced techniques could be used to perform top in $O(n + k \log k)$ time.

We could easily achieve $O(n \log n)$ time if we use a standard sorting algorithm to reorder the temporary list before reporting the top k (see Chapter 12); this approach would be preferred to the original in the case that k is $\Omega(\log n)$. (Recall the big-Omega notation introduced in Section 3.3.1 to give an asymptotic lower bound on the running time of an algorithm.) There is a more specialized sorting algorithm (see Section 12.4.2) that can take advantage of the fact that access counts are integers in order to achieve $O(n)$ time for top, for any value of k.

Implementing the Move-to-Front Heuristic in Python

We give an implementation of a favorites list using the move-to-front heuristic in Code Fragment 7.20. The new FavoritesListMTF class inherits most of its functionality from the original FavoritesList as a base class.

By our original design, the access method of the original class relies on a non-public utility named _move_up to enact the potential shifting of an element forward in the list, after its access count had been incremented. Therefore, we implement the move-to-front heuristic by simply overriding the _move_up method so that each accessed element is moved directly to the front of the list (if not already there). This action is easily implemented by means of the positional list ADT.

The more complex portion of our FavoritesListMTF class is the new definition for the top method. We rely on the first of the approaches outlined above, inserting copies of the items into a temporary list and then repeatedly finding, reporting, and removing an element that has the largest access count of those remaining.

```
class FavoritesListMTF(FavoritesList):
  """List of elements ordered with move-to-front heuristic."""

  # we override _move_up to provide move-to-front semantics
  def _move_up(self, p):
    """Move accessed item at Position p to front of list."""
    if p != self._data.first( ):
      self._data.add_first(self._data.delete(p))        # delete/reinsert

  # we override top because list is no longer sorted
  def top(self, k):
    """Generate sequence of top k elements in terms of access count."""
    if not 1 <= k <= len(self):
      raise ValueError('Illegal value for k')

    # we begin by making a copy of the original list
    temp = PositionalList( )
    for item in self._data:                  # positional lists support iteration
      temp.add_last(item)

    # we repeatedly find, report, and remove element with largest count
    for j in range(k):
      # find and report next highest from temp
      highPos = temp.first( )
      walk = temp.after(highPos)
      while walk is not None:
        if walk.element( )._count > highPos.element( )._count:
          highPos = walk
        walk = temp.after(walk)
      # we have found the element with highest count
      yield highPos.element( )._value                # report element to user
      temp.delete(highPos)                           # remove from temp list
```

Code Fragment 7.20: Class FavoritesListMTF implementing the move-to-front heuristic. This class extends FavoritesList (Code Fragments 7.18 and 7.19) and overrides methods _move_up and top.

7.7 Link-Based vs. Array-Based Sequences

We close this chapter by reflecting on the relative pros and cons of array-based and link-based data structures that have been introduced thus far. The dichotomy between these approaches presents a common design decision when choosing an appropriate implementation of a data structure. There is not a one-size-fits-all solution, as each offers distinct advantages and disadvantages.

Advantages of Array-Based Sequences

- *Arrays provide $O(1)$-time access to an element based on an integer index.* The ability to access the kth element for any k in $O(1)$ time is a hallmark advantage of arrays (see Section 5.2). In contrast, locating the kth element in a linked list requires $O(k)$ time to traverse the list from the beginning, or possibly $O(n-k)$ time, if traversing backward from the end of a doubly linked list.
- *Operations with equivalent asymptotic bounds typically run a constant factor more efficiently with an array-based structure versus a linked structure.* As an example, consider the typical enqueue operation for a queue. Ignoring the issue of resizing an array, this operation for the ArrayQueue class (see Code Fragment 6.7) involves an arithmetic calculation of the new index, an increment of an integer, and storing a reference to the element in the array. In contrast, the process for a LinkedQueue (see Code Fragment 7.8) requires the instantiation of a node, appropriate linking of nodes, and an increment of an integer. While this operation completes in $O(1)$ time in either model, the actual number of CPU operations will be more in the linked version, especially given the instantiation of the new node.
- *Array-based representations typically use proportionally less memory than linked structures.* This advantage may seem counterintuitive, especially given that the length of a dynamic array may be longer than the number of elements that it stores. Both array-based lists and linked lists are referential structures, so the primary memory for storing the actual objects that are elements is the same for either structure. What differs is the auxiliary amounts of memory that are used by the two structures. For an array-based container of n elements, a typical worst case may be that a recently resized dynamic array has allocated memory for $2n$ object references. With linked lists, memory must be devoted not only to store a reference to each contained object, but also explicit references that link the nodes. So a singly linked list of length n already requires $2n$ references (an element reference and next reference for each node). With a doubly linked list, there are $3n$ references.

Advantages of Link-Based Sequences

- *Link-based structures provide worst-case time bounds for their operations.* This is in contrast to the amortized bounds associated with the expansion or contraction of a dynamic array (see Section 5.3).

  When many individual operations are part of a larger computation, and we only care about the total time of that computation, an amortized bound is as good as a worst-case bound precisely because it gives a guarantee on the sum of the time spent on the individual operations.

  However, if data structure operations are used in a real-time system that is designed to provide more immediate responses (e.g., an operating system, Web server, air traffic control system), a long delay caused by a single (amortized) operation may have an adverse effect.

- *Link-based structures support $O(1)$-time insertions and deletions at arbitrary positions.* The ability to perform a constant-time insertion or deletion with the PositionalList class, by using a Position to efficiently describe the location of the operation, is perhaps the most significant advantage of the linked list.

  This is in stark contrast to an array-based sequence. Ignoring the issue of resizing an array, inserting or deleting an element from the end of an array-based list can be done in constant time. However, more general insertions and deletions are expensive. For example, with Python's array-based list class, a call to insert or pop with index k uses $O(n-k+1)$ time because of the loop to shift all subsequent elements (see Section 5.4).

  As an example application, consider a text editor that maintains a document as a sequence of characters. Although users often add characters to the end of the document, it is also possible to use the cursor to insert or delete one or more characters at an arbitrary position within the document. If the character sequence were stored in an array-based sequence (such as a Python list), each such edit operation may require linearly many characters to be shifted, leading to $O(n)$ performance for each edit operation. With a linked-list representation, an arbitrary edit operation (insertion or deletion of a character at the cursor) can be performed in $O(1)$ worst-case time, assuming we are given a position that represents the location of the cursor.

7.8 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-7.1 Give an algorithm for finding the second-to-last node in a singly linked list in which the last node is indicated by a next reference of None.

R-7.2 Describe a good algorithm for concatenating two singly linked lists L and M, given only references to the first node of each list, into a single list L' that contains all the nodes of L followed by all the nodes of M.

R-7.3 Describe a recursive algorithm that counts the number of nodes in a singly linked list.

R-7.4 Describe in detail how to swap two nodes x and y (and not just their contents) in a singly linked list L given references only to x and y. Repeat this exercise for the case when L is a doubly linked list. Which algorithm takes more time?

R-7.5 Implement a function that counts the number of nodes in a circularly linked list.

R-7.6 Suppose that x and y are references to nodes of circularly linked lists, although not necessarily the same list. Describe a fast algorithm for telling if x and y belong to the same list.

R-7.7 Our CircularQueue class of Section 7.2.2 provides a rotate( ) method that has semantics equivalent to Q.enqueue(Q.dequeue( )), for a nonempty queue. Implement such a method for the LinkedQueue class of Section 7.1.2 without the creation of any new nodes.

R-7.8 Describe a nonrecursive method for finding, by link hopping, the middle node of a doubly linked list with header and trailer sentinels. In the case of an even number of nodes, report the node slightly left of center as the "middle." (Note: This method must only use link hopping; it cannot use a counter.) What is the running time of this method?

R-7.9 Give a fast algorithm for concatenating two doubly linked lists L and M, with header and trailer sentinel nodes, into a single list L'.

R-7.10 There seems to be some redundancy in the repertoire of the positional list ADT, as the operation L.add_first(e) could be enacted by the alternative L.add_before(L.first( ), e). Likewise, L.add_last(e) might be performed as L.add_after(L.last( ), e). Explain why the methods add_first and add_last are necessary.

R-7.11 Implement a function, with calling syntax max(L), that returns the maximum element from a PositionalList instance L containing comparable elements.

R-7.12 Redo the previously problem with max as a method of the PositionalList class, so that calling syntax L.max( ) is supported.

R-7.13 Update the PositionalList class to support an additional method find(e), which returns the position of the (first occurrence of) element e in the list (or None if not found).

R-7.14 Repeat the previous process using recursion. Your method should not contain any loops. How much space does your method use in addition to the space used for L?

R-7.15 Provide support for a __reversed__ method of the PositionalList class that is similar to the given __iter__, but that iterates the elements in *reversed* order.

R-7.16 Describe an implementation of the PositionalList methods add_last and add_before realized by using only methods in the set {is_empty, first, last, prev, next, add_after, and add_first}.

R-7.17 In the FavoritesListMTF class, we rely on public methods of the positional list ADT to move an element of a list at position p to become the first element of the list, while keeping the relative order of the remaining elements unchanged. Internally, that combination of operations causes one node to be removed and a new node to be inserted. Augment the PositionalList class to support a new method, move_to_front(p), that accomplishes this goal more directly, by relinking the existing node.

R-7.18 Given the set of element $\{a,b,c,d,e,f\}$ stored in a list, show the final state of the list, assuming we use the move-to-front heuristic and access the elements according to the following sequence: $(a,b,c,d,e,f,a,c,f,b,d,e)$.

R-7.19 Suppose that we have made kn total accesses to the elements in a list L of n elements, for some integer $k \geq 1$. What are the minimum and maximum number of elements that have been accessed fewer than k times?

R-7.20 Let L be a list of n items maintained according to the move-to-front heuristic. Describe a series of $O(n)$ accesses that will reverse L.

R-7.21 Suppose we have an n-element list L maintained according to the move-to-front heuristic. Describe a sequence of n^2 accesses that is guaranteed to take $\Omega(n^3)$ time to perform on L.

R-7.22 Implement a clear( ) method for the FavoritesList class that returns the list to empty.

R-7.23 Implement a reset_counts( ) method for the FavoritesList class that resets all elements' access counts to zero (while leaving the order of the list unchanged).

Creativity

C-7.24 Give a complete implementation of the stack ADT using a singly linked list that includes a header sentinel.

C-7.25 Give a complete implementation of the queue ADT using a singly linked list that includes a header sentinel.

C-7.26 Implement a method, concatenate(Q2) for the LinkedQueue class that takes all elements of LinkedQueue Q2 and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in Q2 being an empty queue.

C-7.27 Give a recursive implementation of a singly linked list class, such that an instance of a nonempty list stores its first element and a reference to a list of remaining elements.

C-7.28 Describe a fast recursive algorithm for reversing a singly linked list.

C-7.29 Describe in detail an algorithm for reversing a singly linked list L using only a constant amount of additional space and not using any recursion.

C-7.30 Exercise P-6.35 describes a LeakyStack abstraction. Implement that ADT using a ***singly*** linked list for storage.

C-7.31 Design a *forward list* ADT that abstracts the operations on a singly linked list, much as the positional list ADT abstracts the use of a doubly linked list. Implement a ForwardList class that supports such an ADT.

C-7.32 Design a circular positional list ADT that abstracts a circularly linked list in the same way that the positional list ADT abstracts a doubly linked list, with a notion of a designated "cursor" position within the list.

C-7.33 Modify the _DoublyLinkedBase class to include a reverse method that reverses the order of the list, yet without creating or destroying any nodes.

C-7.34 Modify the PositionalList class to support a method swap(p, q) that causes the underlying nodes referenced by positions p and q to be exchanged for each other. Relink the existing nodes; do not create any new nodes.

C-7.35 To implement the iter method of the PositionalList class, we relied on the convenience of Python's generator syntax and the yield statement. Give an alternative implementation of iter by designing a nested iterator class. (See Section 2.3.4 for discussion of iterators.)

C-7.36 Give a complete implementation of the positional list ADT using a doubly linked list that does not include any sentinel nodes.

C-7.37 Implement a function that accepts a PositionalList L of n integers sorted in nondecreasing order, and another value V, and determines in $O(n)$ time if there are two elements of L that sum precisely to V. The function should return a pair of positions of such elements, if found, or None otherwise.

C-7.38 There is a simple, but inefficient, algorithm, called ***bubble-sort***, for sorting a list L of n comparable elements. This algorithm scans the list $n-1$ times, where, in each scan, the algorithm compares the current element with the next one and swaps them if they are out of order. Implement a bubble_sort function that takes a positional list L as a parameter. What is the running time of this algorithm, assuming the positional list is implemented with a doubly linked list?

C-7.39 To better model a FIFO queue in which entries may be deleted before reaching the front, design a PositionalQueue class that supports the complete queue ADT, yet with enqueue returning a position instance and support for a new method, delete(p), that removes the element associated with position p from the queue. You may use the adapter design pattern (Section 6.1.2), using a PositionalList as your storage.

C-7.40 Describe an efficient method for maintaining a favorites list L, with move-to-front heuristic, such that elements that have not been accessed in the most recent n accesses are automatically purged from the list.

C-7.41 Exercise C-5.29 introduces the notion of a ***natural join*** of two databases. Describe and analyze an efficient algorithm for computing the natural join of a linked list A of n pairs and a linked list B of m pairs.

C-7.42 Write a Scoreboard class that maintains the top 10 scores for a game application using a singly linked list, rather than the array that was used in Section 5.5.1.

C-7.43 Describe a method for performing a ***card shuffle*** of a list of $2n$ elements, by converting it into two lists. A card shuffle is a permutation where a list L is cut into two lists, L_1 and L_2, where L_1 is the first half of L and L_2 is the second half of L, and then these two lists are merged into one by taking the first element in L_1, then the first element in L_2, followed by the second element in L_1, the second element in L_2, and so on.

Projects

P-7.44 Write a simple text editor that stores and displays a string of characters using the positional list ADT, together with a cursor object that highlights a position in this string. A simple interface is to print the string and then to use a second line of output to underline the position of the cursor. Your editor should support the following operations:

- left: Move cursor left one character (do nothing if at beginning).
- right: Move cursor right one character (do nothing if at end).
- insert c: Insert the character c just after the cursor.
- delete: Delete the character just after the cursor (do nothing at end).

P-7.45 An array A is ***sparse*** if most of its entries are empty (i.e., None). A list L can be used to implement such an array efficiently. In particular, for each nonempty cell $A[i]$, we can store an entry (i,e) in L, where e is the element stored at $A[i]$. This approach allows us to represent A using $O(m)$ storage, where m is the number of nonempty entries in A. Provide such a SparseArray class that minimally supports methods __getitem__(j) and __setitem__(j, e) to provide standard indexing operations. Analyze the efficiency of these methods.

P-7.46 Although we have used a doubly linked list to implement the positional list ADT, it is possible to support the ADT with an array-based implementation. The key is to use the composition pattern and store a sequence of position items, where each item stores an element as well as that element's current index in the array. Whenever an element's place in the array is changed, the recorded index in the position must be updated to match. Given a complete class providing such an array-based implementation of the positional list ADT. What is the efficiency of the various operations?

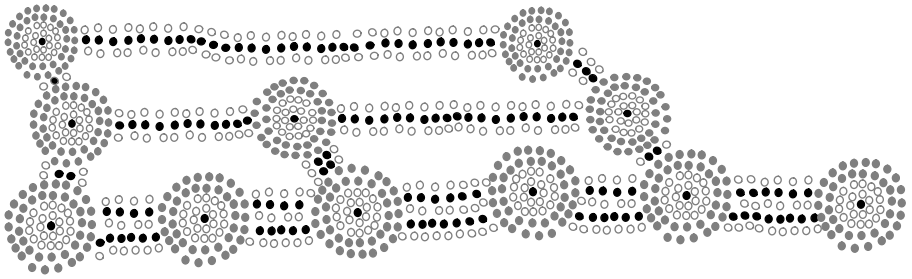
P-7.47 Implement a CardHand class that supports a person arranging a group of cards in his or her hand. The simulator should represent the sequence of cards using a single positional list ADT so that cards of the same suit are kept together. Implement this strategy by means of four "fingers" into the hand, one for each of the suits of hearts, clubs, spades, and diamonds, so that adding a new card to the person's hand or playing a correct card from the hand can be done in constant time. The class should support the following methods:

- add_card(r, s): Add a new card with rank r and suit s to the hand.
- play(s): Remove and return a card of suit s from the player's hand; if there is no card of suit s, then remove and return an arbitrary card from the hand.
- __iter__(): Iterate through all cards currently in the hand.
- all_of_suit(s): Iterate through all cards of suit s that are currently in the hand.

Chapter Notes

A view of data structures as collections (and other principles of object-oriented design) can be found in object-oriented design books by Booch [17], Budd [20], Goldberg and Robson [42], and Liskov and Guttag [71]. Our positional list ADT is derived from the "position" abstraction introduced by Aho, Hopcroft, and Ullman [6], and the list ADT of Wood [104]. Implementations of linked lists are discussed by Knuth [64].

Chapter 10 Maps, Hash Tables, and Skip Lists



Contents

10.1 Maps and Dictionaries 402
10.1.1 The Map ADT 403
10.1.2 Application: Counting Word Frequencies 405
10.1.3 Python's MutableMapping Abstract Base Class 406
10.1.4 Our MapBase Class 407
10.1.5 Simple Unsorted Map Implementation 408
10.2 Hash Tables 410
10.2.1 Hash Functions 411
10.2.2 Collision-Handling Schemes 417
10.2.3 Load Factors, Rehashing, and Efficiency 420
10.2.4 Python Hash Table Implementation 422
10.3 Sorted Maps 427
10.3.1 Sorted Search Tables 428
10.3.2 Two Applications of Sorted Maps 434
10.4 Skip Lists 437
10.4.1 Search and Update Operations in a Skip List 439
10.4.2 Probabilistic Analysis of Skip Lists ★ 443
10.5 Sets, Multisets, and Multimaps 446
10.5.1 The Set ADT 446
10.5.2 Python's MutableSet Abstract Base Class 448
10.5.3 Implementing Sets, Multisets, and Multimaps 450
10.6 Exercises 452

10.1 Maps and Dictionaries

Python's **dict** class is arguably the most significant data structure in the language. It represents an abstraction known as a ***dictionary*** in which unique ***keys*** are mapped to associated ***values***. Because of the relationship they express between keys and values, dictionaries are commonly known as ***associative arrays*** or ***maps***. In this book, we use the term *dictionary* when specifically discussing Python's dict class, and the term *map* when discussing the more general notion of the abstract data type.

As a simple example, Figure 10.1 illustrates a map from the names of countries to their associated units of currency.

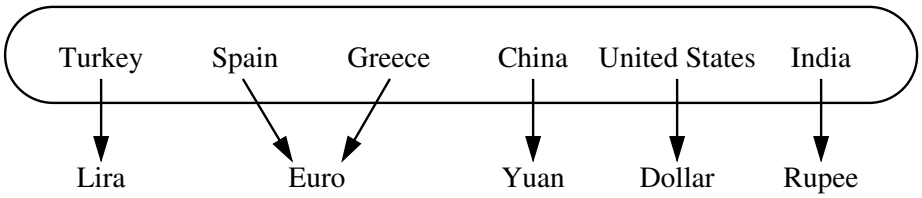


Figure 10.1: A map from countries (the keys) to their units of currency (the values).

We note that the keys (the country names) are assumed to be unique, but the values (the currency units) are not necessarily unique. For example, we note that Spain and Greece both use the euro for currency. Maps use an array-like syntax for indexing, such as currency['Greece'] to access a value associated with a given key or currency['Greece'] = 'Drachma' to remap it to a new value. Unlike a standard array, indices for a map need not be consecutive nor even numeric. Common applications of maps include the following.

- A university's information system relies on some form of a student ID as a key that is mapped to that student's associated record (such as the student's name, address, and course grades) serving as the value.
- The domain-name system (DNS) maps a host name, such as www.wiley.com, to an Internet-Protocol (IP) address, such as 208.215.179.146.
- A social media site typically relies on a (nonnumeric) username as a key that can be efficiently mapped to a particular user's associated information.
- A computer graphics system may map a color name, such as 'turquoise', to the triple of numbers that describes the color's RGB (red-green-blue) representation, such as (64,224,208).
- Python uses a dictionary to represent each namespace, mapping an identifying string, such as 'pi', to an associated object, such as 3.14159.

In this chapter and the next we demonstrate that a map may be implemented so that a search for a key, and its associated value, can be performed very efficiently, thereby supporting fast lookup in such applications.

10.1.1 The Map ADT

In this section, we introduce the ***map ADT***, and define its behaviors to be consistent with those of Python's built-in dict class. We begin by listing what we consider the most significant five behaviors of a map *M* as follows:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError. In Python, this is implemented with the special method __getitem__.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k. In Python, this is implemented with the special method __setitem__.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError. In Python, this is implemented with the special method __delitem__.

len(M): Return the number of items in map M. In Python, this is implemented with the special method __len__.

iter(M): The default iteration for a map generates a sequence of *keys* in the map. In Python, this is implemented with the special method __iter__, and it allows loops of the form, **for** k **in** M.

We have highlighted the above five behaviors because they demonstrate the core functionality of a map—namely, the ability to query, add, modify, or delete a key-value pair, and the ability to report all such pairs. For additional convenience, map *M* should also support the following behaviors:

k in M: Return True if the map contains an item with key k. In Python, this is implemented with the special __contains__ method.

M.get(k, d=None): Return M[k] if key k exists in the map; otherwise return default value d. This provides a form to query M[k] without risk of a KeyError.

M.setdefault(k, d): If key k exists in the map, simply return M[k]; if key k does not exist, set M[k] = d and return that value.

M.pop(k, d=None): Remove the item associated with key k from the map and return its associated value v. If key k is not in the map, return default value d (or raise KeyError if parameter d is None).

M.popitem( ): Remove an arbitrary key-value pair from the map, and return a (k,v) tuple representing the removed pair. If map is empty, raise a KeyError.

M.clear( ): Remove all key-value pairs from the map.

M.keys( ): Return a set-like view of all keys of M.

M.values( ): Return a set-like view of all values of M.

M.items( ): Return a set-like view of (k,v) tuples for all entries of M.

M.update(M2): Assign M[k] = v for every (k,v) pair in map M2.

M == M2: Return True if maps M and M2 have identical key-value associations.

M != M2: Return True if maps M and M2 do not have identical key-value associations.

Example 10.1: *In the following, we show the effect of a series of operations on an initially empty map storing items with integer keys and single-character values. We use the literal syntax for Python's dict class to describe the map contents.*

Operation	Return Value	Map
len(M)	0	{ }
M['K'] = 2	–	{'K': 2}
M['B'] = 4	–	{'K': 2, 'B': 4}
M['U'] = 2	–	{'K': 2, 'B': 4, 'U': 2}
M['V'] = 8	–	{'K': 2, 'B': 4, 'U': 2, 'V': 8}
M['K'] = 9	–	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['B']	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['X']	KeyError	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F')	None	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F', 5)	5	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('K', 5)	9	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
len(M)	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
del M['V']	–	{'K': 9, 'B': 4, 'U': 2}
M.pop('K')	9	{'B': 4, 'U': 2}
M.keys( )	'B', 'U'	{'B': 4, 'U': 2}
M.values( )	4, 2	{'B': 4, 'U': 2}
M.items( )	('B', 4), ('U', 2)	{'B': 4, 'U': 2}
M.setdefault('B', 1)	4	{'B': 4, 'U': 2}
M.setdefault('A', 1)	1	{'A': 1, 'B': 4, 'U': 2}
M.popitem( )	('B', 4)	{'A': 1, 'U': 2}

10.1.2 Application: Counting Word Frequencies

As a case study for using a map, consider the problem of counting the number of occurrences of words in a document. This is a standard task when performing a statistical analysis of a document, for example, when categorizing an email or news article. A map is an ideal data structure to use here, for we can use words as keys and word counts as values. We show such an application in Code Fragment 10.1.

We break apart the original document using a combination of file and string methods that results in a loop over a lowercased version of all whitespace separated pieces of the document. We omit all nonalphabetic characters so that parentheses, apostrophes, and other such punctuation are not considered part of a word.

In terms of map operations, we begin with an empty Python dictionary named freq. During the first phase of the algorithm, we execute the command

```
freq[word] = 1 + freq.get(word, 0)
```

for each word occurrence. We use the get method on the right-hand side because the current word might not exist in the dictionary; the default value of 0 is appropriate in that case.

During the second phase of the algorithm, after the full document has been processed, we examine the contents of the frequency map, looping over freq.items( ) to determine which word has the most occurrences.

```
1  freq = { }
2  for piece in open(filename).read( ).lower( ).split( ):
3    # only consider alphabetic characters within this piece
4    word = ''.join(c for c in piece if c.isalpha( ))
5    if word:    # require at least one alphabetic character
6      freq[word] = 1 + freq.get(word, 0)
7
8  max_word = ''
9  max_count = 0
10 for (w,c) in freq.items( ):        # (key, value) tuples represent (word, count)
11   if c > max_count:
12     max_word = w
13     max_count = c
14 print('The most frequent word is', max_word)
15 print('Its number of occurrences is', max_count)
```

Code Fragment 10.1: A program for counting word frequencies in a document, and reporting the most frequent word. We use Python's dict class for the map. We convert the input to lowercase and ignore any nonalphabetic characters.

10.1.3 Python's MutableMapping Abstract Base Class

Section 2.4.3 provides an introduction to the concept of an ***abstract base class*** and the role of such classes in Python's collections module. Methods that are declared to be abstract in such a base class must be implemented by concrete subclasses. However, an abstract base class may provide *concrete* implementation of other methods that depend upon use of the presumed abstract methods. (This is an example of the ***template method design pattern***.)

The collections module provides two abstract base classes that are relevant to our current discussion: the Mapping and MutableMapping classes. The Mapping class includes all nonmutating methods supported by Python's dict class, while the MutableMapping class extends that to include the mutating methods. What we define as the map ADT in Section 10.1.1 is akin to the MutableMapping abstract base class in Python's collections module.

The significance of these abstract base classes is that they provide a framework to assist in creating a user-defined map class. In particular, the MutableMapping class provides *concrete* implementations for all behaviors other than the first five outlined in Section 10.1.1: __getitem__, __setitem__, __delitem__, __len__, and __iter__. As we implement the map abstraction with various data structures, as long as we provide the five core behaviors, we can inherit all other derived behaviors by simply declaring MutableMapping as a parent class.

To better understand the MutableMapping class, we provide a few examples of how concrete behaviors can be derived from the five core abstractions. For example, the __contains__ method, supporting the syntax k in M, could be implemented by making a guarded attempt to retrieve self[k] to determine if the key exists.

```
def __contains__(self, k):
  try:
    self[k]                          # access via __getitem__ (ignore result)
    return True
  except KeyError:
    return False                     # attempt failed
```

A similar approach might be used to provide the logic of the setdefault method.

```
def setdefault(self, k, d):
  try:
    return self[k]                   # if __getitem__ succeeds, return value
  except KeyError:                   # otherwise:
    self[k] = d                      # set default value with __setitem__
    return d                         # and return that newly assigned value
```

We leave as exercises the implementations of the remaining concrete methods of the MutableMapping class.

10.1.4 Our MapBase Class

We will be providing many different implementations of the map ADT, in the remainder of this chapter and next, using a variety of data structures demonstrating a trade-off of advantages and disadvantages. Figure 10.2 provides a preview of those classes.

The MutableMapping abstract base class, from Python's collections module and discussed in the preceding pages, is a valuable tool when implementing a map. However, in the interest of greater code reuse, we define our own MapBase class, which is itself a subclass of the MutableMapping class. Our MapBase class provides additional support for the composition design pattern. This is a technique we introduced when implementing a priority queue (see Section 9.2.1) in order to group a key-value pair as a single instance for internal use.

More formally, our MapBase class is defined in Code Fragment 10.2, extending the existing MutableMapping abstract base class so that we inherit the many useful concrete methods that class provides. We then define a nonpublic nested _Item class, whose instances are able to store both a key and value. This nested class is reasonably similar in design to the _Item class that was defined within our PriorityQueueBase class in Section 9.2.1, except that for a map we provide support for both equality tests and comparisons, both of which rely on the item's key. The notion of equality is necessary for all of our map implementations, as a way to determine whether a key given as a parameter is equivalent to one that is already stored in the map. The notion of comparisons between keys, using the $<$ operator, will become relevant when we later introduce a ***sorted map ADT*** (Section 10.3).

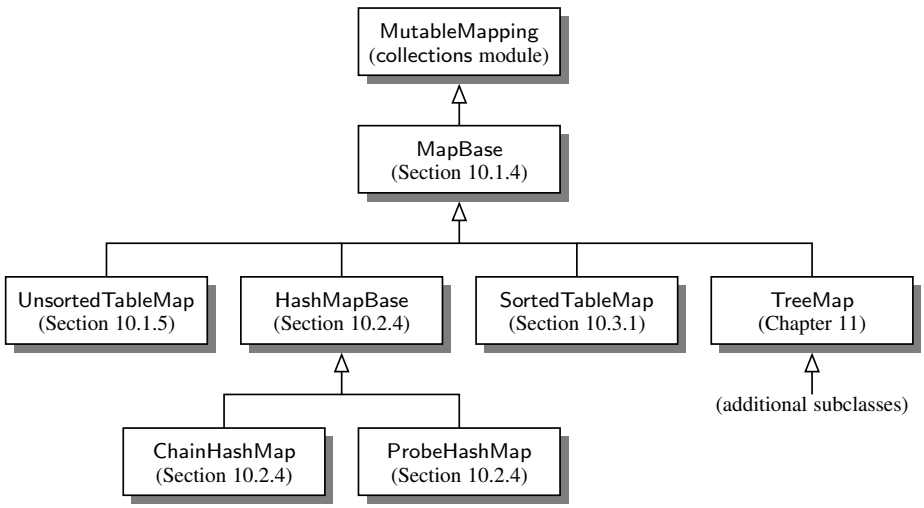


Figure 10.2: Our hierarchy of map types (with references to where they are defined).

```
1  class MapBase(MutableMapping):
2    """Our own abstract base class that includes a nonpublic _Item class."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      """Lightweight composite to store key-value pairs as map items."""
7      __slots__ = '_key', '_value'
8
9      def __init__(self, k, v):
10       self._key = k
11       self._value = v
12
13     def __eq__(self, other):
14       return self._key == other._key     # compare items based on their keys
15
16     def __ne__(self, other):
17       return not (self == other)         # opposite of __eq__
18
19     def __lt__(self, other):
20       return self._key < other._key      # compare items based on their keys
```

Code Fragment 10.2: Extending the MutableMapping abstract base class to provide a nonpublic _Item class for use in our various map implementations.

10.1.5 Simple Unsorted Map Implementation

We demonstrate the use of the MapBase class with a very simple concrete implementation of the map ADT. Code Fragment 10.3 presents an UnsortedTableMap class that relies on storing key-value pairs in arbitrary order within a Python list.

An empty table is initialized as self._table within the constructor for our map. When a new key is entered into the map, via line 22 of the __setitem__ method, we create a new instance of the nested _Item class, which is inherited from our MapBase class.

This list-based map implementation is simple, but it is not particularly efficient. Each of the fundamental methods, __getitem__, __setitem__, and __delitem__, relies on a for loop to scan the underlying list of items in search of a matching key. In a best-case scenario, such a match may be found near the beginning of the list, in which case the loop terminates; in the worst case, the entire list will be examined. Therefore, each of these methods runs in $O(n)$ time on a map with n items.

```
class UnsortedTableMap(MapBase):
  """Map implementation using an unordered list."""

  def __init__(self):
    """Create an empty map."""
    self._table = [ ]                                   # list of _Item's

  def __getitem__(self, k):
    """Return value associated with key k (raise KeyError if not found)."""
    for item in self._table:
      if k == item._key:
        return item._value
    raise KeyError('Key Error: ' + repr(k))

  def __setitem__(self, k, v):
    """Assign value v to key k, overwriting existing value if present."""
    for item in self._table:
      if k == item._key:                                # Found a match:
        item._value = v                                 # reassign value
        return                                          # and quit
    # did not find match for key
    self._table.append(self._Item(k,v))

  def __delitem__(self, k):
    """Remove item associated with key k (raise KeyError if not found)."""
    for j in range(len(self._table)):
      if k == self._table[j]._key:                      # Found a match:
        self._table.pop(j)                              # remove item
        return                                          # and quit
    raise KeyError('Key Error: ' + repr(k))

  def __len__(self):
    """Return number of items in the map."""
    return len(self._table)

  def __iter__(self):
    """Generate iteration of the map's keys."""
    for item in self._table:
      yield item._key                                   # yield the KEY
```

Code Fragment 10.3: An implementation of a map using a Python list as an unsorted table. Parent class MapBase is given in Code Fragment 10.2.

10.2 Hash Tables

In this section, we introduce one of the most practical data structures for implementing a map, and the one that is used by Python's own implementation of the dict class. This structure is known as a ***hash table***.

Intuitively, a map M supports the abstraction of using keys as indices with a syntax such as M[k]. As a mental warm-up, consider a restricted setting in which a map with n items uses keys that are known to be integers in a range from 0 to $N-1$ for some $N \geq n$. In this case, we can represent the map using a ***lookup table*** of length N, as diagrammed in Figure 10.3.

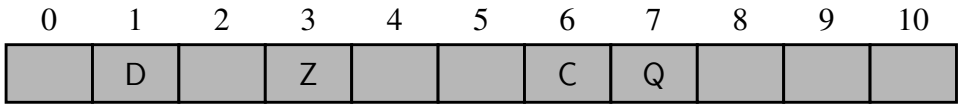


Figure 10.3: A lookup table with length 11 for a map containing items (1,D), (3,Z), (6,C), and (7,Q).

In this representation, we store the value associated with key k at index k of the table (presuming that we have a distinct way to represent an empty slot). Basic map operations of __getitem__, __setitem__, and __delitem__ can be implemented in $O(1)$ worst-case time.

There are two challenges in extending this framework to the more general setting of a map. First, we may not wish to devote an array of length N if it is the case that $N \gg n$. Second, we do not in general require that a map's keys be integers. The novel concept for a hash table is the use of a ***hash function*** to map general keys to corresponding indices in a table. Ideally, keys will be well distributed in the range from 0 to $N-1$ by a hash function, but in practice there may be two or more distinct keys that get mapped to the same index. As a result, we will conceptualize our table as a ***bucket array***, as shown in Figure 10.4, in which each bucket may manage a collection of items that are sent to a specific index by the hash function. (To save space, an empty bucket may be replaced by None.)

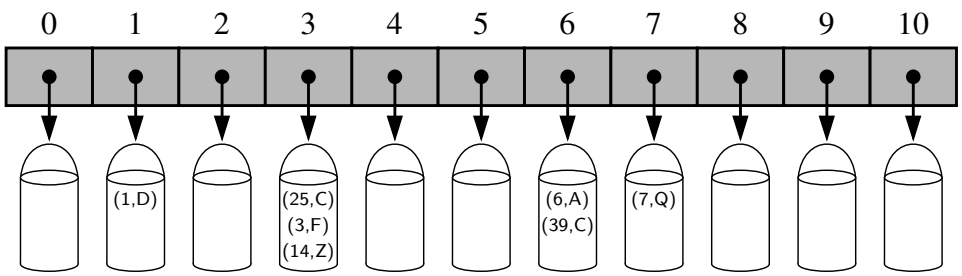


Figure 10.4: A bucket array of capacity 11 with items (1,D), (25,C), (3,F), (14,Z), (6,A), (39,C), and (7,Q), using a simple hash function.

10.2.1 Hash Functions

The goal of a ***hash function***, h, is to map each key k to an integer in the range $[0, N-1]$, where N is the capacity of the bucket array for a hash table. Equipped with such a hash function, h, the main idea of this approach is to use the hash function value, $h(k)$, as an index into our bucket array, A, instead of the key k (which may not be appropriate for direct use as an index). That is, we store the item (k, v) in the bucket $A[h(k)]$.

If there are two or more keys with the same hash value, then two different items will be mapped to the same bucket in A. In this case, we say that a ***collision*** has occurred. To be sure, there are ways of dealing with collisions, which we will discuss later, but the best strategy is to try to avoid them in the first place. We say that a hash function is "good" if it maps the keys in our map so as to sufficiently minimize collisions. For practical reasons, we also would like a hash function to be fast and easy to compute.

It is common to view the evaluation of a hash function, $h(k)$, as consisting of two portions—a ***hash code*** that maps a key k to an integer, and a ***compression function*** that maps the hash code to an integer within a range of indices, $[0, N-1]$, for a bucket array. (See Figure 10.5.)

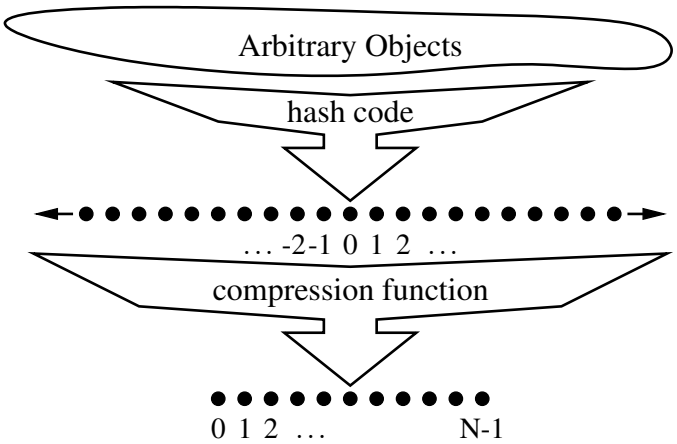


Figure 10.5: Two parts of a hash function: a hash code and a compression function.

The advantage of separating the hash function into two such components is that the hash code portion of that computation is independent of a specific hash table size. This allows the development of a general hash code for each object that can be used for a hash table of any size; only the compression function depends upon the table size. This is particularly convenient, because the underlying bucket array for a hash table may be dynamically resized, depending on the number of items currently stored in the map. (See Section 10.2.3.)

Hash Codes

The first action that a hash function performs is to take an arbitrary key k in our map and compute an integer that is called the ***hash code*** for k; this integer need not be in the range $[0, N-1]$, and may even be negative. We desire that the set of hash codes assigned to our keys should avoid collisions as much as possible. For if the hash codes of our keys cause collisions, then there is no hope for our compression function to avoid them. In this subsection, we begin by discussing the theory of hash codes. Following that, we discuss practical implementations of hash codes in Python.

Treating the Bit Representation as an Integer

To begin, we note that, for any data type X that is represented using at most as many bits as our integer hash codes, we can simply take as a hash code for X an integer interpretation of its bits. For example, the hash code for key 314 could simply be 314. The hash code for a floating-point number such as 3.14 could be based upon an interpretation of the bits of the floating-point representation as an integer.

For a type whose bit representation is longer than a desired hash code, the above scheme is not immediately applicable. For example, Python relies on 32-bit hash codes. If a floating-point number uses a 64-bit representation, its bits cannot be viewed directly as a hash code. One possibility is to use only the high-order 32 bits (or the low-order 32 bits). This hash code, of course, ignores half of the information present in the original key, and if many of the keys in our map only differ in these bits, then they will collide using this simple hash code.

A better approach is to combine in some way the high-order and low-order portions of a 64-bit key to form a 32-bit hash code, which takes all the original bits into consideration. A simple implementation is to add the two components as 32-bit numbers (ignoring overflow), or to take the exclusive-or of the two components. These approaches of combining components can be extended to any object x whose binary representation can be viewed as an n-tuple $(x_0, x_1, \ldots, x_{n-1})$ of 32-bit integers, for example, by forming a hash code for x as $\sum_{i=0}^{n-1} x_i$, or as $x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}$, where the $\oplus$ symbol represents the bitwise exclusive-or operation (which is ^ in Python).

Polynomial Hash Codes

The summation and exclusive-or hash codes, described above, are not good choices for character strings or other variable-length objects that can be viewed as tuples of the form $(x_0, x_1, \ldots, x_{n-1})$, where the order of the x_i's is significant. For example, consider a 16-bit hash code for a character string s that sums the Unicode values of the characters in s. This hash code unfortunately produces lots of unwanted

collisions for common groups of strings. In particular, "temp01" and "temp10" collide using this function, as do "stop", "tops", "pots", and "spot". A better hash code should somehow take into consideration the positions of the x_i's. An alternative hash code, which does exactly this, is to choose a nonzero constant, $a \neq 1$, and use as a hash code the value

$$x_0 a^{n-1} + x_1 a^{n-2} + \cdots + x_{n-2} a + x_{n-1}.$$

Mathematically speaking, this is simply a polynomial in a that takes the components $(x_0, x_1, \ldots, x_{n-1})$ of an object x as its coefficients. This hash code is therefore called a ***polynomial hash code***. By Horner's rule (see Exercise C-3.50), this polynomial can be computed as

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_2 + a(x_1 + a x_0)) \cdots)).$$

Intuitively, a polynomial hash code uses multiplication by different powers as a way to spread out the influence of each component across the resulting hash code.

Of course, on a typical computer, evaluating a polynomial will be done using the finite bit representation for a hash code; hence, the value will periodically overflow the bits used for an integer. Since we are more interested in a good spread of the object x with respect to other keys, we simply ignore such overflows. Still, we should be mindful that such overflows are occurring and choose the constant a so that it has some nonzero, low-order bits, which will serve to preserve some of the information content even as we are in an overflow situation.

We have done some experimental studies that suggest that 33, 37, 39, and 41 are particularly good choices for a when working with character strings that are English words. In fact, in a list of over 50,000 English words formed as the union of the word lists provided in two variants of Unix, we found that taking a to be 33, 37, 39, or 41 produced less than 7 collisions in each case!

Cyclic-Shift Hash Codes

A variant of the polynomial hash code replaces multiplication by a with a cyclic shift of a partial sum by a certain number of bits. For example, a 5-bit cyclic shift of the 32-bit value 00111011001011010101000010101000 is achieved by taking the leftmost five bits and placing those on the rightmost side of the representation, resulting in 10110010110101010001010100000111. While this operation has little natural meaning in terms of arithmetic, it accomplishes the goal of varying the bits of the calculation. In Python, a cyclic shift of bits can be accomplished through careful use of the bitwise operators $<<$ and $>>$, taking care to truncate results to 32-bit integers.

An implementation of a cyclic-shift hash code computation for a character string in Python appears as follows:

```
def hash_code(s):
  mask = (1 << 32) - 1                # limit to 32-bit integers
  h = 0
  for character in s:
    h = (h << 5 & mask) | (h >> 27)   # 5-bit cyclic shift of running sum
    h += ord(character)               # add in value of next character
  return h
```

As with the traditional polynomial hash code, fine-tuning is required when using a cyclic-shift hash code, as we must wisely choose the amount to shift by for each new character. Our choice of a 5-bit shift is justified by experiments run on a list of just over 230,000 English words, comparing the number of collisions for various shift amounts (see Table 10.1).

	Collisions	
Shift	**Total**	**Max**
0	234735	623
1	165076	43
2	38471	13
3	7174	5
4	1379	3
5	190	3
6	502	2
7	560	2
8	5546	4
9	393	3
10	5194	5
11	11559	5
12	822	2
13	900	4
14	2001	4
15	19251	8
16	211781	37

Table 10.1: Comparison of collision behavior for the cyclic-shift hash code as applied to a list of 230,000 English words. The "Total" column records the total number of words that collide with at least one other, and the "Max" column records the maximum number of words colliding at any one hash code. Note that with a cyclic shift of 0, this hash code reverts to the one that simply sums all the characters.

Hash Codes in Python

The standard mechanism for computing hash codes in Python is a built-in function with signature hash(x) that returns an integer value that serves as the hash code for object x. However, only ***immutable*** data types are deemed hashable in Python. This restriction is meant to ensure that a particular object's hash code remains constant during that object's lifespan. This is an important property for an object's use as a key in a hash table. A problem could occur if a key were inserted into the hash table, yet a later search were performed for that key based on a different hash code than that which it had when inserted; the wrong bucket would be searched.

Among Python's built-in data types, the immutable int, float, str, tuple, and frozenset classes produce robust hash codes, via the hash function, using techniques similar to those discussed earlier in this section. Hash codes for character strings are well crafted based on a technique similar to polynomial hash codes, except using exclusive-or computations rather than additions. If we repeat the experiment described in Table 10.1 using Python's built-in hash codes, we find that only 8 strings out of the set of more than 230,000 collide with another. Hash codes for tuples are computed with a similar technique based upon a combination of the hash codes of the individual elements of the tuple. When hashing a frozenset, the order of the elements should be irrelevant, and so a natural option is to compute the exclusive-or of the individual hash codes without any shifting. If hash(x) is called for an instance x of a mutable type, such as a list, a TypeError is raised.

Instances of user-defined classes are treated as unhashable by default, with a TypeError raised by the hash function. However, a function that computes hash codes can be implemented in the form of a special method named __hash__ within a class. The returned hash code should reflect the immutable attributes of an instance. It is common to return a hash code that is itself based on the computed hash of the combination of such attributes. For example, a Color class that maintains three numeric red, green, and blue components might implement the method as:

```
def __hash__(self):
  return hash( (self._red, self._green, self._blue) )   # hash combined tuple
```

An important rule to obey is that if a class defines equivalence through __eq__, then any implementation of __hash__ must be consistent, in that if x == y, then hash(x) == hash(y). This is important because if two instances are considered to be equivalent and one is used as a key in a hash table, a search for the second instance should result in the discovery of the first. It is therefore important that the hash code for the second match the hash code for the first, so that the proper bucket is examined. This rule extends to any well-defined comparisons between objects of different classes. For example, since Python treats the expression 5 == 5.0 as true, it ensures that hash(5) and hash(5.0) are the same.

Compression Functions

The hash code for a key k will typically not be suitable for immediate use with a bucket array, because the integer hash code may be negative or may exceed the capacity of the bucket array. Thus, once we have determined an integer hash code for a key object k, there is still the issue of mapping that integer into the range $[0, N-1]$. This computation, known as a ***compression function***, is the second action performed as part of an overall hash function. A good compression function is one that minimizes the number of collisions for a given set of distinct hash codes.

The Division Method

A simple compression function is the ***division method***, which maps an integer i to

$$i \bmod N,$$

where N, the size of the bucket array, is a fixed positive integer. Additionally, if we take N to be a prime number, then this compression function helps "spread out" the distribution of hashed values. Indeed, if N is not prime, then there is greater risk that patterns in the distribution of hash codes will be repeated in the distribution of hash values, thereby causing collisions. For example, if we insert keys with hash codes $\{200, 205, 210, 215, 220, \ldots, 600\}$ into a bucket array of size 100, then each hash code will collide with three others. But if we use a bucket array of size 101, then there will be no collisions. If a hash function is chosen well, it should ensure that the probability of two different keys getting hashed to the same bucket is $1/N$. Choosing N to be a prime number is not always enough, however, for if there is a repeated pattern of hash codes of the form $pN + q$ for several different p's, then there will still be collisions.

The MAD Method

A more sophisticated compression function, which helps eliminate repeated patterns in a set of integer keys, is the ***Multiply-Add-and-Divide*** (or "MAD") method. This method maps an integer i to

$$[(ai + b) \bmod p] \bmod N,$$

where N is the size of the bucket array, p is a prime number larger than N, and a and b are integers chosen at random from the interval $[0, p-1]$, with $a > 0$. This compression function is chosen in order to eliminate repeated patterns in the set of hash codes and get us closer to having a "good" hash function, that is, one such that the probability any two different keys collide is $1/N$. This good behavior would be the same as we would have if these keys were "thrown" into A uniformly at random.

10.2.2 Collision-Handling Schemes

The main idea of a hash table is to take a bucket array, A, and a hash function, h, and use them to implement a map by storing each item (k, v) in the "bucket" $A[h(k)]$. This simple idea is challenged, however, when we have two distinct keys, k_1 and k_2, such that $h(k_1) = h(k_2)$. The existence of such ***collisions*** prevents us from simply inserting a new item (k, v) directly into the bucket $A[h(k)]$. It also complicates our procedure for performing insertion, search, and deletion operations.

Separate Chaining

A simple and efficient way for dealing with collisions is to have each bucket $A[j]$ store its own secondary container, holding items (k, v) such that $h(k) = j$. A natural choice for the secondary container is a small map instance implemented using a list, as described in Section 10.1.5. This ***collision resolution*** rule is known as ***separate chaining***, and is illustrated in Figure 10.6.

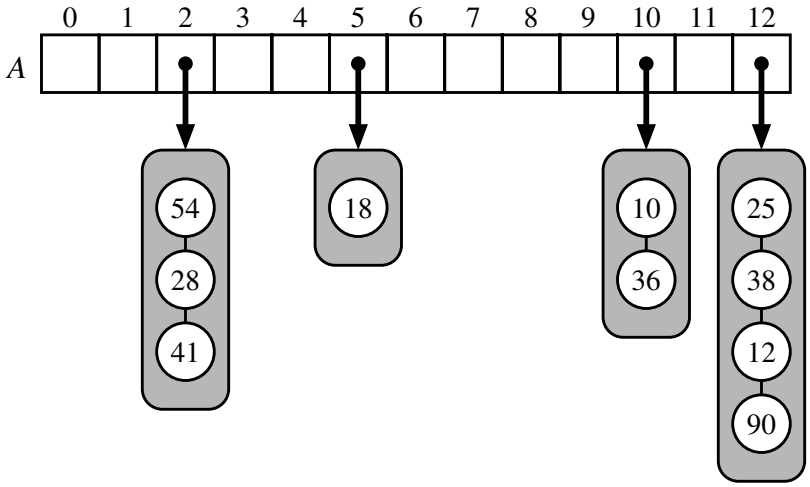


Figure 10.6: A hash table of size 13, storing 10 items with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

In the worst case, operations on an individual bucket take time proportional to the size of the bucket. Assuming we use a good hash function to index the n items of our map in a bucket array of capacity N, the expected size of a bucket is n/N. Therefore, if given a good hash function, the core map operations run in $O(\lceil n/N \rceil)$. The ratio $\lambda = n/N$, called the ***load factor*** of the hash table, should be bounded by a small constant, preferably below 1. As long as λ is $O(1)$, the core operations on the hash table run in $O(1)$ expected time.

Open Addressing

The separate chaining rule has many nice properties, such as affording simple implementations of map operations, but it nevertheless has one slight disadvantage: It requires the use of an auxiliary data structure—a list—to hold items with colliding keys. If space is at a premium (for example, if we are writing a program for a small handheld device), then we can use the alternative approach of always storing each item directly in a table slot. This approach saves space because no auxiliary structures are employed, but it requires a bit more complexity to deal with collisions. There are several variants of this approach, collectively referred to as ***open addressing*** schemes, which we discuss next. Open addressing requires that the load factor is always at most 1 and that items are stored directly in the cells of the bucket array itself.

Linear Probing and Its Variants

A simple method for collision handling with open addressing is ***linear probing***. With this approach, if we try to insert an item (k, v) into a bucket $A[j]$ that is already occupied, where $j = h(k)$, then we next try $A[(j+1) \bmod N]$. If $A[(j+1) \bmod N]$ is also occupied, then we try $A[(j+2) \bmod N]$, and so on, until we find an empty bucket that can accept the new item. Once this bucket is located, we simply insert the item there. Of course, this collision resolution strategy requires that we change the implementation when searching for an existing key—the first step of all `__getitem__`, `__setitem__`, or `__delitem__` operations. In particular, to attempt to locate an item with key equal to k, we must examine consecutive slots, starting from $A[h(k)]$, until we either find an item with that key or we find an empty bucket. (See Figure 10.7.) The name "linear probing" comes from the fact that accessing a cell of the bucket array can be viewed as a "probe."

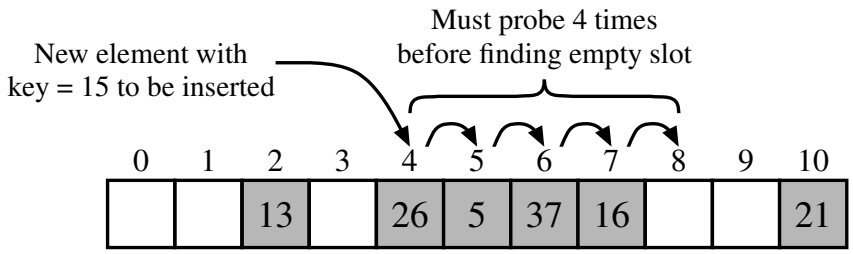


Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

To implement a deletion, we cannot simply remove a found item from its slot in the array. For example, after the insertion of key 15 portrayed in Figure 10.7, if the item with key 37 were trivially deleted, a subsequent search for 15 would fail because that search would start by probing at index 4, then index 5, and then index 6, at which an empty cell is found. A typical way to get around this difficulty is to replace a deleted item with a special "available" marker object. With this special marker possibly occupying spaces in our hash table, we modify our search algorithm so that the search for a key k will skip over cells containing the available marker and continue probing until reaching the desired item or an empty bucket (or returning back to where we started from). Additionally, our algorithm for `__setitem__` should remember an available cell encountered during the search for k, since this is a valid place to put a new item (k, v), if no existing item is found.

Although use of an open addressing scheme can save space, linear probing suffers from an additional disadvantage. It tends to cluster the items of a map into contiguous runs, which may even overlap (particularly if more than half of the cells in the hash table are occupied). Such contiguous runs of occupied hash cells cause searches to slow down considerably.

Another open addressing strategy, known as ***quadratic probing***, iteratively tries the buckets $A[(h(k) + f(i)) \bmod N]$, for $i = 0, 1, 2, \ldots$, where $f(i) = i^2$, until finding an empty bucket. As with linear probing, the quadratic probing strategy complicates the removal operation, but it does avoid the kinds of clustering patterns that occur with linear probing. Nevertheless, it creates its own kind of clustering, called ***secondary clustering***, where the set of filled array cells still has a non-uniform pattern, even if we assume that the original hash codes are distributed uniformly. When N is prime and the bucket array is less than half full, the quadratic probing strategy is guaranteed to find an empty slot. However, this guarantee is not valid once the table becomes at least half full, or if N is not chosen as a prime number; we explore the cause of this type of clustering in an exercise (C-10.36).

An open addressing strategy that does not cause clustering of the kind produced by linear probing or the kind produced by quadratic probing is the ***double hashing*** strategy. In this approach, we choose a secondary hash function, h', and if h maps some key k to a bucket $A[h(k)]$ that is already occupied, then we iteratively try the buckets $A[(h(k) + f(i)) \bmod N]$ next, for $i = 1, 2, 3, \ldots$, where $f(i) = i \cdot h'(k)$. In this scheme, the secondary hash function is not allowed to evaluate to zero; a common choice is $h'(k) = q - (k \bmod q)$, for some prime number $q < N$. Also, N should be a prime.

Another approach to avoid clustering with open addressing is to iteratively try buckets $A[(h(k) + f(i)) \bmod N]$ where $f(i)$ is based on a pseudo-random number generator, providing a repeatable, but somewhat arbitrary, sequence of subsequent probes that depends upon bits of the original hash code. This is the approach currently used by Python's dictionary class.

10.2.3 Load Factors, Rehashing, and Efficiency

In the hash table schemes described thus far, it is important that the load factor, $\lambda = n/N$, be kept below 1. With separate chaining, as λ gets very close to 1, the probability of a collision greatly increases, which adds overhead to our operations, since we must revert to linear-time list-based methods in buckets that have collisions. Experiments and average-case analyses suggest that we should maintain $\lambda < 0.9$ for hash tables with separate chaining.

With open addressing, on the other hand, as the load factor λ grows beyond 0.5 and starts approaching 1, clusters of entries in the bucket array start to grow as well. These clusters cause the probing strategies to "bounce around" the bucket array for a considerable amount of time before they find an empty slot. In Exercise C-10.36, we explore the degradation of quadratic probing when $\lambda \geq 0.5$. Experiments suggest that we should maintain $\lambda < 0.5$ for an open addressing scheme with linear probing, and perhaps only a bit higher for other open addressing schemes (for example, Python's implementation of open addressing enforces that $\lambda < 2/3$).

If an insertion causes the load factor of a hash table to go above the specified threshold, then it is common to resize the table (to regain the specified load factor) and to reinsert all objects into this new table. Although we need not define a new hash code for each object, we do need to reapply a new compression function that takes into consideration the size of the new table. Each ***rehashing*** will generally scatter the items throughout the new bucket array. When rehashing to a new table, it is a good requirement for the new array's size to be at least double the previous size. Indeed, if we always double the size of the table with each rehashing operation, then we can amortize the cost of rehashing all the entries in the table against the time used to insert them in the first place (as with dynamic arrays; see Section 5.3).

Efficiency of Hash Tables

Although the details of the average-case analysis of hashing are beyond the scope of this book, its probabilistic basis is quite intuitive. If our hash function is good, then we expect the entries to be uniformly distributed in the N cells of the bucket array. Thus, to store n entries, the expected number of keys in a bucket would be $\lceil n/N \rceil$, which is $O(1)$ if n is $O(N)$.

The costs associated with a periodic rehashing, to resize a table after occasional insertions or deletions can be accounted for separately, leading to an additional $O(1)$ amortized cost for `__setitem__` and `__getitem__`.

In the worst case, a poor hash function could map every item to the same bucket. This would result in linear-time performance for the core map operations with separate chaining, or with any open addressing model in which the secondary sequence of probes depends only on the hash code. A summary of these costs is given in Table 10.2.

Operation	List	Hash Table	
		expected	worst case
__getitem__	$O(n)$	$O(1)$	$O(n)$
__setitem__	$O(n)$	$O(1)$	$O(n)$
__delitem__	$O(n)$	$O(1)$	$O(n)$
__len__	$O(1)$	$O(1)$	$O(1)$
__iter__	$O(n)$	$O(n)$	$O(n)$

Table 10.2: Comparison of the running times of the methods of a map realized by means of an unsorted list (as in Section 10.1.5) or a hash table. We let n denote the number of items in the map, and we assume that the bucket array supporting the hash table is maintained such that its capacity is proportional to the number of items in the map.

In practice, hash tables are among the most efficient means for implementing a map, and it is essentially taken for granted by programmers that their core operations run in constant time. Python's dict class is implemented with hashing, and the Python interpreter relies on dictionaries to retrieve an object that is referenced by an identifier in a given namespace. (See Sections 1.10 and 2.5.) The basic command c = a + b involves two calls to __getitem__ in the dictionary for the local namespace to retrieve the values identified as a and b, and a call to __setitem__ to store the result associated with name c in that namespace. In our own algorithm analysis, we simply presume that such dictionary operations run in constant time, independent of the number of entries in the namespace. (Admittedly, the number of entries in a typical namespace can almost surely be bounded by a constant.)

In a 2003 academic paper [31], researchers discuss the possibility of exploiting a hash table's worst-case performance to cause a denial-of-service (DoS) attack of Internet technologies. For many published algorithms that compute hash codes, they note that an attacker could precompute a very large number of moderate-length strings that all hash to the identical 32-bit hash code. (Recall that by any of the hashing schemes we describe, other than double hashing, if two keys are mapped to the same hash code, they will be inseparable in the collision resolution.)

In late 2011, another team of researchers demonstrated an implementation of just such an attack [61]. Web servers allow a series of key-value parameters to be embedded in a URL using a syntax such as `?key1=val1&key2=val2&key3=val3`. Typically, those key-value pairs are immediately stored in a map by the server, and a limit is placed on the length and number of such parameters presuming that storage time in the map will be linear in the number of entries. If all keys were to collide, that storage requires quadratic time (causing the server to perform an inordinate amount of work). In spring of 2012, Python developers distributed a security patch that introduces randomization into the computation of hash codes for strings, making it less tractable to reverse engineer a set of colliding strings.

10.2.4 Python Hash Table Implementation

In this section, we develop two implementations of a hash table, one using separate chaining and the other using open addressing with linear probing. While these approaches to collision resolution are quite different, there are a great many commonalities to the hashing algorithms. For that reason, we extend the MapBase class (from Code Fragment 10.2), to define a new HashMapBase class (see Code Fragment 10.4), providing much of the common functionality to our two hash table implementations. The main design elements of the HashMapBase class are:

- The bucket array is represented as a Python list, named self._table, with all entries initialized to None.
- We maintain an instance variable self._n that represents the number of distinct items that are currently stored in the hash table.
- If the load factor of the table increases beyond 0.5, we double the size of the table and rehash all items into the new table.
- We define a _hash_function utility method that relies on Python's built-in hash function to produce hash codes for keys, and a randomized Multiply-Add-and-Divide (MAD) formula for the compression function.

What is not implemented in the base class is any notion of how a "bucket" should be represented. With separate chaining, each bucket will be an independent structure. With open addressing, however, there is no tangible container for each bucket; the "buckets" are effectively interleaved due to the probing sequences.

In our design, the HashMapBase class presumes the following to be abstract methods, which must be implemented by each concrete subclass:

- _bucket_getitem(j, k)
  This method should search bucket j for an item having key k, returning the associated value, if found, or else raising a KeyError.
- _bucket_setitem(j, k, v)
  This method should modify bucket j so that key k becomes associated with value v. If the key already exists, the new value overwrites the existing value. Otherwise, a new item is inserted and *this method is responsible for incrementing* self._n.
- _bucket_delitem(j, k)
  This method should remove the item from bucket j having key k, or raise a KeyError if no such item exists. (self._n is decremented *after* this method.)
- __iter__
  This is the standard map method to iterate through all keys of the map. Our base class does not delegate this on a per-bucket basis because "buckets" in open addressing are not inherently disjoint.

```
1  class HashMapBase(MapBase):
2    """Abstract base class for map using hash-table with MAD compression."""
3
4    def __init__(self, cap=11, p=109345121):
5      """Create an empty hash-table map."""
6      self._table = cap * [ None ]
7      self._n = 0                                 # number of entries in the map
8      self._prime = p                             # prime for MAD compression
9      self._scale = 1 + randrange(p-1)            # scale from 1 to p-1 for MAD
10     self._shift = randrange(p)                  # shift from 0 to p-1 for MAD
11
12   def _hash_function(self, k):
13     return (hash(k)*self._scale + self._shift) % self._prime % len(self._table)
14
15   def __len__(self):
16     return self._n
17
18   def __getitem__(self, k):
19     j = self._hash_function(k)
20     return self._bucket_getitem(j, k)           # may raise KeyError
21
22   def __setitem__(self, k, v):
23     j = self._hash_function(k)
24     self._bucket_setitem(j, k, v)               # subroutine maintains self._n
25     if self._n > len(self._table) // 2:         # keep load factor <= 0.5
26       self._resize(2 * len(self._table) - 1)    # number 2^x - 1 is often prime
27
28   def __delitem__(self, k):
29     j = self._hash_function(k)
30     self._bucket_delitem(j, k)                  # may raise KeyError
31     self._n -= 1
32
33   def _resize(self, c):                 # resize bucket array to capacity c
34     old = list(self.items())            # use iteration to record existing items
35     self._table = c * [None]            # then reset table to desired capacity
36     self._n = 0                         # n recomputed during subsequent adds
37     for (k,v) in old:
38       self[k] = v                       # reinsert old key-value pair
```

Code Fragment 10.4: A base class for our hash table implementations, extending our MapBase class from Code Fragment 10.2.

Separate Chaining

Code Fragment 10.5 provides a concrete implementation of a hash table with separate chaining, in the form of the ChainHashMap class. To represent a single bucket, it relies on an instance of the UnsortedTableMap class from Code Fragment 10.3.

The first three methods in the class use index j to access the potential bucket in the bucket array, and a check for the special case in which that table entry is None. The only time we need a new bucket structure is when _bucket_setitem is called on an otherwise empty slot. The remaining functionality relies on map behaviors that are already supported by the individual UnsortedTableMap instances. We need a bit of forethought to determine whether the application of __setitem__ on the chain causes a net increase in the size of the map (that is, whether the given key is new).

```
1  class ChainHashMap(HashMapBase):
2    """Hash map implemented with separate chaining for collision resolution."""
3
4    def _bucket_getitem(self, j, k):
5      bucket = self._table[j]
6      if bucket is None:
7        raise KeyError('Key Error: ' + repr(k))          # no match found
8      return bucket[k]                                   # may raise KeyError
9
10   def _bucket_setitem(self, j, k, v):
11     if self._table[j] is None:
12       self._table[j] = UnsortedTableMap( )             # bucket is new to the table
13     oldsize = len(self._table[j])
14     self._table[j][k] = v
15     if len(self._table[j]) > oldsize:                  # key was new to the table
16       self._n += 1                                     # increase overall map size
17
18   def _bucket_delitem(self, j, k):
19     bucket = self._table[j]
20     if bucket is None:
21       raise KeyError('Key Error: ' + repr(k))          # no match found
22     del bucket[k]                                      # may raise KeyError
23
24   def __iter__(self):
25     for bucket in self._table:
26       if bucket is not None:                           # a nonempty slot
27         for key in bucket:
28           yield key
```

Code Fragment 10.5: Concrete hash map class with separate chaining.

Linear Probing

Our implementation of a ProbeHashMap class, using open addressing with linear probing, is given in Code Fragments 10.6 and 10.7. In order to support deletions, we use a technique described in Section 10.2.2 in which we place a special marker in a table location at which an item has been deleted, so that we can distinguish between it and a location that has always been empty. In our implementation, we declare a class-level attribute, _AVAIL, as a sentinel. (We use an instance of the built-in object class because we do not care about any behaviors of the sentinel, just our ability to differentiate it from other objects.)

The most challenging aspect of open addressing is to properly trace the series of probes when collisions occur during an insertion or search for an item. To this end, we define a nonpublic utility, _find_slot, that searches for an item with key k in "bucket" j (that is, where j is the index returned by the hash function for key k).

```
1  class ProbeHashMap(HashMapBase):
2    """Hash map implemented with linear probing for collision resolution."""
3    _AVAIL = object( )       # sentinal marks locations of previous deletions
4
5    def _is_available(self, j):
6      """Return True if index j is available in table."""
7      return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL
8
9    def _find_slot(self, j, k):
10     """Search for key k in bucket at index j.
11
12     Return (success, index) tuple, described as follows:
13     If match was found, success is True and index denotes its location.
14     If no match found, success is False and index denotes first available slot.
15     """
16     firstAvail = None
17     while True:
18       if self._is_available(j):
19         if firstAvail is None:
20           firstAvail = j                          # mark this as first avail
21         if self._table[j] is None:
22           return (False, firstAvail)              # search has failed
23       elif k == self._table[j]._key:
24         return (True, j)                          # found a match
25       j = (j + 1) % len(self._table)              # keep looking (cyclically)
```

Code Fragment 10.6: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued in Code Fragment 10.7).

```
26   def _bucket_getitem(self, j, k):
27     found, s = self._find_slot(j, k)
28     if not found:
29       raise KeyError('Key Error: ' + repr(k))          # no match found
30     return self._table[s]._value
31
32   def _bucket_setitem(self, j, k, v):
33     found, s = self._find_slot(j, k)
34     if not found:
35       self._table[s] = self._Item(k,v)                 # insert new item
36       self._n += 1                                     # size has increased
37     else:
38       self._table[s]._value = v                        # overwrite existing
39
40   def _bucket_delitem(self, j, k):
41     found, s = self._find_slot(j, k)
42     if not found:
43       raise KeyError('Key Error: ' + repr(k))          # no match found
44     self._table[s] = ProbeHashMap._AVAIL               # mark as vacated
45
46   def __iter__(self):
47     for j in range(len(self._table)):                  # scan entire table
48       if not self._is_available(j):
49         yield self._table[j]._key
```

Code Fragment 10.7: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued from Code Fragment 10.6).

The three primary map operations each rely on the _find_slot utility. When attempting to retrieve the value associated with a given key, we must continue probing until we find the key, or until we reach a table slot with the None value. We cannot stop the search upon reaching an _AVAIL sentinel, because it represents a location that may have been filled when the desired item was once inserted.

When a key-value pair is being assigned in the map, we must attempt to find an existing item with the given key, so that we might overwrite its value, before adding a new item to the map. Therefore, we must search beyond any occurrences of the _AVAIL sentinel when inserting. However, if no match is found, we prefer to repurpose the first slot marked with _AVAIL, if any, when placing the new element in the table. The _find_slot method enacts this logic, continuing the search until a truly empty slot, but returning the index of the first available slot for an insertion.

When deleting an existing item within _bucket_delitem, we intentionally set the table entry to the _AVAIL sentinel in accordance with our strategy.

10.3 Sorted Maps

The traditional map ADT allows a user to look up the value associated with a given key, but the search for that key is a form known as an ***exact search***.

For example, computer systems often maintain information about events that have occurred (such as financial transactions), organizing such events based upon what are known as ***time stamps***. If we can assume that time stamps are unique for a particular system, then we might organize a map with a time stamp serving as the key, and a record about the event that occurred at that time as the value. A particular time stamp could serve as a reference ID for an event, in which case we can quickly retrieve information about that event from the map. However, the map ADT does not provide any way to get a list of all events ordered by the time at which they occur, or to search for which event occurred closest to a particular time. In fact, the fast performance of hash-based implementations of the map ADT relies on the intentionally scattering of keys that may seem very "near" to each other in the original domain, so that they are more uniformly distributed in a hash table.

In this section, we introduce an extension known as the ***sorted map*** ADT that includes all behaviors of the standard map, plus the following:

M.find_min( ): Return the (key,value) pair with minimum key (or None, if map is empty).

M.find_max( ): Return the (key,value) pair with maximum key (or None, if map is empty).

M.find_lt(k): Return the (key,value) pair with the greatest key that is strictly less than k (or None, if no such item exists).

M.find_le(k): Return the (key,value) pair with the greatest key that is less than or equal to k (or None, if no such item exists).

M.find_gt(k): Return the (key,value) pair with the least key that is strictly greater than k (or None, if no such item exists).

M.find_ge(k): Return the (key,value) pair with the least key that is greater than or equal to k (or None, if no such item).

M.find_range(start, stop): Iterate all (key,value) pairs with start $<=$ key $<$ stop. If start is None, iteration begins with minimum key; if stop is None, iteration concludes with maximum key.

iter(M): Iterate all keys of the map according to their natural order, from smallest to largest.

reversed(M): Iterate all keys of the map in reverse order; in Python, this is implemented with the __reversed__ method.

10.3.1 Sorted Search Tables

Several data structures can efficiently support the sorted map ADT, and we will examine some advanced techniques in Section 10.4 and Chapter 11. In this section, we begin by exploring a simple implementation of a sorted map. We store the map's items in an array-based sequence A so that they are in increasing order of their keys, assuming the keys have a naturally defined order. (See Figure 10.8.) We refer to this implementation of a map as a ***sorted search table***.

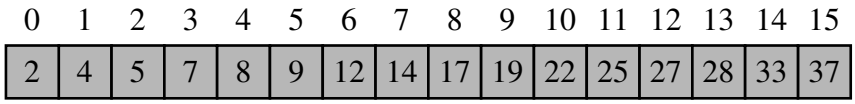


Figure 10.8: Realization of a map by means of a sorted search table. We show only the keys for this map, so as to highlight their ordering.

As was the case with the unsorted table map of Section 10.1.5, the sorted search table has a space requirement that is $O(n)$, assuming we grow and shrink the array to keep its size proportional to the number of items in the map. The primary advantage of this representation, and our reason for insisting that A be array-based, is that it allows us to use the ***binary search*** algorithm for a variety of efficient operations.

Binary Search and Inexact Searches

We originally presented the binary search algorithm in Section 4.1.3, as a means for detecting whether a given target is stored within a sorted sequence. In our original presentation (Code Fragment 4.3 on page 156), a binary_search function returned True of False to designate whether the desired target was found. While such an approach could be used to implement the __contains__ method of the map ADT, we can adapt the binary search algorithm to provide far more useful information when performing forms of inexact search in support of the sorted map ADT.

The important realization is that while performing a binary search, we can determine the index at or near where a target might be found. During a successful search, the standard implementation determines the precise index at which the target is found. During an unsuccessful search, although the target is not found, the algorithm will effectively determine a pair of indices designating elements of the collection that are just less than or just greater than the missing target.

As a motivating example, our original simulation from Figure 4.5 on page 156 shows a successful binary search for a target of 22, using the same data we portray in Figure 10.8. Had we instead been searching for 21, the first four steps of the algorithm would be the same. The subsequent difference is that we would make an additional call with inverted parameters high=9 and low=10, effectively concluding that the missing target lies in the gap between values 19 and 22 in that example.

Implementation

In Code Fragments 10.8 through 10.10, we present a complete implementation of a class, SortedTableMap, that supports the sorted map ADT. The most notable feature of our design is the inclusion of a _find_index utility function. This method using the binary search algorithm, but by convention returns the *index* of the leftmost item in the search interval having key greater than or equal to k. Therefore, if the key is present, it will return the index of the item having that key. (Recall that keys are unique in a map.) When the key is missing, the function returns the index of the item in the search interval that is just beyond where the key would have been located. As a technicality, the method returns index high $+ 1$ to indicate that no items of the interval had a key greater than k.

We rely on this utility method when implementing the traditional map operations and the new sorted map operations. The body of each of the __getitem__, __setitem__, and __delitem__ methods begins with a call to _find_index to determine a candidate index at which a matching key might be found. For __getitem__, we simply check whether that is a valid index containing the target to determine the result. For __setitem__, recall that the goal is to replace the value of an existing item, if one with key k is found, but otherwise to insert a new item into the map. The index returned by _find_index will be the index of the match, if one exists, or otherwise the exact index at which the new item should be inserted. For __delitem__, we again rely on the convenience of _find_index to determine the location of the item to be popped, if any.

Our _find_index utility is equally valuable when implementing the various inexact search methods given in Code Fragment 10.10. For each of the methods find_lt, find_le, find_gt, and find_ge, we begin with a call to _find_index utility, which locates the first index at which there is an element with key $\geq k$, if any. This is precisely what we want for find_ge, if valid, and just beyond the index we want for find_lt. For find_gt and find_le we need some extra case analysis to distinguish whether the indicated index has a key equal to k. For example, if the indicated item has a matching key, our find_gt implementation increments the index before continuing with the process. (We omit the implementation of find_le, for brevity.) In all cases, we must properly handle boundary cases, reporting None when unable to find a key with the desired property.

Our strategy for implementing find_range is to use the _find_index utility to locate the first item with key $\geq$ start (assuming start is not None). With that knowledge, we use a while loop to sequentially report items until reaching one that has a key greater than or equal to the stopping value (or until reaching the end of the table). It is worth noting that the while loop may trivially iterate zero items if the first key that is greater than or equal to start also happens to be greater than or equal to stop. This represents an empty range in the map.

```
1  class SortedTableMap(MapBase):
2    """Map implementation using a sorted table."""
3
4    #---------------------------- nonpublic behaviors ----------------------------
5    def _find_index(self, k, low, high):
6      """Return index of the leftmost item with key greater than or equal to k.
7
8      Return high + 1 if no such item qualifies.
9
10     That is, j will be returned such that:
11       all items of slice table[low:j] have key < k
12       all items of slice table[j:high+1] have key >= k
13     """
14     if high < low:
15       return high + 1                                  # no element qualifies
16     else:
17       mid = (low + high) // 2
18       if k == self._table[mid]._key:
19         return mid                                     # found exact match
20       elif k < self._table[mid]._key:
21         return self._find_index(k, low, mid - 1)       # Note: may return mid
22       else:
23         return self._find_index(k, mid + 1, high)      # answer is right of mid
24
25   #---------------------------- public behaviors ----------------------------
26   def __init__(self):
27     """Create an empty map."""
28     self._table = [ ]
29
30   def __len__(self):
31     """Return number of items in the map."""
32     return len(self._table)
33
34   def __getitem__(self, k):
35     """Return value associated with key k (raise KeyError if not found)."""
36     j = self._find_index(k, 0, len(self._table) - 1)
37     if j == len(self._table) or self._table[j]._key != k:
38       raise KeyError('Key Error: ' + repr(k))
39     return self._table[j]._value
```

Code Fragment 10.8: An implementation of a SortedTableMap class (continued in Code Fragments 10.9 and 10.10).

```
40    def __setitem__(self, k, v):
41      """Assign value v to key k, overwriting existing value if present."""
42      j = self._find_index(k, 0, len(self._table) - 1)
43      if j < len(self._table) and self._table[j]._key == k:
44        self._table[j]._value = v                          # reassign value
45      else:
46        self._table.insert(j, self._Item(k,v))             # adds new item
47
48    def __delitem__(self, k):
49      """Remove item associated with key k (raise KeyError if not found)."""
50      j = self._find_index(k, 0, len(self._table) - 1)
51      if j == len(self._table) or self._table[j]._key != k:
52        raise KeyError('Key Error: ' + repr(k))
53      self._table.pop(j)                                   # delete item
54
55    def __iter__(self):
56      """Generate keys of the map ordered from minimum to maximum."""
57      for item in self._table:
58        yield item._key
59
60    def __reversed__(self):
61      """Generate keys of the map ordered from maximum to minimum."""
62      for item in reversed(self._table):
63        yield item._key
64
65    def find_min(self):
66      """Return (key,value) pair with minimum key (or None if empty)."""
67      if len(self._table) > 0:
68        return (self._table[0]._key, self._table[0]._value)
69      else:
70        return None
71
72    def find_max(self):
73      """Return (key,value) pair with maximum key (or None if empty)."""
74      if len(self._table) > 0:
75        return (self._table[-1]._key, self._table[-1]._value)
76      else:
77        return None
```

Code Fragment 10.9: An implementation of a SortedTableMap class (together with Code Fragments 10.8 and 10.10).

```
78    def find_ge(self, k):
79      """Return (key,value) pair with least key greater than or equal to k."""
80      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
81      if j < len(self._table):
82        return (self._table[j]._key, self._table[j]._value)
83      else:
84        return None
85
86    def find_lt(self, k):
87      """Return (key,value) pair with greatest key strictly less than k."""
88      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
89      if j > 0:
90        return (self._table[j-1]._key, self._table[j-1]._value) # Note use of j-1
91      else:
92        return None
93
94    def find_gt(self, k):
95      """Return (key,value) pair with least key strictly greater than k."""
96      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
97      if j < len(self._table) and self._table[j]._key == k:
98        j += 1                                             # advanced past match
99      if j < len(self._table):
100       return (self._table[j]._key, self._table[j]._value)
101     else:
102       return None
103
104   def find_range(self, start, stop):
105     """Iterate all (key,value) pairs such that start <= key < stop.
106
107     If start is None, iteration begins with minimum key of map.
108     If stop is None, iteration continues through the maximum key of map.
109     """
110     if start is None:
111       j = 0
112     else:
113       j = self._find_index(start, 0, len(self._table)-1)       # find first result
114     while j < len(self._table) and (stop is None or self._table[j]._key < stop):
115       yield (self._table[j]._key, self._table[j]._value)
116       j += 1
```

Code Fragment 10.10: An implementation of a SortedTableMap class (continued from Code Fragments 10.9 and 10.10). We omit the find_le method due to space.

Analysis

We conclude by analyzing the performance of our SortedTableMap implementation. A summary of the running times for all methods of the sorted map ADT (including the traditional map operations) is given in Table 10.3. It should be clear that the __len__, find_min, and find_max methods run in $O(1)$ time, and that iterating the keys of the table in either direction can be peformed in $O(n)$ time.

The analysis for the various forms of search all depend on the fact that a binary search on a table with n entries runs in $O(\log n)$ time. This claim was originally shown as Proposition 4.2 in Section 4.2, and that analysis clearly applies to our _find_index method as well. We therefore claim an $O(\log n)$ worst-case running time for methods __getitem__, find_lt, find_gt, find_le, and find_ge. Each of these makes a single call to _find_index, followed by a constant number of additional steps to determine the appropriate answer based on the index. The analysis of find_range is a bit more interesting. It begins with a binary search to find the first item within the range (if any). After that, it executes a loop that takes $O(1)$ time per iteration to report subsequent values until reaching the end of the range. If there are s items reported in the range, the total running time is $O(s + \log n)$.

In contrast to the efficient search operations, update operations for a sorted table may take considerable time. Although binary search can help identify the index at which an update occurs, both insertions and deletions require, in the worst case, that linearly many existing elements be shifted in order to maintain the sorted order of the table. Specifically, the potential call to _table.insert from within __setitem__ and _table.pop from within __delitem__ lead to $O(n)$ worst-case time. (See the discussion of corresponding operations of the list class in Section 5.4.1.)

In conclusion, sorted tables are primarily used in situations where we expect many searches but relatively few updates.

Operation	Running Time
len(M)	$O(1)$
k in M	$O(\log n)$
M[k] = v	$O(n)$ worst case; $O(\log n)$ if existing k
del M[k]	$O(n)$ worst case
M.find_min( ), M.find_max( )	$O(1)$
M.find_lt(k), M.find_gt(k) M.find_le(k), M.find_ge(k)	$O(\log n)$
M.find_range(start, stop)	$O(s + \log n)$ where s items are reported
iter(M), reversed(M)	$O(n)$

Table 10.3: Performance of a sorted map, as implemented with SortedTableMap. We use n to denote the number of items in the map at the time the operation is performed. The space requirement is $O(n)$.

10.3.2 Two Applications of Sorted Maps

In this section, we explore applications in which there is particular advantage to using a *sorted* map rather than a traditional (unsorted) map. To apply a sorted map, keys must come from a domain that is totally ordered. Furthermore, to take advantage of the inexact or range searches afforded by a sorted map, there should be some reason why nearby keys have relevance to a search.

Flight Databases

There are several Web sites on the Internet that allow users to perform queries on flight databases to find flights between various cities, typically with the intent to buy a ticket. To make a query, a user specifies origin and destination cities, a departure date, and a departure time. To support such queries, we can model the flight database as a map, where keys are Flight objects that contain fields corresponding to these four parameters. That is, a key is a tuple

$$k = (\mathsf{origin}, \mathsf{destination}, \mathsf{date}, \mathsf{time}).$$

Additional information about a flight, such as the flight number, the number of seats still available in first (F) and coach (Y) class, the flight duration, and the fare, can be stored in the value object.

Finding a requested flight is not simply a matter of finding an exact match for a requested query. Although a user typically wants to exactly match the origin and destination cities, he or she may have flexibility for the departure date, and certainly will have some flexibility for the departure time on a specific day. We can handle such a query by ordering our keys lexicographically. Then, an efficient implementation for a sorted map would be a good way to satisfy users' queries. For instance, given a user query key k, we could call find_ge(k) to return the first flight between the desired cities, having a departure date and time matching the desired query or later. Better yet, with well-constructed keys, we could use find_range(k1, k2) to find all flights within a given range of times. For example, if $k1 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{09{:}30})$, and $k2 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{20{:}00})$, a respective call to find_range(k1, k2) might result in the following sequence of key-value pairs:

(ORD, PVD, 05May, 09:53) : (AA 1840, F5, Y15, 02:05, $251),
(ORD, PVD, 05May, 13:29) : (AA 600, F2, Y0, 02:16, $713),
(ORD, PVD, 05May, 17:39) : (AA 416, F3, Y9, 02:09, $365),
(ORD, PVD, 05May, 19:50) : (AA 1828, F9, Y25, 02:13, $186)

Maxima Sets

Life is full of trade-offs. We often have to trade off a desired performance measure against a corresponding cost. Suppose, for the sake of an example, we are interested in maintaining a database rating automobiles by their maximum speeds and their cost. We would like to allow someone with a certain amount of money to query our database to find the fastest car they can possibly afford.

We can model such a trade-off problem as this by using a key-value pair to model the two parameters that we are trading off, which in this case would be the pair $(\mathsf{cost}, \mathsf{speed})$ for each car. Notice that some cars are strictly better than other cars using this measure. For example, a car with cost-speed pair $(20000, 100)$ is strictly better than a car with cost-speed pair $(30000, 90)$. At the same time, there are some cars that are not strictly dominated by another car. For example, a car with cost-speed pair $(20000, 100)$ may be better or worse than a car with cost-speed pair $(30000, 120)$, depending on how much money we have to spend. (See Figure 10.9.)

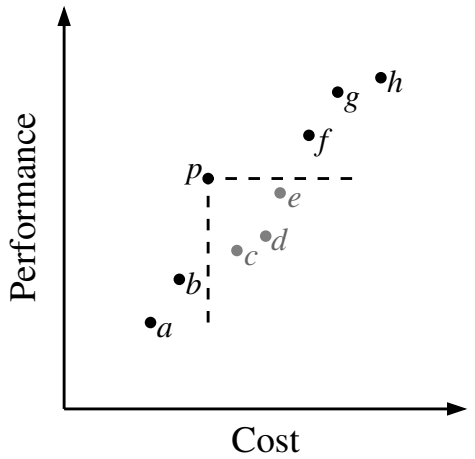


Figure 10.9: Illustrating the cost-performance trade-off with pairs represented by points in the plane. Notice that point p is strictly better than points c, d, and e, but may be better or worse than points a, b, f, g, and h, depending on the price we are willing to pay. Thus, if we were to add p to our set, we could remove the points c, d, and e, but not the others.

Formally, we say a cost-performance pair (a,b) ***dominates*** pair $(c,d) \neq (a,b)$ if $a \leq c$ and $b \geq d$, that is, if the first pair has no greater cost and at least as good performance. A pair (a,b) is called a ***maximum*** pair if it is not dominated by any other pair. We are interested in maintaining the set of maxima of a collection of cost-performance pairs. That is, we would like to add new pairs to this collection (for example, when a new car is introduced), and to query this collection for a given dollar amount, d, to find the fastest car that costs no more than d dollars.

Maintaining a Maxima Set with a Sorted Map

We can store the set of maxima pairs in a sorted map, M, so that the cost is the key field and performance (speed) is the value field. We can then implement operations add(c,p), which adds a new cost-performance pair (c,p), and best(c), which returns the best pair with cost at most c, as shown in Code Fragment 10.11.

```
1  class CostPerformanceDatabase:
2    """Maintain a database of maximal (cost,performance) pairs."""
3
4    def __init__(self):
5      """Create an empty database."""
6      self._M = SortedTableMap( )              # or a more efficient sorted map
7
8    def best(self, c):
9      """Return (cost,performance) pair with largest cost not exceeding c.
10
11     Return None if there is no such pair.
12     """
13     return self._M.find_le(c)
14
15   def add(self, c, p):
16     """Add new entry with cost c and performance p."""
17     # determine if (c,p) is dominated by an existing pair
18     other = self._M.find_le(c)                # other is at least as cheap as c
19     if other is not None and other[1] >= p: # if its performance is as good,
20         return                                # (c,p) is dominated, so ignore
21     self._M[c] = p                            # else, add (c,p) to database
22     # and now remove any pairs that are dominated by (c,p)
23     other = self._M.find_gt(c)                # other more expensive than c
24     while other is not None and other[1] <= p:
25       del self._M[other[0]]
26       other = self._M.find_gt(c)
```

Code Fragment 10.11: An implementation of a class maintaining a set of maxima cost-performance pairs using a sorted map.

Unfortunately, if we implement M using the SortedTableMap, the add behavior has $O(n)$ worst-case running time. If, on the other hand, we implement M using a skip list, which we next describe, we can perform best(c) queries in $O(\log n)$ expected time and add(c,p) updates in $O((1+r)\log n)$ expected time, where r is the number of points removed.

10.4 Skip Lists

An interesting data structure for realizing the sorted map ADT is the ***skip list***. In Section 10.3.1, we saw that a sorted array will allow $O(\log n)$-time searches via the binary search algorithm. Unfortunately, update operations on a sorted array have $O(n)$ worst-case running time because of the need to shift elements. In Chapter 7 we demonstrated that linked lists support very efficient update operations, as long as the position within the list is identified. Unfortunately, we cannot perform fast searches on a standard linked list; for example, the binary search algorithm requires an efficient means for direct accessing an element of a sequence by index.

Skip lists provide a clever compromise to efficiently support search and update operations. A ***skip list*** S for a map M consists of a series of lists $\{S_0, S_1, \ldots, S_h\}$. Each list S_i stores a subset of the items of M sorted by increasing keys, plus items with two sentinel keys denoted $-\infty$ and $+\infty$, where $-\infty$ is smaller than every possible key that can be inserted in M and $+\infty$ is larger than every possible key that can be inserted in M. In addition, the lists in S satisfy the following:

- List S_0 contains every item of the map M (plus sentinels $-\infty$ and $+\infty$).
- For $i = 1, \ldots, h-1$, list S_i contains (in addition to $-\infty$ and $+\infty$) a randomly generated subset of the items in list S_{i-1}.
- List S_h contains only $-\infty$ and $+\infty$.

An example of a skip list is shown in Figure 10.10. It is customary to visualize a skip list S with list S_0 at the bottom and lists $S_1, \ldots, S_h$ above it. Also, we refer to h as the ***height*** of skip list S.

Intuitively, the lists are set up so that S_{i+1} contains more or less alternate items of S_i. As we shall see in the details of the insertion method, the items in S_{i+1} are chosen at random from the items in S_i by picking each item from S_i to also be in S_{i+1} with probability $1/2$. That is, in essence, we "flip a coin" for each item in S_i

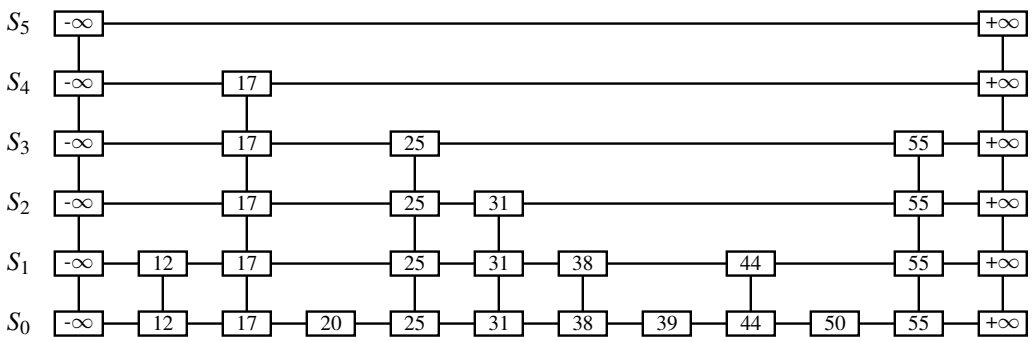


Figure 10.10: Example of a skip list storing 10 items. For simplicity, we show only the items' keys, not their associated values.

and place that item in S_{i+1} if the coin comes up "heads." Thus, we expect S_1 to have about $n/2$ items, S_2 to have about $n/4$ items, and, in general, S_i to have about $n/2^i$ items. In other words, we expect the height h of S to be about $\log n$. The halving of the number of items from one list to the next is not enforced as an explicit property of skip lists, however. Instead, randomization is used.

Functions that generate numbers that can be viewed as random numbers are built into most modern computers, because they are used extensively in computer games, cryptography, and computer simulations, Some functions, called ***pseudo-random number generators***, generate random-like numbers, starting with an initial ***seed***. (See discusion of random module in Section 1.11.1.) Other methods use hardware devices to extract "true" random numbers from nature. In any case, we will assume that our computer has access to numbers that are sufficiently random for our analysis.

The main advantage of using ***randomization*** in data structure and algorithm design is that the structures and functions that result are usually simple and efficient. The skip list has the same logarithmic time bounds for searching as is achieved by the binary search algorithm, yet it extends that performance to update methods when inserting or deleting items. Nevertheless, the bounds are ***expected*** for the skip list, while binary search has a ***worst-case*** bound with a sorted table.

A skip list makes random choices in arranging its structure in such a way that search and update times are $O(\log n)$ ***on average***, where n is the number of items in the map. Interestingly, the notion of average time complexity used here does not depend on the probability distribution of the keys in the input. Instead, it depends on the use of a random-number generator in the implementation of the insertions to help decide where to place the new item. The running time is averaged over all possible outcomes of the random numbers used when inserting entries.

Using the position abstraction used for lists and trees, we view a skip list as a two-dimensional collection of positions arranged horizontally into ***levels*** and vertically into ***towers***. Each level is a list S_i and each tower contains positions storing the same item across consecutive lists. The positions in a skip list can be traversed using the following operations:

next(p): Return the position following p on the same level.

prev(p): Return the position preceding p on the same level.

below(p): Return the position below p in the same tower.

above(p): Return the position above p in the same tower.

We conventionally assume that the above operations return None if the position requested does not exist. Without going into the details, we note that we can easily implement a skip list by means of a linked structure such that the individual traversal methods each take $O(1)$ time, given a skip-list position p. Such a linked structure is essentially a collection of h doubly linked lists aligned at towers, which are also doubly linked lists.

10.4.1 Search and Update Operations in a Skip List

The skip-list structure affords simple map search and update algorithms. In fact, all of the skip-list search and update algorithms are based on an elegant SkipSearch method that takes a key k and finds the position p of the item in list S_0 that has the largest key less than or equal to k (which is possibly $-\infty$).

Searching in a Skip List

Suppose we are given a search key k. We begin the SkipSearch method by setting a position variable p to the topmost, left position in the skip list S, called the ***start position*** of S. That is, the start position is the position of S_h storing the special entry with key $-\infty$. We then perform the following steps (see Figure 10.11), where $\mathsf{key}(p)$ denotes the key of the item at position p:

1. If $S.\mathsf{below}(p)$ is None, then the search terminates—we are ***at the bottom*** and have located the item in S with the largest key less than or equal to the search key k. Otherwise, we ***drop down*** to the next lower level in the present tower by setting $p = S.\mathsf{below}(p)$.
2. Starting at position p, we move p forward until it is at the rightmost position on the present level such that $\mathsf{key}(p) \leq k$. We call this the ***scan forward*** step. Note that such a position always exists, since each level contains the keys $+\infty$ and $-\infty$. It may be that p remains where it started after we perform such a forward scan for this level.
3. Return to step 1.

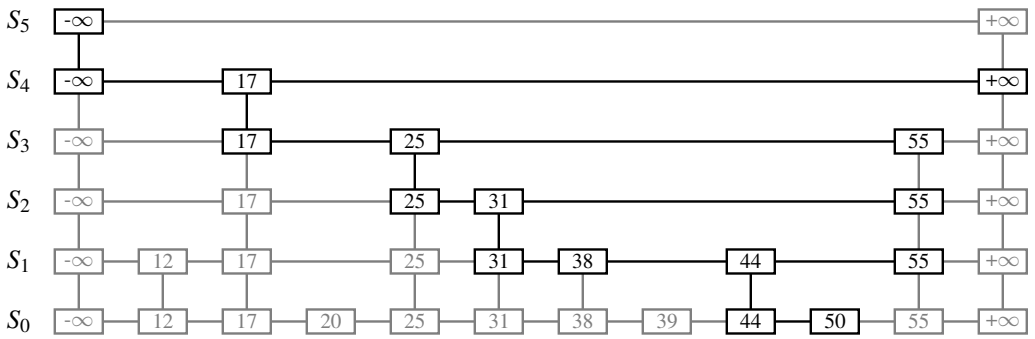


Figure 10.11: Example of a search in a skip list. The positions examined when searching for key 50 are highlighted.

We give a pseudo-code description of the skip-list search algorithm, SkipSearch, in Code Fragment 10.12. Given this method, the map operation $M[k]$ is performed by computing $p = \mathsf{SkipSearch}(k)$ and testing whether or not $\mathsf{key}(p) = k$. If these two keys are equal, we return the associated value; otherwise, we raise a KeyError.

Algorithm SkipSearch(k):
Input: A search key k
Output: Position p in the bottom list S_0 with the largest key such that key(p) $\leq$ k

```
p = start                                   {begin at start position}
while below(p) ≠ None do
  p = below(p)                              {drop down}
  while k ≥ key(next(p)) do
    p = next(p)                             {scan forward}
return p.
```

Code Fragment 10.12: Algorithm to search a skip list S for key k.

As it turns out, the expected running time of algorithm SkipSearch on a skip list with n entries is $O(\log n)$. We postpone the justification of this fact, however, until after we discuss the implementation of the update methods for skip lists. Navigation starting at the position identified by SkipSearch(k) can be easily used to provide the additional forms of searches in the sorted map ADT (e.g., find_gt, find_range).

Insertion in a Skip List

The execution of the map operation M[k] = v begins with a call to SkipSearch(k). This gives us the position p of the bottom-level item with the largest key less than or equal to k (note that p may hold the special item with key $-\infty$). If key(p) $= k$, the associated value is overwritten with v. Otherwise, we need to create a new tower for item (k,v). We insert (k,v) immediately after position p within S_0. After inserting the new item at the bottom level, we use randomization to decide the height of the tower for the new item. We "flip" a coin, and if the flip comes up tails, then we stop here. Else (the flip comes up heads), we backtrack to the previous (next higher) level and insert (k,v) in this level at the appropriate position. We again flip a coin; if it comes up heads, we go to the next higher level and repeat. Thus, we continue to insert the new item (k,v) in lists until we finally get a flip that comes up tails. We link together all the references to the new item (k,v) created in this process to create its tower. A coin flip can be simulated with Python's built-in pseudo-random number generator from the random module by calling randrange(2), which returns 0 or 1, each with probability $1/2$.

We give the insertion algorithm for a skip list S in Code Fragment 10.13 and we illustrate it in Figure 10.12. The algorithm uses an insertAfterAbove($p,q,(k,v)$) method that inserts a position storing the item (k,v) after position p (on the same level as p) and above position q, returning the new position r (and setting internal references so that next, prev, above, and below methods will work correctly for p, q, and r). The expected running time of the insertion algorithm on a skip list with n entries is $O(\log n)$, which we show in Section 10.4.2.

Algorithm SkipInsert(k,v):
Input: Key k and value v
Output: Topmost position of the item inserted in the skip list

```
p = SkipSearch(k)
q = None                          {q will represent top node in new item's tower}
i = −1
repeat
  i = i + 1
  if i ≥ h then
    h = h + 1                                  {add a new level to the skip list}
    t = next(s)
    s = insertAfterAbove(None, s, (−∞, None))      {grow leftmost tower}
    insertAfterAbove(s, t, (+∞, None))             {grow rightmost tower}
  while above(p) is None do
    p = prev(p)                                            {scan backward}
  p = above(p)                                        {jump up to higher level}
  q = insertAfterAbove(p, q, (k, v))    {increase height of new item's tower}
until coinFlip() == tails
n = n + 1
return q
```

Code Fragment 10.13: Insertion in a skip list. Method coinFlip() returns "heads" or "tails", each with probability $1/2$. Instance variables n, h, and s hold the number of entries, the height, and the start node of the skip list.

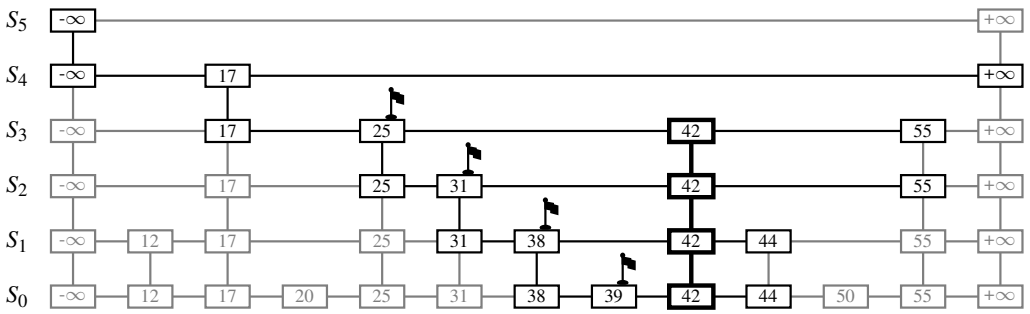


Figure 10.12: Insertion of an entry with key 42 into the skip list of Figure 10.10. We assume that the random "coin flips" for the new entry came up heads three times in a row, followed by tails. The positions visited are highlighted. The positions inserted to hold the new entry are drawn with thick lines, and the positions preceding them are flagged.

Removal in a Skip List

Like the search and insertion algorithms, the removal algorithm for a skip list is quite simple. In fact, it is even easier than the insertion algorithm. That is, to perform the map operation del M[k] we begin by executing method SkipSearch(k). If the position p stores an entry with key different from k, we raise a KeyError. Otherwise, we remove p and all the positions above p, which are easily accessed by using above operations to climb up the tower of this entry in S starting at position p. While removing levels of the tower, we reestablish links between the horizontal neighbors of each removed position. The removal algorithm is illustrated in Figure 10.13 and a detailed description of it is left as an exercise (R-10.24). As we show in the next subsection, deletion operation in a skip list with n entries has $O(\log n)$ expected running time.

Before we give this analysis, however, there are some minor improvements to the skip-list data structure we would like to discuss. First, we do not actually need to store references to values at the levels of the skip list above the bottom level, because all that is needed at these levels are references to keys. In fact, we can more efficiently represent a tower as a single object, storing the key-value pair, and maintaining j previous references and j next references if the tower reaches level S_j. Second, for the horizontal axes, it is possible to keep the list singly linked, storing only the next references. We can perform insertions and removals in strictly a top-down, scan-forward fashion. We explore the details of this optimization in Exercise C-10.44. Neither of these optimizations improve the asymptotic performance of skip lists by more than a constant factor, but these improvements can, nevertheless, be meaningful in practice. In fact, experimental evidence suggests that optimized skip lists are faster in practice than AVL trees and other balanced search trees, which are discussed in Chapter 11.

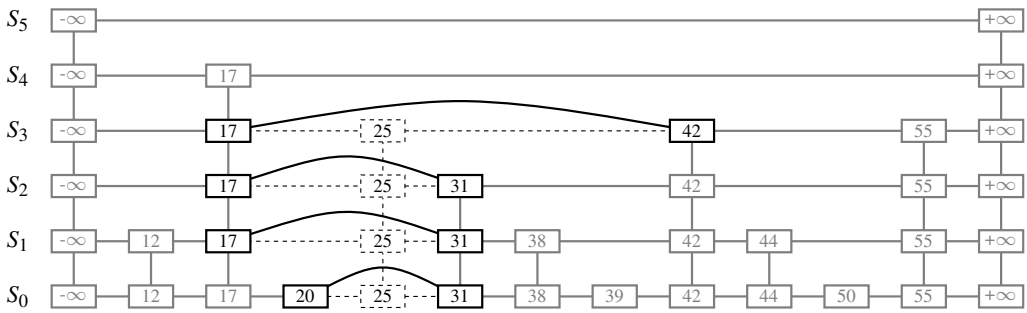


Figure 10.13: Removal of the entry with key 25 from the skip list of Figure 10.12. The positions visited after the search for the position of S_0 holding the entry are highlighted. The positions removed are drawn with dashed lines.

Maintaining the Topmost Level

A skip list S must maintain a reference to the start position (the topmost, left position in S) as an instance variable, and must have a policy for any insertion that wishes to continue inserting a new entry past the top level of S. There are two possible courses of action we can take, both of which have their merits.

One possibility is to restrict the top level, h, to be kept at some fixed value that is a function of n, the number of entries currently in the map (from the analysis we will see that $h = \max\{10, 2\lceil \log n \rceil\}$ is a reasonable choice, and picking $h = 3\lceil \log n \rceil$ is even safer). Implementing this choice means that we must modify the insertion algorithm to stop inserting a new position once we reach the topmost level (unless $\lceil \log n \rceil < \lceil \log(n+1) \rceil$, in which case we can now go at least one more level, since the bound on the height is increasing).

The other possibility is to let an insertion continue inserting a new position as long as heads keeps getting returned from the random number generator. This is the approach taken by algorithm SkipInsert of Code Fragment 10.13. As we show in the analysis of skip lists, the probability that an insertion will go to a level that is more than $O(\log n)$ is very low, so this design choice should also work.

Either choice will still result in the expected $O(\log n)$ time to perform search, insertion, and removal, however, which we show in the next section.

10.4.2 Probabilistic Analysis of Skip Lists ⋆

As we have shown above, skip lists provide a simple implementation of a sorted map. In terms of worst-case performance, however, skip lists are not a superior data structure. In fact, if we do not officially prevent an insertion from continuing significantly past the current highest level, then the insertion algorithm can go into what is almost an infinite loop (it is not actually an infinite loop, however, since the probability of having a fair coin repeatedly come up heads forever is 0). Moreover, we cannot infinitely add positions to a list without eventually running out of memory. In any case, if we terminate position insertion at the highest level h, then the ***worst-case*** running time for performing the __getitem__, __setitem__, and __delitem__ map operations in a skip list S with n entries and height h is $O(n+h)$. This worst-case performance occurs when the tower of every entry reaches level $h-1$, where h is the height of S. However, this event has very low probability. Judging from this worst case, we might conclude that the skip-list structure is strictly inferior to the other map implementations discussed earlier in this chapter. But this would not be a fair analysis, for this worst-case behavior is a gross overestimate.

Bounding the Height of a Skip List

Because the insertion step involves randomization, a more accurate analysis of skip lists involves a bit of probability. At first, this might seem like a major undertaking, for a complete and thorough probabilistic analysis could require deep mathematics (and, indeed, there are several such deep analyses that have appeared in data structures research literature). Fortunately, such an analysis is not necessary to understand the expected asymptotic behavior of skip lists. The informal and intuitive probabilistic analysis we give below uses only basic concepts of probability theory.

Let us begin by determining the expected value of the height h of a skip list S with n entries (assuming that we do not terminate insertions early). The probability that a given entry has a tower of height $i \geq 1$ is equal to the probability of getting i consecutive heads when flipping a coin, that is, this probability is $1/2^i$. Hence, the probability P_i that level i has at least one position is at most

$$P_i \leq \frac{n}{2^i},$$

for the probability that any one of n different events occurs is at most the sum of the probabilities that each occurs.

The probability that the height h of S is larger than i is equal to the probability that level i has at least one position, that is, it is no more than P_i. This means that h is larger than, say, $3\log n$ with probability at most

$$\begin{aligned} P_{3\log n} &\leq \frac{n}{2^{3\log n}} \\ &= \frac{n}{n^3} = \frac{1}{n^2}. \end{aligned}$$

For example, if $n = 1000$, this probability is a one-in-a-million long shot. More generally, given a constant $c > 1$, h is larger than $c\log n$ with probability at most $1/n^{c-1}$. That is, the probability that h is smaller than $c\log n$ is at least $1 - 1/n^{c-1}$. Thus, with high probability, the height h of S is $O(\log n)$.

Analyzing Search Time in a Skip List

Next, consider the running time of a search in skip list S, and recall that such a search involves two nested **while** loops. The inner loop performs a scan forward on a level of S as long as the next key is no greater than the search key k, and the outer loop drops down to the next level and repeats the scan forward iteration. Since the height h of S is $O(\log n)$ with high probability, the number of drop-down steps is $O(\log n)$ with high probability.

So we have yet to bound the number of scan-forward steps we make. Let n_i be the number of keys examined while scanning forward at level i. Observe that, after the key at the starting position, each additional key examined in a scan-forward at level i cannot also belong to level $i+1$. If any of these keys were on the previous level, we would have encountered them in the previous scan-forward step. Thus, the probability that any key is counted in n_i is $1/2$. Therefore, the expected value of n_i is exactly equal to the expected number of times we must flip a fair coin before it comes up heads. This expected value is 2. Hence, the expected amount of time spent scanning forward at any level i is $O(1)$. Since S has $O(\log n)$ levels with high probability, a search in S takes expected time $O(\log n)$. By a similar analysis, we can show that the expected running time of an insertion or a removal is $O(\log n)$.

Space Usage in a Skip List

Finally, let us turn to the space requirement of a skip list S with n entries. As we observed above, the expected number of positions at level i is $n/2^i$, which means that the expected total number of positions in S is

$$\sum_{i=0}^{h} \frac{n}{2^i} = n \sum_{i=0}^{h} \frac{1}{2^i}.$$

Using Proposition 3.5 on geometric summations, we have

$$\sum_{i=0}^{h} \frac{1}{2^i} = \frac{\left(\frac{1}{2}\right)^{h+1} - 1}{\frac{1}{2} - 1} = 2 \cdot \left(1 - \frac{1}{2^{h+1}}\right) < 2 \quad \text{for all } h \geq 0.$$

Hence, the expected space requirement of S is $O(n)$.

Table 10.4 summarizes the performance of a sorted map realized by a skip list.

Operation	Running Time
len(M)	$O(1)$
k **in** M	$O(\log n)$ expected
M[k] = v	$O(\log n)$ expected
del M[k]	$O(\log n)$ expected
M.find_min( ), M.find_max( )	$O(1)$
M.find_lt(k), M.find_gt(k) M.find_le(k), M.find_ge(k)	$O(\log n)$ expected
M.find_range(start, stop)	$O(s + \log n)$ expected, with s items reported
iter(M), reversed(M)	$O(n)$

Table 10.4: Performance of a sorted map implemented with a skip list. We use n to denote the number of entries in the dictionary at the time the operation is performed. The expected space requirement is $O(n)$.

10.5 Sets, Multisets, and Multimaps

We conclude this chapter by examining several additional abstractions that are closely related to the map ADT, and that can be implemented using data structures similar to those for a map.

- A ***set*** is an unordered collection of elements, without duplicates, that typically supports efficient membership tests. In essence, elements of a set are like keys of a map, but without any auxiliary values.
- A ***multiset*** (also known as a ***bag***) is a set-like container that allows duplicates.
- A ***multimap*** is similar to a traditional map, in that it associates values with keys; however, in a multimap the same key can be mapped to multiple values. For example, the index of this book maps a given term to one or more locations at which the term occurs elsewhere in the book.

10.5.1 The Set ADT

Python provides support for representing the mathematical notion of a set through the built-in classes **frozenset** and **set**, as originally discussed in Chapter 1, with frozenset being an immutable form. Both of those classes are implemented using hash tables in Python.

Python's collections module defines abstract base classes that essentially mirror these built-in classes. Although the choice of names is counterintuitive, the abstract base class collections.Set matches the concrete frozenset class, while the abstract base class collections.MutableSet is akin to the concrete set class.

In our own discussion, we equate the "set ADT" with the behavior of the built-in set class (and thus, the collections.MutableSet base class). We begin by listing what we consider to be the five most fundamental behaviors for a set S:

S.add(e): Add element e to the set. This has no effect if the set already contains e.

S.discard(e): Remove element e from the set, if present. This has no effect if the set does not contain e.

e in S: Return True if the set contains element e. In Python, this is implemented with the special __contains__ method.

len(S): Return the number of elements in set S. In Python, this is implemented with the special method __len__.

iter(S): Generate an iteration of all elements of the set. In Python, this is implemented with the special method __iter__.

In the next section, we will see that the above five methods suffice for deriving all other behaviors of a set. Those remaining behaviors can be naturally grouped as follows. We begin by describing the following additional operations for removing one or more elements from a set:

S.remove(e): Remove element e from the set. If the set does not contain e, raise a KeyError.

S.pop( ): Remove and return an arbitrary element from the set. If the set is empty, raise a KeyError.

S.clear( ): Remove all elements from the set.

The next group of behaviors perform Boolean comparisons between two sets.

S == T: Return True if sets S and T have identical contents.

S != T: Return True if sets S and T are not equivalent.

S <= T: Return True if set S is a subset of set T.

S < T: Return True if set S is a *proper* subset of set T.

S >= T: Return True if set S is a superset of set T.

S > T: Return True if set S is a *proper* superset of set T.

S.isdisjoint(T): Return True if sets S and T have no common elements.

Finally, there exists a variety of behaviors that either update an existing set, or compute a new set instance, based on classical set theory operations.

S | T: Return a new set representing the union of sets S and T.

S |= T: Update set S to be the union of S and set T.

S & T: Return a new set representing the intersection of sets S and T.

S &= T: Update set S to be the intersection of S and set T.

S ^ T: Return a new set representing the symmetric difference of sets S and T, that is, a set of elements that are in precisely one of S or T.

S ^= T: Update set S to become the symmetric difference of itself and set T.

S − T: Return a new set containing elements in S but not T.

S −= T: Update set S to remove all common elements with set T.

10.5.2 Python's MutableSet Abstract Base Class

To aid in the creation of user-defined set classes, Python's collections module provides a MutableSet abstract base class (just as it provides the MutableMapping abstract base class discussed in Section 10.1.3). The MutableSet base class provides concrete implementations for all methods described in Section 10.5.1, except for five core behaviors (add, discard, __contains__, __len__, and __iter__) that must be implemented by any concrete subclass. This design is an example of what is known as the ***template method pattern***, as the concrete methods of the MutableSet class rely on the presumed abstract methods that will subsequently be provided by a subclass.

For the purpose of illustration, we examine algorithms for implementing several of the derived methods of the MutableSet base class. For example, to determine if one set is a proper subset of another, we must verify two conditions: a proper subset must have size strictly smaller than that of its superset, and each element of a subset must be contained in the superset. An implementation of the corresponding __lt__ method based on this logic is given in Code Fragment 10.14.

```
def __lt__(self, other):       # supports syntax S < T
  """Return true if this set is a proper subset of other."""
  if len(self) >= len(other):
    return False               # proper subset must have strictly smaller size
  for e in self:
    if e not in other:
      return False             # not a subset since element missing from other
  return True                  # success; all conditions are met
```

Code Fragment 10.14: A possible implementation of the MutableSet.__lt__ method, which tests if one set is a proper subset of another.

As another example, we consider the computation of the union of two sets. The set ADT includes two forms for computing a union. The syntax S | T should produce a new set that has contents equal to the union of existing sets S and T. This operation is implemented through the special method __or__ in Python. Another syntax, S |= T is used to *update* existing set S to become the union of itself and set T. Therefore, all elements of T that are not already contained in S should be added to S. We note that this "in-place" operation may be implemented more efficiently than if we were to rely on the first form, using the syntax S = S | T, in which identifier S is reassigned to a new set instance that represents the union. For convenience, Python's built-in set class supports named version of these behaviors, with S.union(T) equivalent to S | T, and S.update(T) equivalent to S |= T (yet, those named versions are not formally provided by the MutableSet abstract base class).

```
def __or__(self, other):                    # supports syntax S | T
  """Return a new set that is the union of two existing sets."""
  result = type(self)( )                    # create new instance of concrete class
  for e in self:
    result.add(e)
  for e in other:
    result.add(e)
  return result
```

Code Fragment 10.15: An implementation of the MutableSet.__or__ method, which computes the union of two existing sets.

An implementation of the behavior that computes a new set as a union of two others is given in the form of the __or__ special method, in Code Fragment 10.15. An important subtlety in this implementation is the instantiation of the resulting set. Since the MutableSet class is designed as an abstract base class, instances must belong to a concrete subclass. When computing the union of two such concrete instances, the result should presumably be an instance of the same class as the operands. The function type(self) returns a reference to the actual class of the instance identified as self, and the subsequent parentheses in expression type(self)( ) call the default constructor for that class.

In terms of efficiency, we analyze such set operations while letting n denote the size of S and m denote the size of set T for an operation such as S | T. If the concrete sets are implemented with hashing, the expected running time of the implementation in Code Fragment 10.15 is $O(m+n)$, because it loops over both sets, performing constant-time operations in the form of a containment check and a possible insertion into the result.

Our implementation of the in-place version of a union is given in Code Fragment 10.16, in the form of the __ior__ special method that supports syntax S |= T. Notice that in this case, we do not create a new set instance, instead we modify and return the existing set, after updating its contents to reflect the union operation. The in-place version of the union has expected running time $O(m)$ where m is the size of the second set, because we only have to loop through that second set.

```
def __ior__(self, other):                   # supports syntax S |= T
  """Modify this set to be the union of itself an another set."""
  for e in other:
    self.add(e)
  return self                               # technical requirement of in-place operator
```

Code Fragment 10.16: An implementation of the MutableSet.__ior__ method, which performs an in-place union of one set with another.

10.5.3 Implementing Sets, Multisets, and Multimaps

Sets

Although sets and maps have very different public interfaces, they are really quite similar. A set is simply a map in which keys do not have associated values. Any data structure used to implement a map can be modified to implement the set ADT with similar performance guarantees. We could trivially adapt any map class by storing set elements as keys, and using None as an irrelevant value, but such an implementation is unnecessarily wasteful. An efficient set implementation should abandon the _Item composite that we use in our MapBase class and instead store set elements directly in a data structure.

Multisets

The same element may occur several times in a multiset. All of the data structures we have seen can be reimplemented to allow for duplicates to appear as separate elements. However, another way to implement a multiset is by using a *map* in which the map key is a (distinct) element of the multiset, and the associated value is a count of the number of occurrences of that element within the multiset. In fact, that is essentially what we did in Section 10.1.2 when computing the frequency of words within a document.

Python's standard collections module includes a definition for a class named Counter that is in essence a multiset. Formally, the Counter class is a subclass of dict, with the expectation that values are integers, and with additional functionality like a most_common(n) method that returns a list of the n most common elements. The standard __iter__ reports each element only once (since those are formally the keys of the dictionary). There is another method named elements( ) that iterates through the multiset with each element being repeated according to its count.

Multimaps

Although there is no multimap in Python's standard libraries, a common implementation approach is to use a standard map in which the value associated with a key is itself a container class storing any number of associated values. We give an example of such a MultiMap class in Code Fragment 10.17. Our implementation uses the standard dict class as the map, and a list of values as a composite value in the dictionary. We have designed the class so that a different map implementation can easily be substituted by overriding the class-level _MapType attribute at line 3.

```
class MultiMap:
  """A multimap class built upon use of an underlying map for storage."""
  _MapType = dict                    # Map type; can be redefined by subclass

  def __init__(self):
    """Create a new empty multimap instance."""
    self._map = self._MapType( )          # create map instance for storage
    self._n = 0

  def __iter__(self):
    """Iterate through all (k,v) pairs in multimap."""
    for k,secondary in self._map.items( ):
      for v in secondary:
        yield (k,v)

  def add(self, k, v):
    """Add pair (k,v) to multimap."""
    container = self._map.setdefault(k, [ ])      # create empty list, if needed
    container.append(v)
    self._n += 1

  def pop(self, k):
    """Remove and return arbitrary (k,v) with key k (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    v = secondary.pop( )
    if len(secondary) == 0:
      del self._map[k]                            # no pairs left
    self._n -= 1
    return (k, v)

  def find(self, k):
    """Return arbitrary (k,v) pair with given key (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    return (k, secondary[0])

  def find_all(self, k):
    """Generate iteration of all (k,v) pairs with given key."""
    secondary = self._map.get(k, [ ])             # empty list, by default
    for v in secondary:
      yield (k,v)
```

Code Fragment 10.17: An implementation of a MultiMap using a dict for storage. The __len__ method, which returns self._n, is omitted from this listing.

10.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-10.1 Give a concrete implementation of the pop method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class.

R-10.2 Give a concrete implementation of the items( ) method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class. What would its running time be if directly applied to the UnsortedTableMap subclass?

R-10.3 Give a concrete implementation of the items( ) method directly within the UnsortedTableMap class, ensuring that the entire iteration runs in $O(n)$ time.

R-10.4 What is the worst-case running time for inserting n key-value pairs into an initially empty map M that is implemented with the UnsortedTableMap class?

R-10.5 Reimplement the UnsortedTableMap class from Section 10.1.5, using the PositionalList class from Section 7.4 rather than a Python list.

R-10.6 Which of the hash table collision-handling schemes could tolerate a load factor above 1 and which could not?

R-10.7 Our Position classes for lists and trees support the __eq__ method so that two distinct position instances are considered equivalent if they refer to the same underlying node in a structure. For positions to be allowed as keys in a hash table, there must be a definition for the __hash__ method that is consistent with this notion of equivalence. Provide such a __hash__ method.

R-10.8 What would be a good hash code for a vehicle identification number that is a string of numbers and letters of the form "9X9XX99X9XX999999," where a "9" represents a digit and an "X" represents a letter?

R-10.9 Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i+5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.

R-10.10 What is the result of the previous exercise, assuming collisions are handled by linear probing?

R-10.11 Show the result of Exercise R-10.9, assuming collisions are handled by quadratic probing, up to the point where the method fails.

R-10.12 What is the result of Exercise R-10.9 when collisions are handled by double hashing using the secondary hash function $h'(k) = 7 - (k \bmod 7)$?

R-10.13 What is the worst-case time for putting n entries in an initially empty hash table, with collisions resolved by chaining? What is the best case?

R-10.14 Show the result of rehashing the hash table shown in Figure 10.6 into a table of size 19 using the new hash function $h(k) = 3k \bmod 17$.

R-10.15 Our HashMapBase class maintains a load factor $\lambda \leq 0.5$. Reimplement that class to allow the user to specify the maximum load, and adjust the concrete subclasses accordingly.

R-10.16 Give a pseudo-code description of an insertion into a hash table that uses quadratic probing to resolve collisions, assuming we also use the trick of replacing deleted entries with a special "deactivated entry" object.

R-10.17 Modify our ProbeHashMap to use quadratic probing.

R-10.18 Explain why a hash table is not suited to implement a sorted map.

R-10.19 Describe how a sorted list implemented as a doubly linked list could be used to implement the sorted map ADT.

R-10.20 What is the worst-case asymptotic running time for performing n deletions from a SortedTableMap instance that initially contains $2n$ entries?

R-10.21 Consider the following variant of the _find_index method from Code Fragment 10.8, in the context of the SortedTableMap class:

```
def _find_index(self, k, low, high):
  if high < low:
    return high + 1
  else:
    mid = (low + high) // 2
    if self._table[mid]._key < k:
      return self._find_index(k, mid + 1, high)
    else:
      return self._find_index(k, low, mid - 1)
```

Does this always produce the same result as the original version? Justify your answer.

R-10.22 What is the expected running time of the methods for maintaining a maxima set if we insert n pairs such that each pair has lower cost and performance than one before it? What is contained in the sorted map at the end of this series of operations? What if each pair had a lower cost and higher performance than the one before it?

R-10.23 Draw an example skip list S that results from performing the following series of operations on the skip list shown in Figure 10.13: del S[38], S[48] = 'x', S[24] = 'y', del S[55]. Record your coin flips, as well.

R-10.24 Give a pseudo-code description of the __delitem__ map operation when using a skip list.

R-10.25 Give a concrete implementation of the pop method, in the context of a MutableSet abstract base class, that relies only on the five core set behaviors described in Section 10.5.2.

R-10.26 Give a concrete implementation of the isdisjoint method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class. Your algorithm should run in $O(\min(n,m))$ where n and m denote the respective cardinalities of the two sets.

R-10.27 What abstraction would you use to manage a database of friends' birthdays in order to support efficient queries such as "find all friends whose birthday is today" and "find the friend who will be the next to celebrate a birthday"?

Creativity

C-10.28 On page 406 of Section 10.1.3, we give an implementation of the method setdefault as it might appear in the MutableMapping abstract base class. While that method accomplishes the goal in a general fashion, its efficiency is less than ideal. In particular, when the key is new, there will be a failed search due to the initial use of __getitem__, and then a subsequent insertion via __setitem__. For a concrete implementation, such as the UnsortedTableMap, this is twice the work because a complete scan of the table will take place during the failed __getitem__, and then another complete scan of the table takes place due to the implementation of __setitem__. A better solution is for the UnsortedTableMap class to override setdefault to provide a direct solution that performs a single search. Give such an implementation of UnsortedTableMap.setdefault.

C-10.29 Repeat Exercise C-10.28 for the ProbeHashMap class.

C-10.30 Repeat Exercise C-10.28 for the ChainHashMap class.

C-10.31 For an ideal compression function, the capacity of the bucket array for a hash table should be a prime number. Therefore, we consider the problem of locating a prime number in a range $[M, 2M]$. Implement a method for finding such a prime by using the ***sieve algorithm***. In this algorithm, we allocate a $2M$ cell Boolean array A, such that cell i is associated with the integer i. We then initialize the array cells to all be "true" and we "mark off" all the cells that are multiples of 2, 3, 5, 7, and so on. This process can stop after it reaches a number larger than $\sqrt{2M}$. (Hint: Consider a bootstrapping method for finding the primes up to $\sqrt{2M}$.)

C-10.32 Perform experiments on our ChainHashMap and ProbeHashMap classes to measure its efficiency using random key sets and varying limits on the load factor (see Exercise R-10.15).

C-10.33 Our implementation of separate chaining in ChainHashMap conserves memory by representing empty buckets in the table as None, rather than as empty instances of a secondary structure. Because many of these buckets will hold a single item, a better optimization is to have those slots of the table directly reference the _Item instance, and to reserve use of secondary containers for buckets that have two or more items. Modify our implementation to provide this additional optimization.

C-10.34 Computing a hash code can be expensive, especially for lengthy keys. In our hash table implementations, we compute the hash code when first inserting an item, and recompute each item's hash code each time we resize our table. Python's dict class makes an interesting trade-off. The hash code is computed once, when an item is inserted, and the hash code is stored as an extra field of the item composite, so that it need not be recomputed. Reimplement our HashTableBase class to use such an approach.

C-10.35 Describe how to perform a removal from a hash table that uses linear probing to resolve collisions where we do not use a special marker to represent deleted elements. That is, we must rearrange the contents so that it appears that the removed entry was never inserted in the first place.

C-10.36 The quadratic probing strategy has a clustering problem related to the way it looks for open slots. Namely, when a collision occurs at bucket $h(k)$, it checks buckets $A[(h(k)+i^2) \bmod N]$, for $i = 1,2,\ldots,N-1$.

a. Show that $i^2 \bmod N$ will assume at most $(N+1)/2$ distinct values, for N prime, as i ranges from 1 to $N-1$. As a part of this justification, note that $i^2 \bmod N = (N-i)^2 \bmod N$ for all i.

b. A better strategy is to choose a prime N such that $N \bmod 4 = 3$ and then to check the buckets $A[(h(k) \pm i^2) \bmod N]$ as i ranges from 1 to $(N-1)/2$, alternating between plus and minus. Show that this alternate version is guaranteed to check every bucket in A.

C-10.37 Refactor our ProbeHashMap design so that the sequence of secondary probes for collision resolution can be more easily customized. Demonstrate your new framework by providing separate concrete subclasses for linear probing and quadratic probing.

C-10.38 Design a variation of binary search for performing the multimap operation find_all(k) implemented with a sorted search table that includes duplicates, and show that it runs in time $O(s+\log n)$, where n is the number of elements in the dictionary and s is the number of items with given key k.

C-10.39 Although keys in a map are distinct, the binary search algorithm can be applied in a more general setting in which an array stores possibly duplicative elements in nondecreasing order. Consider the goal of identifying the index of the *leftmost* element with key greater than or equal to given k. Does the _find_index method as given in Code Fragment 10.8 guarantee such a result? Does the _find_index method as given in Exercise R-10.21 guarantee such a result? Justify your answers.

C-10.40 Suppose we are given two sorted search tables S and T, each with n entries (with S and T being implemented with arrays). Describe an $O(\log^2 n)$-time algorithm for finding the kth smallest key in the union of the keys from S and T (assuming no duplicates).

C-10.41 Give an $O(\log n)$-time solution for the previous problem.

C-10.42 Suppose that each row of an $n \times n$ array A consists of 1's and 0's such that, in any row of A, all the 1's come before any 0's in that row. Assuming A is already in memory, describe a method running in $O(n \log n)$ time (not $O(n^2)$ time!) for counting the number of 1's in A.

C-10.43 Given a collection C of n cost-performance pairs (c, p), describe an algorithm for finding the maxima pairs of C in $O(n \log n)$ time.

C-10.44 Show that the methods above(p) and prev(p) are not actually needed to efficiently implement a map using a skip list. That is, we can implement insertions and deletions in a skip list using a strictly top-down, scan-forward approach, without ever using the above or prev methods. (Hint: In the insertion algorithm, first repeatedly flip the coin to determine the level where you should start inserting the new entry.)

C-10.45 Describe how to modify a skip-list representation so that index-based operations, such as retrieving the item at index j, can be performed in $O(\log n)$ expected time.

C-10.46 For sets S and T, the syntax S ^ T returns a new set that is the symmetric difference, that is, a set of elements that are in precisely one of S or T. This syntax is supported by the special __xor__ method. Provide an implementation of that method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class.

C-10.47 In the context of the MutableSet abstract base class, describe a concrete implementation of the __and__ method, which supports the syntax S & T for computing the intersection of two existing sets.

C-10.48 An ***inverted file*** is a critical data structure for implementing a search engine or the index of a book. Given a document D, which can be viewed as an unordered, numbered list of words, an inverted file is an ordered list of words, L, such that, for each word w in L, we store the indices of the places in D where w appears. Design an efficient algorithm for constructing L from D.

C-10.49 Python's collections module provides an OrderedDict class that is unrelated to our sorted map abstraction. An OrderedDict is a subclass of the standard hash-based dict class that retains the expected $O(1)$ performance for the primary map operations, but that also guarantees that the `__iter__` method reports items of the map according to first-in, first-out (FIFO) order. That is, the key that has been in the dictionary the longest is reported first. (The order is unaffected when the value for an existing key is overwritten.) Describe an algorithmic approach for achieving such performance.

Projects

P-10.50 Perform a comparative analysis that studies the collision rates for various hash codes for character strings, such as various polynomial hash codes for different values of the parameter a. Use a hash table to determine collisions, but only count collisions where different strings map to the same hash code (not if they map to the same location in this hash table). Test these hash codes on text files found on the Internet.

P-10.51 Perform a comparative analysis as in the previous exercise, but for 10-digit telephone numbers instead of character strings.

P-10.52 Implement an OrderedDict class, as described in Exercise C-10.49, ensuring that the primary map operations run in $O(1)$ expected time.

P-10.53 Design a Python class that implements the skip-list data structure. Use this class to create a complete implementation of the sorted map ADT.

P-10.54 Extend the previous project by providing a graphical animation of the skip-list operations. Visualize how entries move up the skip list during insertions and are linked out of the skip list during removals. Also, in a search operation, visualize the scan-forward and drop-down actions.

P-10.55 Write a spell-checker class that stores a lexicon of words, W, in a Python set, and implements a method, check(s), which performs a ***spell check*** on the string s with respect to the set of words, W. If s is in W, then the call to check(s) returns a list containing only s, as it is assumed to be spelled correctly in this case. If s is not in W, then the call to check(s) returns a list of every word in W that might be a correct spelling of s. Your program should be able to handle all the common ways that s might be a misspelling of a word in W, including swapping adjacent characters in a word, inserting a single character in between two adjacent characters in a word, deleting a single character from a word, and replacing a character in a word with another character. For an extra challenge, consider phonetic substitutions as well.

Chapter Notes

Hashing is a well-studied technique. The reader interested in further study is encouraged to explore the book by Knuth [65], as well as the book by Vitter and Chen [100]. Skip lists were introduced by Pugh [86]. Our analysis of skip lists is a simplification of a presentation given by Motwani and Raghavan [80]. For a more in-depth analysis of skip lists, please see the various research papers on skip lists that have appeared in the data structures literature [59, 81, 84]. Exercise C-10.36 was contributed by James Lee.